

PROYECTO FIN DE GRADO

TITLE:

Artificial Intelligence based models to improve quality classification of steel coils in automotive industry.

AUTOR/A: Alejandro Álvarez Castro

TITULACIÓN: Grado en Ingeniería y Sistemas de Datos

DIRECTOR/A: Joaquín Ordieres Meré

TUTOR/A: Miguel Ortega Mier

DEPARTAMENTO: Ingeniería de Organización, Administración de Empresas y Estadística

VºBº TUTOR/A

Miembros del Tribunal Calificador:

PRESIDENTE/A: Daniel Berjón Díez

TUTOR/A: Miguel Ortega Mier

SECRETARIO/A: Álvaro García Sánchez

Fecha de lectura:

Calificación:

El Secretario/La Secretaria,

“Success usually comes to those who are too busy to be looking for it.”

- Henry David Thoreau

Agradecimientos

Gracias a mis padres, por su amor incondicional, su apoyo constante y su fe en mí. A mi familia, por ser siempre una fuente de inspiración, apoyo y cariño.

A mis amigos, los nuevos y los viejos, que gracias a ellos he recorrido un camino lleno de momentos memorables.

Y por último a Joaquín y a Miguel por confiar en mí y dedicar su tiempo y esfuerzo en guiarme para la realización de este proyecto.

Resumen

El control de calidad de las bobinas de acero galvanizado es crucial debido a su amplia gama de aplicaciones y a la necesidad de mantener altos estándares en sectores industriales exigentes. Estas bobinas se utilizan en las industrias del automóvil, la construcción y la fabricación de electrodomésticos, así como en la producción de estructuras metálicas, donde la resistencia a la corrosión y la durabilidad son esenciales. Un riguroso control de calidad asegura un recubrimiento uniforme de zinc, que proporciona una protección eficaz contra la corrosión y garantiza la integridad estructural del material.

En el sector de la automoción, por ejemplo, la calidad del acero galvanizado repercute directamente en la seguridad y longevidad de los vehículos. Las bobinas defectuosas pueden provocar averías prematuras y elevados costes de mantenimiento. En la construcción, el uso de acero galvanizado de alta calidad es fundamental para garantizar la longevidad de los edificios y otras infraestructuras, reduciendo así los costes asociados a reparaciones y sustituciones. Del mismo modo, en la industria de los electrodomésticos, el aspecto y la funcionalidad del producto final dependen en gran medida de la calidad del acero utilizado. Los defectos en las bobinas pueden dar lugar a productos finales que no cumplan las normas de calidad esperadas, lo que afecta a la reputación y competitividad de la empresa.

El control de calidad también desempeña un papel vital en la eficiencia de la cadena de suministro. Garantizar que las bobinas cumplen las normas especificadas reduce el riesgo de interrupciones de la producción, optimiza el uso de materiales y minimiza los residuos. Además, la aplicación de controles de calidad rigurosos puede mejorar la sostenibilidad medioambiental de la producción al reducir la necesidad de reprocesado y los residuos.

En resumen, el control de calidad de las bobinas de acero galvanizado es esencial para garantizar que los productos finales cumplan las expectativas de rendimiento y durabilidad, manteniendo la seguridad, funcionalidad y eficiencia en sus diversas aplicaciones industriales.

Aunque el control de calidad implica una cantidad significativa de tecnología, como medidores de rayos X, cámaras lineales de alta definición etc., la decisión final sobre la calidad implica principalmente reglas de decisión, que resultan ineficaces para procesar la diversidad de situaciones y el elevado volumen de producción. Por lo tanto, esta parte del control de calidad es gestionada principalmente por operadores humanos, lo que no sólo es laborioso, sino también propenso al error humano.

Para abordar este problema, se han desarrollado modelos basados en IA, los cuales surgen como una solución prometedora para aumentar la precisión y la eficacia en la clasificación de la calidad de estos materiales, utilizando Redes Neuronales Convolucionales (CNN) para clasificar las bobinas de acero según el patrón de espesor del recubrimiento de zinc. Comparamos este enfoque de aprendizaje profundo con un método de Aprendizaje Profundo por Transferencia (DTL), utilizando un modelo YOLOv8 preentrenado específicamente para la tarea de clasificación. Además, se evalúa una técnica de Aprendizaje Contrastivo (CL) parcialmente supervisada para mejorar aún más el rendimiento de la clasificación.

El alcance de la investigación incluye la implementación de los modelos en un entorno de producción utilizando herramientas de gestión de flujos de trabajo como NiFi™ y el establecimiento de un sistema para el reentrenamiento automático cuando el rendimiento del modelo disminuya.

Al integrar estos modelos de IA en el flujo de trabajo de control de calidad, este proyecto tiene como objetivo mejorar significativamente el proceso de toma de decisiones, ejemplificando

una colaboración efectiva entre humanos y máquinas en la Industria 5.0. El marco propuesto proporciona una mejora continua, estableciendo un nuevo estándar en los procesos de control de calidad de la industria automotriz.

Abstract

Quality control in galvanized steel coils is crucial due to their wide range of applications and the need to maintain high standards in demanding industrial sectors. These coils are used in the automotive, construction, and appliance manufacturing industries, as well as in the production of metal structures, where corrosion resistance and durability are essential. Rigorous quality control ensures a uniform zinc coating, providing effective corrosion protection and guaranteeing the structural integrity of the material.

In the automotive sector, for instance, the quality of galvanized steel directly impacts vehicle safety and longevity. Defective coils can lead to premature failures and high maintenance costs. In construction, the use of high-quality galvanized steel is fundamental to ensure the longevity of buildings and other infrastructures, thus reducing costs associated with repairs and replacements. Similarly, in the appliance industry, the appearance and functionality of the final product heavily depend on the quality of the steel used. Defects in coils can result in end products that do not meet the expected quality standards, affecting the company's reputation and competitiveness.

Quality control also plays a vital role in the efficiency of the supply chain. Ensuring that coils meet specified standards reduces the risk of production disruptions, optimizes material usage, and minimizes waste. Furthermore, implementing rigorous quality controls can enhance the environmental sustainability of production by reducing the need for reprocessing and waste.

In summary, quality control in galvanized steel coils is essential to ensure that final products meet performance and durability expectations, maintaining safety, functionality, and efficiency in their various industrial applications.

Although the quality control involves a significant amount of technology, such as X-ray gauges, high definition linear cameras, advanced light systems, high speed image processing buffers, etc., the final decision about the quality involves mainly decision rules, which are inefficient to process the diversity of situations and the high volume of production. Therefore, this part of the quality control is mainly handled by human operators, which is not only laborious but also prone to human error. In this context, integrating Artificial Intelligence (AI) models emerges as a promising solution to increase the accuracy and efficiency in the quality classification of these materials.

To tackle this problem, AI-based models were developed using Convolutional Neural Networks (CNNs) to classify steel coils based on the pattern of zinc coating thickness. We compare this Deep Learning approach with a Deep Transfer Learning (DTL) method, utilizing a fine-tuned YOLOv8 model, specifically pretrained for classification task. Additionally, a partially unsupervised Contrastive Learning (CL) technique is evaluated to further enhance classification performance. An ensemble model integrating these approaches is constructed to improve decision-making accuracy and reliability.

The scope of the research includes implementing the models within a production environment using workflow management tools like NiFi™ and establishing a system for automatic retraining when model performance degrades.

By integrating these AI models into the quality control workflow, this project aims to significantly enhance the decision-making process, exemplifying effective human-machine collaboration in Industry 5.0. The proposed framework not only automates quality assessment but also provides continuous improvement, setting a new standard in the automotive industry's quality control processes



Plantilla en LaTeX acorde con la “Guía del alumno Trabajo Fin de Grado del Grado de Ingeniería y Sistemas de Datos (actualizado octubre 2023)” by Miguel Ortega-Mier is licensed under a Creative Commons Attribution

4.0 International License.

Table of Contents

Agradecimientos	iv
Resumen	vi
Abstract	viii
List of Tables	xiv
List of Figures	xix
List of Abbreviations	xx
1 Introduction	1
1.1 Motivation and Context	1
1.2 Objectives	4
1.3 Methodology	5
1.4 Structure of the document	7
2 State of the Art	9
2.1 Artificial Intelligence	9
2.2 Deep Learning	9
2.2.1 Convolutional Neural Networks	10
2.3 Python	12
2.3.1 Tensorflow and Keras	13
2.3.2 PyTorch	13
2.4 Deep Transfer Learning	14
2.5 Contrastive Learning	18
2.6 Evidently AI	19
2.7 Apache NiFi and Airflow	20
3 Design specifications and requirements	23

3.1	Flow Diagram	23
3.2	Requirements	23
4	Description of the proposed solution	25
4.1	Dataset	25
4.2	Evaluation metrics	26
4.3	Deep Learning	28
4.4	Deep Transfer Learning	33
4.4.1	Dataset processing	33
4.4.2	Cross validation	34
4.4.3	DTL Systematic Study	35
4.5	Contrastive Learning	37
4.5.1	Image Pairs Generation	37
4.5.2	Reference Images	38
4.5.3	Classification method and Study	42
5	Results	47
5.1	Deep Learning	47
5.2	Deep Transfer Learning	62
5.3	Contrastive Learning	69
6	Deployment	79
6.1	Coil Assessment Workflow	79
6.2	Model Score Workflow	80
6.3	Model Retraining Workflow	81
7	Impacts of the Project	83
7.1	Social impact	83
7.2	Economic impact	83
7.3	Technological impact	84
7.4	Contribution to Sustainable Development Goals (SDG)	84

8	Planning and Budget	87
8.1	Planning	87
8.2	Budget	88
9	Conclusions and Future Works	91
9.1	Conclusions	91
9.2	Future Works	92
9.3	Personal Learning	92

List of Tables

5.1	Shapiro-Wilk Test's Results	51
5.2	Levene's Test Results	51
5.3	T-Test Results for Dropout Rate	52
5.4	T-Test Results for Number of Filters	53
5.5	T-Test Results for Kernel Width	53
5.6	DL Metrics Test Set	54
5.7	DL Metrics Gold Standard Set [Own Image]	55
5.8	DTL Metrics Test Set	62
5.9	DTL Metrics Gold Standard Set [Own Image]	63
5.10	CL Metrics Test Set	69
5.11	CL Metrics Gold Standard Set [Own Image]	70
5.12	CL Metrics Gold Standard Set Square Images [Own Image]	74
8.1	Materials and Costs	88
8.2	Personal Costs	89
8.3	Total Costs	89

List of Figures

1.1	Inspection System [28]	1
1.2	Zinc Coating Bath [28]	2
1.3	Hot-Dip-Galvanizing-Line [28]	3
1.4	Zinc Coat [28]	3
1.5	Methodology CRISP-DM [89]	6
2.1	AI / ML / DL [Own Image]	10
2.2	CNN Layers [Own Image]	11
2.3	Transfer Learning Diagram [Own Image]	15
2.4	DTL Types [58]	16
2.5	Pretrained YOLO Models [80]	16
2.6	YOLO Family Comparison [81]	17
2.7	Siamese Model [Own Image]	18
3.1	System Flow Diagram [Own image]	23
3.2	YOLOV8 Architecture [Own image]	24
4.1	Zinc Measures	26
4.2	Explained Confusion Matrix [18]	27
4.3	Example Model [Own Image]	31
4.4	Complete System [Own Image]	32
4.5	Arguments [Own Image]	32
4.6	Image Processing [Own Image]	34
4.7	Example Config DTL [Own Image]	36
4.8	Complete System DTL [Own Image]	37
4.9	Single Cluster [Own Image]	39
4.10	Clusters [Own Image]	40
4.11	Subclustering on OK Cluster [Own Image]	41
4.12	KMeans Reference Images of OK Cluster [Own Image]	41
4.13	CL Classifier [Own Image]	43

4.14	CL YAML [Own Image]	44
4.15	CL System [Own Image]	45
5.1	Architecture Comparaison 1 [Own Image]	48
5.2	Architecture Comparaison 2 [Own Image]	49
5.3	Compare 20 configurations [Own Image]	50
5.4	Q-Q Plot [Own Image]	51
5.5	Dropout Rate Scatter/Box Plot [Own Image]	52
5.6	Number of Filters Scatter/Box Plot [Own Image]	52
5.7	Kernel Width Scatter/Box Plot [Own Image]	53
5.8	DL Mean Confusion Matrix Test Set [Own Image]	54
5.9	Confusion Matrix Gold Standard Set [Own Image]	55
5.10	Metrics per Class Gold Standard Set [Own Image]	55
5.11	Compare Configs [Own Image]	56
5.12	Pooling Type [Own Image]	56
5.13	Normalization Type [Own Image]	57
5.14	Dropout Type [Own Image]	57
5.15	Dense Layers Number [Own Image]	58
5.16	Dense Precision [Own Image]	58
5.17	Dense Recall [Own Image]	58
5.18	Dense ROC/AUC [Own Image]	59
5.19	Convolutions Accuracy [Own Image]	59
5.20	Convolutions Precision [Own Image]	59
5.21	Convolutions Recall [Own Image]	59
5.22	Convolutions F1 Score [Own Image]	59
5.23	Convolutions ROC/AUC [Own Image]	60
5.24	Compare Conv/Dense [Own Image]	61
5.25	DTL Mean Confusion Matrix Test Set [Own Image]	63
5.26	DTL Confusion Matrix Gold Standard Set [Own Image]	63
5.27	DTL Metrics per Class Gold Standard Set [Own Image]	64

5.28	Pretrained Model Accuracy [Own Image]	65
5.29	Pretrained Model Precision [Own Image]	65
5.30	Pretrained Model Recall [Own Image]	65
5.31	Pretrained Model F1 Score [Own Image]	65
5.32	Pretrained Model ROC/AUC [Own Image]	65
5.33	Frozen Layers Accuracy [Own Image]	66
5.34	Frozen Layers Precision [Own Image]	66
5.35	Frozen Layers Recall [Own Image]	66
5.36	Frozen Layers F1 Score [Own Image]	66
5.37	Frozen Layers ROC/AUC [Own Image]	67
5.38	Epochs Accuracy [Own Image]	67
5.39	Epochs F1 Score [Own Image]	67
5.40	Frozen Layers / Pretrained Model [Own Image]	68
5.41	CL Gold Confusion Matrix [Own Image]	70
5.42	CL Metrics per Class [Own Image]	70
5.43	CL Normalization Type [Own Image]	71
5.44	CL Pooling Type [Own Image]	71
5.45	CL Dropout Type [Own Image]	72
5.46	CL Convolutions F1 Score [Own Image]	72
5.47	CL Dense F1 Score [Own Image]	73
5.48	CL Gold Confusion Matrix Square Images [Own Image]	74
5.49	CL Metrics per Class Square Images [Own Image]	74
5.50	CL Normalization Type Square Images [Own Image]	75
5.51	CL Pooling Type Square Images [Own Image]	75
5.52	CL Dropout Type Square Images [Own Image]	76
5.53	CL Dense F1 Score Square Images [Own Image]	76
5.54	CL Convolutions F1 Score Square Images [Own Image]	77
6.1	Main Workflows [Own Image]	79
6.2	Assess Coils Workflow [Own Image]	80

6.3	Model Score Workflow [Own Image]	81
6.4	Retrain Model Workflow [Own Image]	81
7.1	SDG 8 [54]	85
7.2	SDG 9 [55]	85
7.3	SDG 4 [56]	85
8.1	Gantt Chart [Own Image]	87

List of Abbreviations

AI Artificial Intelligence

ML Machine Learning

DL Deep Learning

DTL Deep Transfer Learning

CL Contrastive Learning

CRISP/DM Cross Industry Standard Process for Data Mining

SOTA State of the Art

CPU Central Processing Unit

GPU Graphics Processing Unit

1 Introduction

1.1 Motivation and Context

Artificial Intelligence (AI) has become a transformative force in various industries, driving innovation and efficiency. AI encompasses a range of technologies, including machine learning, neural networks, and deep learning, which enable machines to mimic human intelligence and perform tasks such as pattern recognition, decision-making, and predictive analytics [68]. In recent years, AI has been increasingly integrated into industrial processes to enhance productivity and quality control. [41]

Quality control is particularly critical in the automotive industry, where stringent processes ensure the production of high-quality components and vehicles. This involves systematic inspection and testing of materials, parts, and final products to identify defects and ensure compliance with industry standards [12]. The use of advanced technologies, such as X-ray gauges and automated inspection systems, has significantly improved the precision and efficiency of these processes [77]. Consequently, integrating AI into quality control processes offers substantial benefits, including reduced inspection times and the ability to handle large volumes of data. [92]

Traditionally, the thickness of the zinc coating on steel coils, which are used in car body panels, was controlled at a single point along the width of the coil. However, given the importance of this measurement for the quality and durability of the final product, it is now measured at multiple points throughout the width of the coil (See Figure 1.1).



Figure 1.1: Inspection System [28]

This comprehensive approach ensures that the entire coil complies with the stringent quality requirements specified by the clients, addressing the potential variability in the coating process. [19]

The galvanized steel coil is produced by applying a protective zinc coating to the steel, significantly increasing its resistance to corrosion and extending its service life [85]. The hot dip galvanizing process, in particular, offers numerous benefits, including excellent durability, exceptional strength, and superior weather resistance [25]. These attributes make galvanized steel coil a preferred material in various industries such as construction, automotive, and manufacturing [3]. Its widespread use is attributed to its ability to withstand harsh environmental conditions while maintaining structural integrity [39].

In comparison to different methods to galvanize steel, in this application the hot-dip method is used. Hot-dip galvanizing involves immersing steel in a bath of molten zinc (See Figures 1.2, 1.3). Zinc coats steel, even reaching hard-to-access areas, forming a protective layer with a thickness ranging from 70 to 150 micrometers [5]. This technique is renowned for its durability and offers superior protection against corrosion [4]. However, it is less attractive for the eyes as a result of the formation of nubs and thickening during the solidification of the zinc. Despite this, hot-dip galvanizing is considered the most robust and long-lasting method of protecting steel [1], making it highly suitable for industrial applications where durability is paramount.



Figure 1.2: Zinc Coating Bath [28]

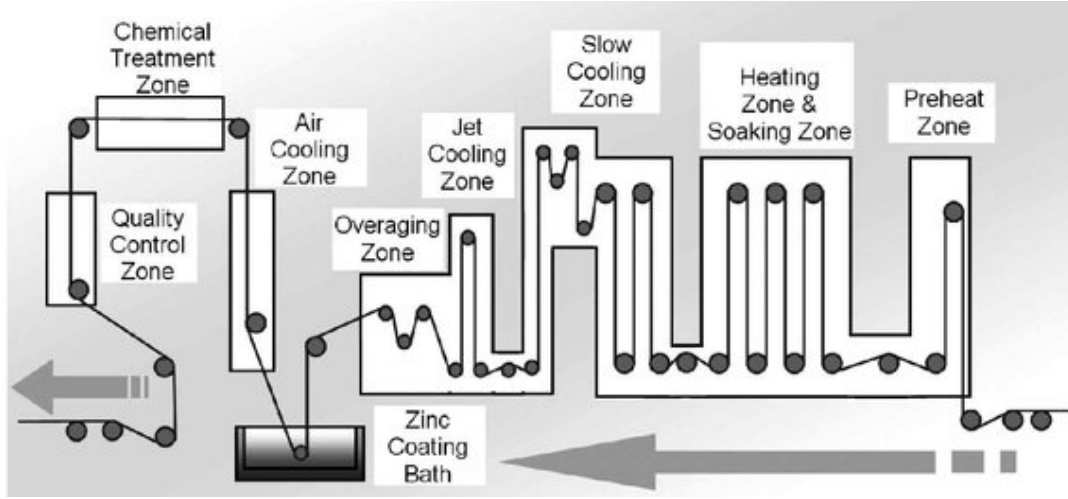


Figure 1.3: Hot-Dip-Galvanizing-Line [28]

The thickness of the zinc coating is a critical factor that influences the performance and longevity of the galvanized steel coil (See Figure 1.4), but also the cost. Thicker zinc coatings generally offer superior corrosion resistance because the zinc acts as a sacrificial layer, protecting the underlying steel from moisture and other corrosive elements. However, excessively thick coatings can negatively affect the appearance and formability of steel, necessitating careful consideration during production to achieve the optimal balance between protection and usability.

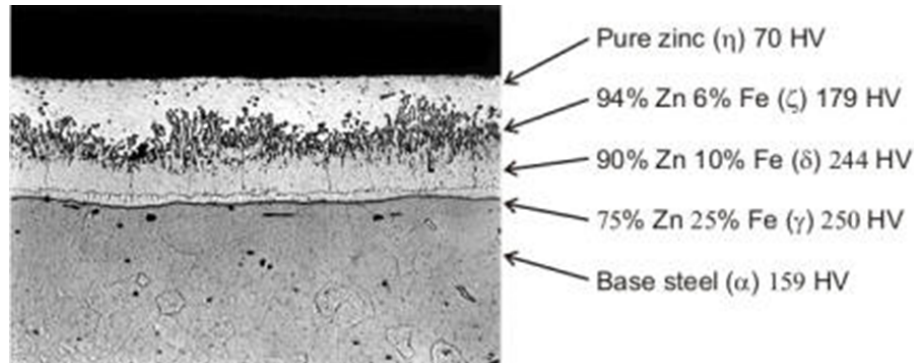


Figure 1.4: Zinc Coat [28]

Current inspection systems for measuring zinc thickness layers in galvanized steel coils primarily rely on rule-based decision-making frameworks. These systems, while functional to some extent, face significant limitations in terms of accuracy and efficiency. Rule-based systems operate by predefined logic that categorizes coils based on fixed criteria. However, this approach has several drawbacks, particularly in the high-stakes environment of coil production where the rate of production is rapid and the need for precise measurements is critical, conducting to an effective classification of 10% of production based on rule based systems decision.

One of the main issues with rule-based systems is their low classification accuracy. This low accuracy often necessitates human intervention, particularly for coils that fall into ambiguous categories. The requirement for human decision-making in the final stages of production can be both stressful and prone to errors, especially under high-pressure conditions. Additionally, the reliance on human judgment can introduce variability and inconsistency into the quality control process, further complicating the assurance of uniform product standards.

AI systems, specifically those utilizing machine learning algorithms, offer a promising solution

to these challenges. Machine learning can enhance the accuracy and efficiency of zinc layer thickness measurement by learning from vast amounts of data and identifying patterns that are not easily discernible through rule-based systems. Studies have shown that AI can significantly improve the classification accuracy of quality control systems. For instance, Zhang et al. (2018) [52] demonstrated that machine learning algorithms could outperform traditional methods in classifying surface defects in steel production, which is closely related to the challenges faced in zinc thickness measurement.

AI systems can also reduce the dependency on human intervention by providing more reliable and consistent decision-making capabilities. With machine learning, the system can continuously improve its accuracy by learning from new data, thereby reducing the need for constant human oversight. This aspect of AI is particularly beneficial in high-speed production environments where the quick and accurate classification of coils is essential. Research by Liu et al. (2020) [46] highlights the advantages of AI in real-time monitoring and decision-making in industrial processes, emphasizing its potential to minimize human error and increase operational efficiency.

Moreover, AI systems can handle the high volume and velocity of data generated in coil production more effectively than rule-based systems. By leveraging deep learning techniques, these systems can analyze complex datasets and provide insights that facilitate better quality control. The use of convolutional neural networks (CNNs), for instance, has been shown to improve the detection of surface defects and irregularities in steel coils, as noted by He et al. (2019) [23]. These advanced algorithms can process images and data in real-time, offering a level of precision and speed unattainable by traditional rule-based systems.

In conclusion, the integration of AI systems into the inspection process for zinc thickness layers in galvanized steel coils can address the limitations of current rule-based systems. By improving classification accuracy, reducing human intervention, and enhancing data processing capabilities, AI can significantly enhance the efficiency and reliability of quality control in coil production. The scientific evidence supports the adoption of AI technologies, demonstrating their potential to transform industrial quality assurance practices.

To tackle this issue, several research questions arise: How can AI be effectively integrated into the quality control process for steel coils? What are the most suitable AI models for classifying zinc coating defects?

To address these questions, it is crucial to clearly define what is considered a defect in the industrial context of zinc coatings. In this industry, a defect does not simply refer to the absence of zinc but to any deviation from the zinc thickness range specified by the client.

Each client sets a range of zinc thickness that they consider acceptable for their products. This range is based on the final application of the product and can vary significantly between different clients. Tolerance refers to the permissible variation in zinc thickness relative to the specified nominal value. A zinc coating is considered defective if the thickness falls outside this tolerance range, whether it is too thin or too thick.

1.2 Objectives

The primary goal of this project is to evaluate how AI-based technology can enhance decision-making processes in industrial settings, specifically addressing the issue of zinc-coated galvanized coils used across various industries, including the automotive sector. This overarching objective can be further divided into several specific objectives, such as:

The specific objectives to achieve the main one include:

- Developing convolutional neural networks (CNNs) for coil classification. The goal is to automate the simulations of different models as much as possible. This requires creating a system capable of being quite general in constructing models with any architecture and configuration specifications.
- Comparison of the CNN approach with a Deep Transfer Learning (DTL) method using a fine-tuned YOLOv8 model.
- Exploring a partially unsupervised Contrastive Learning (CL) approach to complement the CNN and DTL methods.
- Creating an ensemble model to improve decision making and estimate confidence levels.
- Implementing these models in a production environment using workflow management systems like NiFi™.
- Establishing a framework for automatic retraining of models when performance declines.

1.3 Methodology

To achieve these objectives, the project will adopt the Cross Industry Standard Process for Data Mining (CRISP/DM) [71], a widely used framework for data mining projects. This methodology has several stages: (See Figure 1.5)

1. Business Understanding.
Determine project's objectives and define a plan for the following phases.
2. Data Understanding:
Collect the initial data, explore and describe it.
3. Data Preparation.
Select, clean and construct data needed for the project.
4. Modeling.
Select modeling techniques and build models. In this case, Deep Learning, Deep Transfer Learning and partially unsupervised Contrastive Learning. For modeling, it is essential to create a very general system capable of creating models from any specifications that may arrive.
5. Evaluation.
For the various machine learning's approaches, a similar evaluation structure will be followed. Each model is evaluated using the cross-validation technique [8]. This technique ensures that all samples are used in both training and testing. In this case, cross validation is carried out by creating 5 different splits. This 5 splits involves that data is split in 5 groups with the 20% of data each. In consequence, the 80% of data is used for training, while the 20% is used for test.
Cross-validation is performed 10 times to achieve statistical representativeness of each model's performance. Each cross-validation produces 5 models and this process is repeated 10 times, so it results in 50 models with the same parameters.
6. Deployment.
Deploy system to classify galvanized steel coils and final report with results.

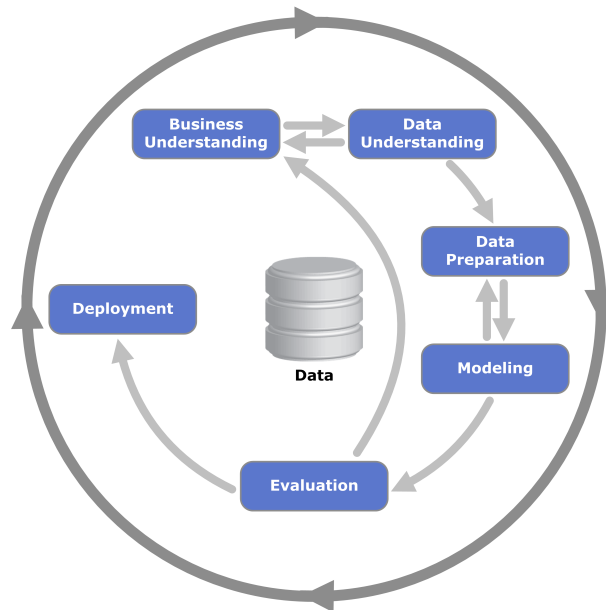


Figure 1.5: Methodology CRISP-DM [89]

The selected CRISP-DM methodology, widely adopted for structuring data mining projects, includes a crucial step known as Modeling. In this work, the Modeling step was executed using an agile methodology, specifically through a sprint-oriented model, which allowed for iterative and incremental development of functionalities.

Implementing the agile methodology within the CRISP-DM framework brings several advantages and implications. Agile methods, characterized by their iterative cycles, known as sprints, enable rapid development and continuous improvement. Each sprint typically lasts for a few weeks, during which a specific set of functionalities are developed, tested, and refined. This approach fosters flexibility and adaptability, crucial elements in the fast-evolving field of AI and data mining.

Scientific literature supports the integration of agile methodologies in data science projects. For example, Thomas et al. (2019) [78] highlight that agile practices can enhance responsiveness to changing requirements and improve stakeholder engagement through regular feedback loops. This iterative process ensures that the model evolves based on real-world performance and stakeholder input, leading to more robust and reliable outcomes.

In the context of this project, using an agile, sprint-oriented model for the Modeling step allowed for the development of AI functionalities that are closely aligned with the specific needs and constraints of the industrial environment dealing with zinc-coated galvanized coils. The iterative nature of agile development facilitated the identification and resolution of issues early in the process, minimizing the risk of significant setbacks later on. Additionally, it enabled the incorporation of domain-specific insights and continuous refinement of the model based on empirical evidence and practical feedback.

Moreover, the sprint-oriented model supports rapid prototyping and validation of AI models. As noted by Beck et al. (2020) [6], agile methodologies promote the frequent delivery of functional software, which is critical for validating hypotheses and ensuring the model meets the expected performance criteria. This approach also allows for parallel exploration of different modeling techniques and algorithms, accelerating the discovery of the most effective solutions for the problem at hand.

The implications of using an agile methodology within the CRISP-DM framework are profound. It enhances collaboration among cross-functional teams, including data scientists, domain experts, and industrial stakeholders, ensuring that the developed models are not only technically sound but also practically relevant. It also supports a culture of continuous learning and improvement, essential for maintaining the competitiveness and innovation of industrial processes.

In summary, the integration of an agile, sprint-oriented approach within the Modeling step of the CRISP-DM methodology brings significant benefits to the project. It facilitates iterative development, rapid prototyping, and continuous refinement of AI models, ensuring they are robust, relevant, and aligned with the dynamic requirements of the industrial environment. The scientific literature corroborates these advantages, highlighting improved responsiveness, stakeholder engagement, and practical relevance as key outcomes of this methodological integration.

Importantly, this project aligns with the principles of Industry 5.0 [90], which focuses on the collaboration between humans and machines to create more efficient, sustainable, and resilient industrial systems. By integrating AI into quality control processes, the project not only improves the decision-making process but also exemplifies the human-machine integration central to Industry 5.0. This integration ensures that human expertise is complemented by advanced AI capabilities, leading to smarter, more adaptive industrial operations [53].

1.4 Structure of the document

This document is organized into nine chapters, each addressing fundamental aspects of the development and application of artificial intelligence models for the quality classification of steel coils in the automotive industry. First, the chapter 1 presents the motivation, objectives, and methodology used, highlighting the importance of artificial intelligence in automotive quality control.

Next, in the chapter 2, the current literature and technologies are reviewed, including deep learning, transfer learning, and contrastive learning, with a focus on their industrial applications. Subsequently, in chapter 3, the system specifications, hardware and software needs, and success criteria are detailed, establishing the foundation for the project's design and development.

The chapter 4 describes the selection and preparation of the data set, the evaluation metrics, and the architecture of the implemented models. Following that, in chapter 5, the analyses and performance comparisons of the models are presented, supported by statistical analysis and graphs.

In the chapter 6, the process of implementing the models in a production environment is explained, using workflow management tools like NiFi™. Next, the chapter 7 analyzes the social, economic, and technological repercussions of the project, as well as its contribution to the Sustainable Development Goals (SDGs).

Chapter 8 details the work plan and estimated budget, including a Gantt chart and a description of the necessary resources. Finally, chapter 9 summarizes the findings of the project and propose lines of research and development for future improvements.

2 State of the Art

2.1 Artificial Intelligence

Artificial intelligence (AI) [70] has significantly evolved from a theoretical concept to a foundational technology driving many modern applications. The field began in the 1950s with the Dartmouth Conference, where the term artificial intelligence was coined by pioneers like John McCarthy and Marvin Minsky. [49]

The 1970s and 1980s saw a period of disillusionment, known as the first “AI winter”, due to technical limitations. However, the development of expert systems in the 1980s revived interest. The 1990s and 2000s marked a resurgence with advancements in machine learning, culminating in IBM’s Deep Blue defeating chess champion Garry Kasparov in 1997. [29]

Nowadays, AI is starting to be a cornerstone of modern technological advancement. It is already playing a significant role in various industries. Its ability to process large volumes of data, learn from patterns, and make informed decisions positions it as an invaluable tool for quality control. In the automotive industry, where precision and reliability are paramount, AI can significantly reduce the dependence on human inspection, thus addressing the limitations and challenges associated with manual processes [44].

The importance of AI in quality control cannot be overstated. As we move into the future, its role is expected to grow exponentially, driven by continuous advancements in machine learning algorithms, improved computational power, and the increasing availability of big data. By implementing AI-driven solutions, we not only improve the current processes but also pave the way for future innovations that can further optimise production lines and ensure higher standards of quality [73].

In summary, this section provides an in-depth exploration of the current state of AI technology, its applications in quality control, and its specific relevance to our project. We will discuss several key technologies, including deep learning, deep transfer learning, contrastive learning, and ensemble models. By understanding these capabilities and their future potential, we can better appreciate AI’s importance in revolutionising the quality control processes within the automotive industry.

2.2 Deep Learning

Deep learning, a subset of machine learning (See Figure 2.1) [7] inspired by the structure and function of the human brain’s neural networks, has emerged as a transformative force in various industrial applications. At its core, deep learning involves training artificial neural networks with large amounts of data to recognize patterns and make predictions or decisions without explicit programming [40]. (See Figure 2.1)

Deep learning algorithms are characterized by their ability to automatically learn representations of data through the use of multiple layers of interconnected nodes (neurons). These layers enable the network to progressively extract higher-level features from raw input data, leading to hierarchical representations that capture complex relationships within the data. This hierarchical feature extraction distinguishes deep learning from traditional machine learning approaches [9].

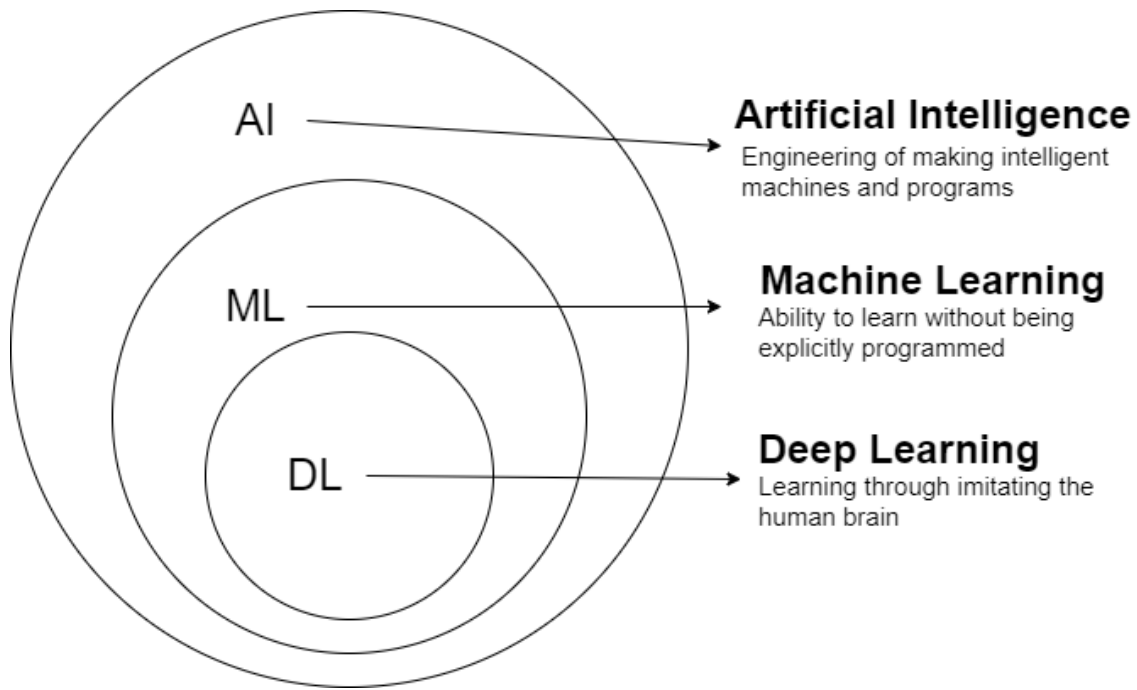


Figure 2.1: AI / ML / DL [Own Image]

In recent years, deep learning has gained immense popularity and recognition due to its remarkable success in various domains. Its ability to handle unstructured data such as images, text, and audio has led to breakthroughs in fields like computer vision, natural language processing, and speech recognition. In the industrial sector, where vast amounts of data are generated from sensors, machines, and production processes, deep learning offers unprecedented opportunities for automation, optimization, and predictive analytics [20] [45].

2.2.1 Convolutional Neural Networks

Convolutional Neural Networks (hereinafter CNNs) are a specific type of deep learning model that have shown exceptional performance in tasks involving image and spatial data. CNNs are designed to automatically and adaptively learn spatial hierarchies of features through the use of convolutional layers, normalization layers, pooling layers, dropout layers and fully connected layers [65]. (See Figure 2.2)

- Convolutional layers:
These layers apply convolution operations to the input data, capturing local patterns such as edges, textures, and shapes. This enables CNNs to learn spatial hierarchies from simple to complex features.
 - Kernel width:
The size of the kernel determines the region of the image considered during convolution. Small filters capture local features, while larger filters can capture more global and complex features of the image.
 - Number of filters:
Specifies the number of filters (or kernels) used in the convolution operation.
- Normalization layers:
Normalization layers in convolutional neural networks (CNNs) are used to standardize

inputs to a layer. This helps in stabilizing, accelerating the training process, and obtaining better results.

- **Pooling layers:**
These layers reduce the spatial dimensions of the data, which helps in decreasing the computational load and controlling overfitting by summarizing the presence of features in patches of the feature map. It can be an average pooling, taking the mean value or max pooling, taking the maximum value of the selected ones.
- **Dropout layers:**
Dropout layers are a regularization technique that is used to prevent overfitting in CNNs. During training, dropout randomly sets a fraction of input units to zero at each update step. This prevents the network from becoming too dependent on specific neurons, promoting a more robust learning process.
 - **Dropout Rate:**
This rate determines the fraction of neurons to be dropped out.
- **Fully connected or Dense layers:**
These layers integrate the learned features to make final predictions or classifications, allowing the network to make informed decisions based on the learned data.

CNNs are particularly effective for image-based applications [69], making them ideal for tasks such as object detection, facial recognition, and image classification.

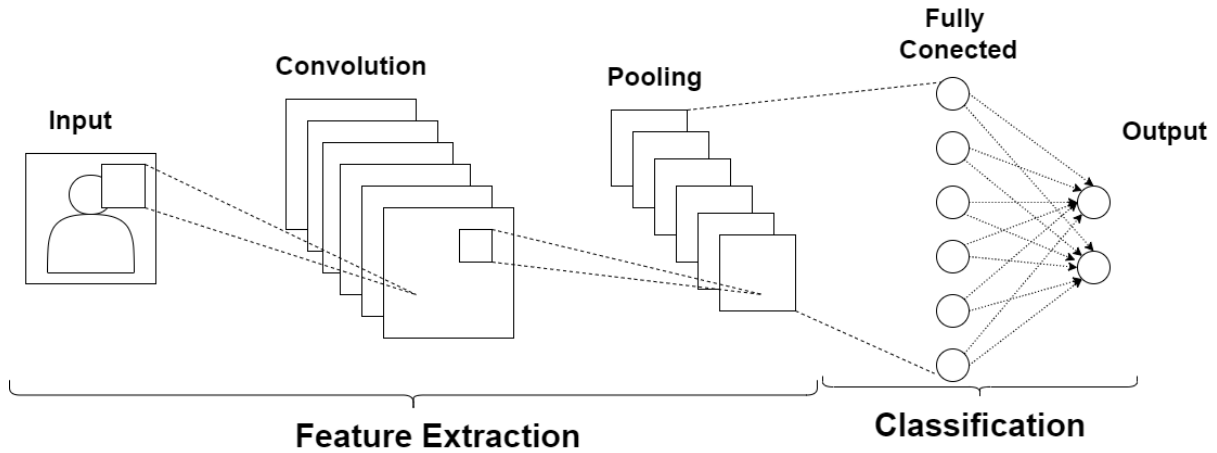


Figure 2.2: CNN Layers [Own Image]

To gain a comprehensive understanding of the role of CNNs in industrial applications, it is crucial to explore their inherent advantages and disadvantages, which are the following [43]:

- **Advantages:**
 - **Reducing Computation Compared to Traditional Neural Networks:** CNNs significantly reduce computational burden compared to traditional neural networks. This reduction is achieved through the convolutional layers that handle the feature extraction process efficiently, minimizing the number of parameters and operations required. [33]
 - **Excellence in Image Classification:** CNNs are particularly well-suited for image classification tasks. Their ability to automatically and adaptively learn spatial hierarchies

of features makes them highly effective at identifying objects and patterns within images. [69]

- Scalability: Deep learning models can be scaled to handle large datasets, continuously improving their performance as more data becomes available. [79]
- Disadvantages:
 - High Data Requirements: CNNs require large amounts of data to perform well. Due to the millions of parameters within a CNN, a substantial dataset is necessary to adequately train the model. Insufficient data can lead to poor performance and ineffective learning. [17]
 - Non-Expressive Logic and Learning: CNNs, like other deep learning models, are often considered "black boxes" due to their non-transparent decision-making processes. This lack of interpretability can be a drawback in applications where understanding the model's logic is crucial. [86]
 - Dependence on Initial Parameter Tuning: The performance of CNNs heavily depends on the initial parameter settings. Achieving optimal results requires careful tuning of hyperparameters, which can be a complex and time-consuming process that necessitates specialist knowledge. [88]

In the industrial sector, advances in deep learning technology have paved the way for innovative solutions in defect detection and quality control processes. This kind of technology is already being implemented for the purpose of detecting defects in surfaces, which addresses the crucial need to maintain and improve the quality of industrial products.

One project in this domain is the 'Edge Intelligence with Light Weight CNN Model for Surface Defect Detection in Manufacturing Industry' by Rani, D Shobha; Burra, Lakshmi Ramani; Kalyani, G; and Rao, B Narendra Kumar. [63] This project addresses the challenges associated with the identification of surface defects, particularly in environments affected by various factors such as reflection, radiance, light, and material properties.

This project showcases the potential of deep learning technology, particularly lightweight CNN models, in revolutionizing defect detection processes in the manufacturing industry. Using edge intelligence and advanced machine learning techniques, manufacturers can improve product quality, reduce waste, and optimize production efficiency.

Given all this information, the utilization of robust frameworks becomes imperative for effective deployment of CNNs in practical applications.

2.3 Python

Python is a versatile, high-level programming language. It has become a cornerstone in modern software development due to its simplicity, readability, and vast ecosystem. Designed with an emphasis on code readability and maintainability, Python's syntax is clear and intuitive, which significantly reduces the learning curve for new programmers while enhancing productivity for experienced developers. [66]

In the realm of data science and machine learning, Python is unparalleled, largely due to libraries such as NumPy for numerical computing, pandas for data manipulation, scikit-learn for machine learning, and TensorFlow, Keras, and PyTorch for deep learning. These libraries offer robust

and efficient tools for data analysis, statistical modeling, and building complex machine learning models, cementing Python’s status as the de facto language for data-driven fields. [83]

Python’s design philosophy promotes code readability and maintainability, which is further supported by its large and active community. This community-driven development ensures a wealth of resources, from extensive documentation and tutorials to a plethora of online forums and conferences, making Python a continuously evolving and improving language.

2.3.1 Tensorflow and Keras

TensorFlow [15], an open source machine learning library developed by Google Brain, provides tools for building and training deep learning models at scale. Known for its flexibility and scalability, TensorFlow supports a wide range of machine learning tasks and is widely used in both research and industry to develop complex deep learning models.

Keras [11], on the other hand, is a high-level neural network API that simplifies the process of prototyping and experimenting with deep learning architectures. With its user-friendly interface, Keras enables rapid development and iteration of deep learning models. It acts as an abstraction layer over TensorFlow, making it easier to build and experiment with neural networks. Together, these libraries offer a comprehensive ecosystem for developing state-of-the-art deep learning solutions.

TensorFlow’s comprehensive suite of tools facilitates the development of simple and complex neural network models. Its flexibility allows for fine-tuning and customization to meet specific project needs. In addition, TensorFlow’s robust community support and extensive documentation make it a valuable resource for developers at all levels.

Keras improves TensorFlow capabilities by providing a more intuitive and accessible interface. It streamlines the process of building neural networks, allowing researchers to focus on experimentation and innovation without being influenced by the complexities of coding. Keras is particularly useful for rapid prototyping, enabling quick iterations and adjustments to model architectures.

2.3.2 PyTorch

PyTorch [2], an open source deep learning framework developed by Facebook’s AI Research lab (FAIR), has gained significant traction in the machine learning community for its dynamic computation graph and intuitive interface. Its dynamic nature allows for more transparent and debuggable model development, facilitating a seamless integration of deep learning into the Python ecosystem. [64]

The framework also offers a comprehensive suite of tools for developing and deploying machine learning models. Its extensive support for GPU acceleration, automatic differentiation via the Autograd module, and robust libraries such as TorchVision for computer vision and TorchText for natural language processing extend its utility across a wide range of domains. Additionally, PyTorch’s strong community support and extensive documentation contribute to its growing popularity and adoption. [59]

By providing a versatile and user-friendly platform, PyTorch empowers researchers and developers to innovate and push the boundaries of deep learning. Its ability to handle dynamic, iterative

processes with ease makes it an ideal choice for rapid prototyping and experimentation, fostering a conducive environment for cutting-edge research and application development.

In summary, PyTorch's flexibility and strong integration with the Python ecosystem position it as a powerful tool for developing state-of-the-art deep learning solutions. Its ongoing development and community-driven enhancements ensure that it remains at the forefront of machine learning research and practice.

2.4 Deep Transfer Learning

Transfer learning is a sophisticated machine learning technique where a model pre-trained on one task is repurposed and fine-tuned for another, typically related, task. This method exploits the knowledge acquired during the initial training phase, significantly improving the performance and efficiency of the new task. By utilizing a pre-trained model, transfer learning can often bypass the need for large amounts of data and extensive computational resources that would otherwise be necessary for training a model from scratch. [75]

The process of transfer learning begins with pretraining a model on a large dataset for a specific task. During this phase, the model learns to identify general features that can be applicable across various tasks. For example, in image recognition, the initial layers of the model might learn to detect edges and textures, which are fundamental aspects of visual data. Once the model has been pre-trained, its learned weights and structures are transferred to a new model tailored for the target task. (See Figure 2.3)

In this transfer phase, the initial layers of the pre-trained model are typically retained because they capture general features useful for a wide range of tasks. The final layers, which are more task-specific, are fine-tuned to adapt to the new task's specific requirements. This fine-tuning phase involves additional training on a smaller task-specific data set, allowing the model to adjust and optimize its parameters for better performance in the new context. [84]

One of the primary advantages of transfer learning is its efficiency. By leveraging pre-trained models, transfer learning reduces the computational burden and time required for training new models. This is particularly beneficial in scenarios where obtaining large labeled datasets is challenging. Moreover, transfer learning can lead to improved model performance, as initial training on a large dataset helps the model learn robust and generalizable features that can enhance its effectiveness on the target task. [67]

However, transfer learning is not without drawbacks. One potential disadvantage is the risk of overfitting, particularly if the pre-trained model's domain is significantly different from the target task. Fine-tuning on a small dataset may cause the model to overly specialize in the new task, losing the generalization benefits obtained during pretraining. Additionally, the success of transfer learning is highly dependent on the similarity between the source and target tasks. If tasks are too dissimilar, the knowledge transferred might not be relevant, leading to suboptimal performance. [57]

Transfer learning can be categorized into three main types (See Figure 2.5): inductive transfer learning, transductive transfer learning, and unsupervised transfer learning, according to the relationship between the source and target domains and tasks [57].

In inductive transfer learning, the target task is different from the source task, regardless of whether the source and target domains are the same. In this scenario, some labeled data in the target domain are required to induce an objective predictive model for use in the target domain.

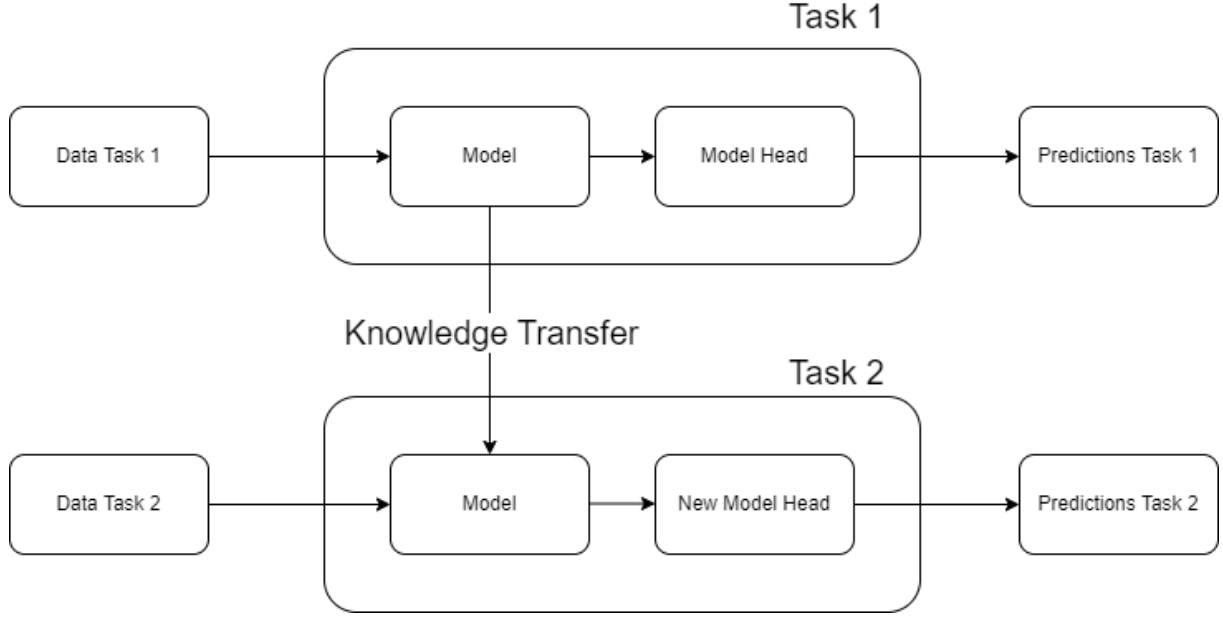


Figure 2.3: Transfer Learning Diagram [Own Image]

Inductive transfer learning can be further subdivided on the basis of the availability of labeled data in the source domain. If a large amount of labeled data is available in the source domain, this setting is similar to multitask learning, though with a focus on optimizing performance for the target task. Conversely, if no labeled data is available in the source domain, the setting resembles self-taught learning, where the label spaces between the source and target domains may differ, preventing direct use of side information from the source domain.

In transductive transfer learning, the source and target tasks are the same, but the source and target domains are different. This setting typically involves having a large amount of labeled data in the source domain and no labeled data in the target domain. Transductive transfer learning can be further categorized based on the feature spaces of the domains. If the feature spaces between the source and target domains differ, it presents a unique challenge. However, if the feature spaces are the same but the marginal probability distributions of the input data are different, this relates to domain adaptation, which is used for knowledge transfer in various tasks like text classification and addressing sample selection bias or covariate shift.

Finally, unsupervised transfer learning is similar to inductive transfer learning in that the target task is different but related to the source task. However, unsupervised transfer learning focuses on solving unsupervised learning tasks in the target domain, such as clustering, dimensionality reduction, and density estimation. In this case, no labeled data are available in both the source and target domains during training, which poses a unique set of challenges and opportunities for the development of robust and versatile models.

To effectively implement transfer learning in this project, we employ YOLOv8 [35], or "You Only Look Once" version 8, which is a state-of-the-art real-time object detection algorithm. Developed by Alexei Bochkovskii in 2020, YOLOv8 builds on the success of its predecessors, YOLOv5 and YOLOv7, offering superior accuracy, speed and efficiency. This cutting-edge technology enhances our ability to detect defects in industrial applications efficiently. YOLO family is based on PyTorch, leveraging the flexibility and performance of this deep learning framework to enhance our ability to detect defects in industrial applications efficiently.

Within each version of YOLO, there are five different pre-trained models based on the number

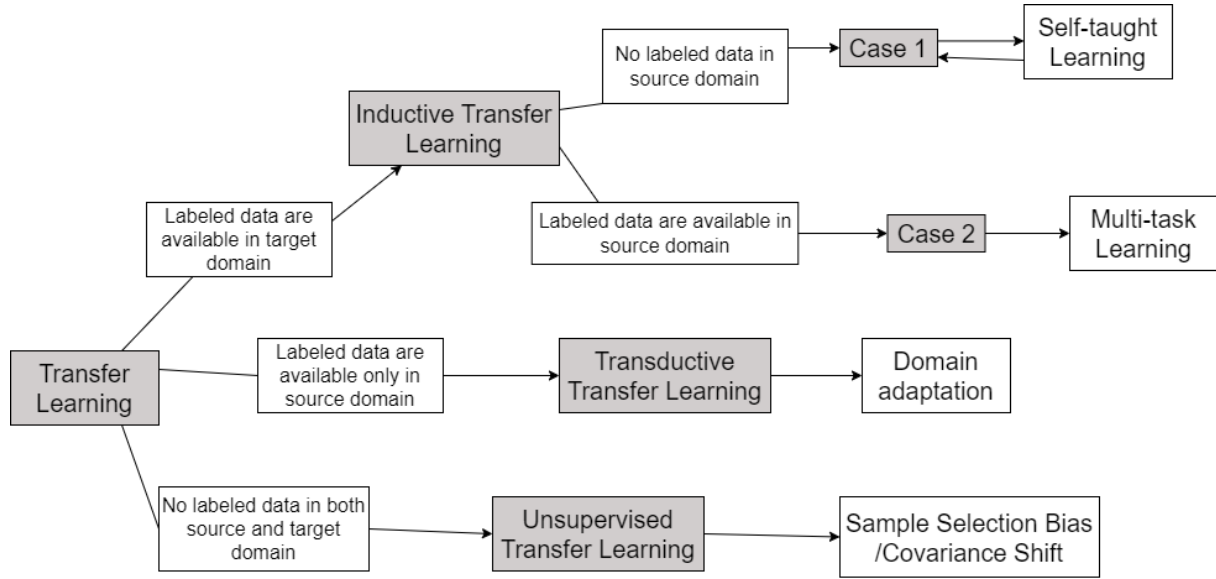


Figure 2.4: DTL Types [58]

of parameters they use. These models have been trained on the ImageNet dataset [14]. For example, for YOLOv8, these are the models: (See Figure 2.5)

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Figure 2.5: Pretrained YOLO Models [80]

The columns of the table mean the following:

- **Model:** Indicates the name of the YOLOv8 model variants (n, s, m, l, x)
- **Size (pixels):** The input size of the images, which is 640 pixels for all models.
- **mAP50-95:** Mean Average Precision averaged over IoU thresholds from 50% to 95%. It measures the model's accuracy in object detection.
- **Speed CPU ONNX (ms):** Inference time in milliseconds when running the model on a CPU using the ONNX format.
- **Speed A100 TensorRT (ms):** Inference time in milliseconds when running the model on an A100 GPU using TensorRT.

- Params (M): Number of model parameters in millions. More parameters often mean a more complex model with greater capacity to learn fine details, but it also requires more resources for training and execution.
- FLOPs (B): Number of floating-point operations the model performs during inference, measured in billions (Billion FLOPs). It indicates the computational complexity of the model.

In these graphs (See Figure 2.6), the differences between the various versions of YOLO can be seen, and within each version, the differences between the 5 different pre-trained models n (nano), s (small), m (medium), l (large), x (extra-large) from smallest to largest. We can observe that, on average, the YOLOv8 version is faster and has fewer parameters than its predecessors.

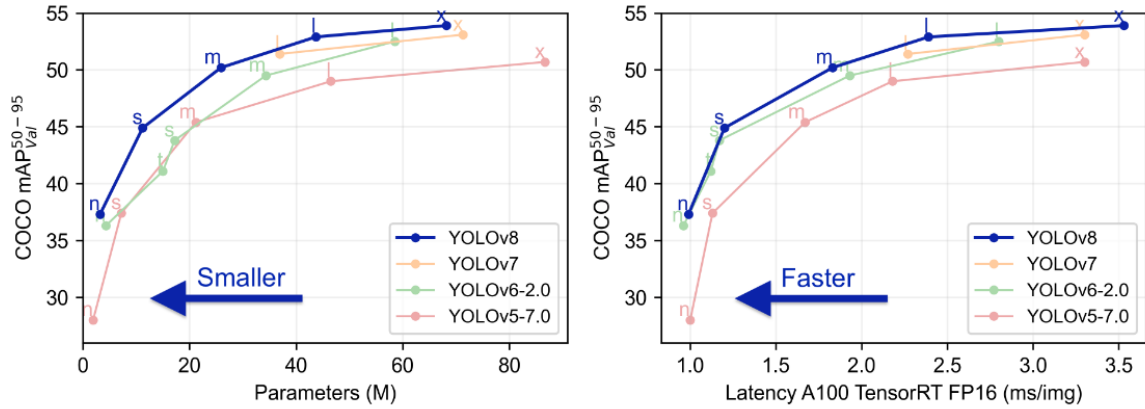


Figure 2.6: YOLO Family Comparison [81]

At the heart of YOLOv8's architecture is a hybrid design that combines convolutional neural networks (CNNs) with spatial attention mechanisms. CNNs are highly effective at extracting features from images, identifying edges, textures, and shapes. Spatial attention mechanisms enhance this capability by helping the model focus on the most relevant parts of the image for object detection, thus improving the accuracy and robustness of the detection process. [61]

Integrating YOLOv8 within the framework of transfer learning involves leveraging a model pre-trained on a large, generic dataset and fine-tuning it for the specific task of defect detection in industrial settings. This falls into the category of inductive transfer learning, where the target task (defect detection) is different from the source task (general object detection) for which the model was originally trained.

In this inductive transfer learning setting, we benefit from having access to labeled data in the target domain. The pretrained YOLOv8 model, initially trained on a comprehensive object detection dataset, can be fine-tuned using a smaller dataset of images specific to our defect detection task. This process adapts the model to the new context, enhancing its ability to identify defects with high precision and speed.

The family of YOLO models has been used in industrial environments for a while, as we can read in the article "YOLOv1 to YOLOv8, the Rise of YOLO and Its Complementary Nature toward Digital Manufacturing and Industrial Defect Detection" [31] "The review explores the key architectural advancements proposed at each iteration, followed by examples of industrial deployment for surface defect detection, endorsing its compatibility with industrial requirements."

In summary, the use of YOLO models in industrial defect detection exemplifies the integration

of cutting-edge AI technologies into manufacturing processes. These models provide a robust solution for real-time surface defect detection, enhancing the efficiency and accuracy of quality control systems. Using advances in YOLO architecture, industries can achieve significant improvements in product quality and operational efficiency, thereby maintaining competitive advantage in a rapidly evolving market. [61]

2.5 Contrastive Learning

Contrastive learning has emerged as a key technique in the realm of machine learning, particularly for tasks involving the estimation of similarity between pairs of data points. This learning paradigm is predicated on the idea of learning by comparing, where the objective is to bring similar data points closer while pushing dissimilar ones apart within the feature space. In the context of image pair similarity estimation, contrastive learning is typically implemented using Siamese neural networks, which are specifically designed to handle paired inputs. [51]

Siamese neural networks [10] take advantage of the principles of contrastive learning by employing a unique twin architecture (See Figure 2.7). This architecture consists of two identical subnetworks that share weights and parameters. Each subnetwork processes one of the two input images, extracting feature representations. These representations are then compared using a distance metric, such as Euclidean distance, to determine the similarity between the image pairs. The training process involves minimizing a contrastive loss function [87], which encourages the network to produce similar representations for similar images and dissimilar representations for dissimilar images.

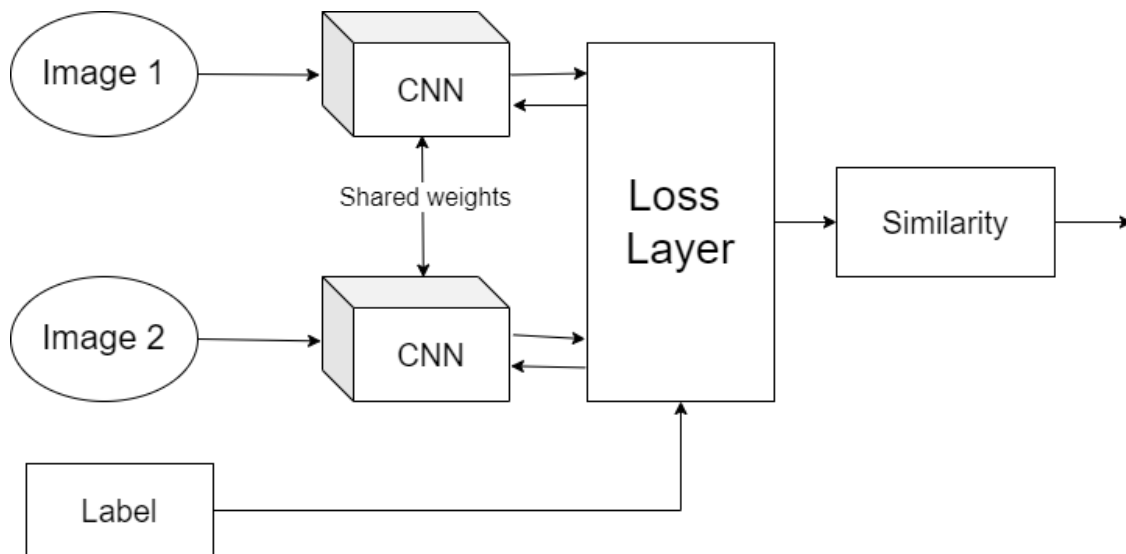


Figure 2.7: Siamese Model [Own Image]

The primary utility of this approach lies in its ability to map images into a high-dimensional embedding space where the proximity of points correlates with the similarity of the image. This is particularly advantageous in applications where the choice of fine-grained differences or similarities is crucial. For example, in facial recognition systems, Siamese neural networks can effectively verify identities by comparing a given face image with a database of known faces. [24]

One of the most significant advantages of Siamese neural networks, when applied to contrastive learning, is their proficiency in addressing the one-shot learning challenge. Traditional machine learning models typically require extensive labeled datasets to perform effectively, a requirement

that is often impractical. In contrast, Siamese networks can achieve strong performance with considerably less labeled data by focusing on the relational information between image pairs. This efficiency makes them particularly well suited for scenarios with limited training data. [47]

In practical applications, Siamese neural networks have been successfully employed in various domains for image similarity estimation. For example, in industrial settings, they have been used in the project “Steel Surface Defect Detection Based on Self-Supervised Contrastive Representation Learning with Matching Metric” [30]. This project addresses the challenge of defect detection in steel surfaces, a crucial aspect of quality control in manufacturing. Traditional supervised methods are highly dependent on large amounts of labeled data, which are often scarce and expensive to annotate. By leveraging self-supervised contrastive learning, the project constructed a novel model that learns robust embedding feature representations from large amounts of unlabeled data.

Another significant application is the project “Contrastive Self-Supervised Representation Learning Framework for Metal Surface Defect Detection” [91]. This study proposes a self-supervised representation-learning model that effectively addresses the limitation of scarce labeled datasets for emerging target defects by leveraging both labelled and unlabeled data. The proposed model was developed on the basis of a contrastive learning framework, supported by an augmentation pipeline and a lightweight convolutional encoder. The model effectiveness was evaluated using an unlabeled pre-training dataset created from three benchmark datasets and validated using the NEU metal surface-defect dataset. Interestingly, the method achieved a classification accuracy of 97.78% with fewer trainable parameters than the benchmark models, effectively extracting meaningful representations from unlabeled image data and improving downstream tasks for the classification of steel defects.

In conclusion, contrastive learning, implemented through Siamese neural networks, offers a powerful and efficient method for estimating image pair similarity. The ability to learn from limited data and the efficacy in various practical applications underscore the importance of this approach in the field of image processing. However, computational demands and sensitivity to model design highlight the need for ongoing research and optimization. As advances continue, Siamese neural networks are poised to play an increasingly integral role in the analysis and understanding of visual data across diverse sectors, including industrial quality control and beyond.

2.6 Evidently AI

Evidently AI is a cutting-edge machine learning (ML) platform designed to enhance the monitoring, evaluation, and overall management of ML models. As an open-source and cloud-based service, Evidently provides robust tools that enable data scientists and ML engineers to systematically track the performance and reliability of their models across various stages of the ML lifecycle.

Evidently offers a comprehensive suite of reports and test suites that provide in-depth insights into model and data performance:

- **Reports:**
A report in the context of machine learning monitoring, like those provided by Evidently, is a comprehensive document or output that aggregates and presents various metrics, statistics, and insights about different aspects of a machine learning model or its input data. These reports are crucial for diagnosing problems, understanding model performance, and making informed decisions about model maintenance and updates.

For this project, the Performance Report was used that provides detailed evaluations of model performance metrics, such as accuracy, precision, recall, F1 score, and ROC-AUC, offering a holistic view of how well the model is performing.

- **Test Suites:**

A test suite is a collection of tests that are designed to systematically evaluate different aspects of a machine learning model or its input data. Each test within a suite checks a specific property or behavior, ensuring that the model meets predefined standards and performs reliably under various conditions.

Performance Test Suite was the selected test for the project, confirming that the model maintains its performance metrics within acceptable ranges.

Evidently's cloud service offers a managed solution with additional features such as scalable infrastructure, automated updates, and enhanced security. This service is tailored for organizations that prefer a hands-off approach to managing their ML monitoring needs, enabling them to focus more on model development and less on infrastructure management.

Evidently's cloud service offers a managed solution, which is particularly advantageous as it allows the platform to be used as a Software as a Service (SaaS) without concerns regarding infrastructure and security. This cloud-based approach enables users to upload their reports and test suites to the cloud, where they can visualize the results through intuitive and detailed graphs.

Using Evidently as a SaaS simplifies the management of reports and tests, ensuring that data are securely stored and easily accessible from any location. This functionality facilitates collaboration among team members, as it provides real-time access to critical information and insights.

Another functionality of Evidently is the presence of tags and metadata, which significantly enhances the organization, filtering, and retrieval of reports and test suites. Tags are keywords or labels assigned to reports and tests, allowing users to categorize and group related items effectively. This categorization facilitates easy management and access to specific reports or tests within a large collection by filtering or searching for relevant tags. Metadata includes essential details such as the creation date, author, version information, and contextual information about the data and model, providing critical insights into the conditions and parameters under which the report or test was created.

Evidently stands out as a state-of-the-art platform in the realm of ML model management, offering essential tools and reports to ensure that models remain accurate, fair, and reliable. Its dual offering as an open source tool and a cloud service provides flexibility for different use cases, making it a versatile choice for data scientists and ML engineers committed to maintaining high standards of model performance and data quality.

2.7 Apache NiFi and Airflow

Apache NiFi and Apache Airflow, when combined, present a robust solution for the management and orchestration of data workflows, essential for continuous retraining of deep learning models. NiFi excels in automating the movement of data between heterogeneous systems using an intuitive graphical interface, making it perfect for real-time data ingestion, transformation, and distribution in applications dealing with high volumes of data from diverse sources such as databases and IoT sensors.[37]

Conversely, Apache Airflow is an open-source workflow management platform that enables the creation, scheduling, and monitoring of programmable workflows. Airflow is crucial for orchestrating complex workflows, managing sequential or parallel tasks, and ensuring the orderly execution of data processes and machine learning. [22]

NiFi can preprocess data in real-time by cleaning and transforming it, while Airflow orchestrates the retraining of models by scheduling training, evaluation, and deployment tasks [50].

In summary, NiFi and Airflow together provide an efficient and flexible infrastructure for data management and continuous retraining of deep learning models, ensuring that artificial intelligence solutions remain up-to-date and precise in real-time.

3 Design specifications and requirements

3.1 Flow Diagram

The global concept of the configured system is presented in Figure 3.1, where the system benefits from the asynchronous feedback of the human operator to enrich the classification categories. Although because of the heavily unbalanced categories the training process includes an embedded factory for data augmentation, looking to keep unbiased the training / retraining processes. The operator's provided feedback for coils is used to measure the performance of the existing models, while underperformance below a established threshold is the signal to request a model retraining, replacing the underperforming models

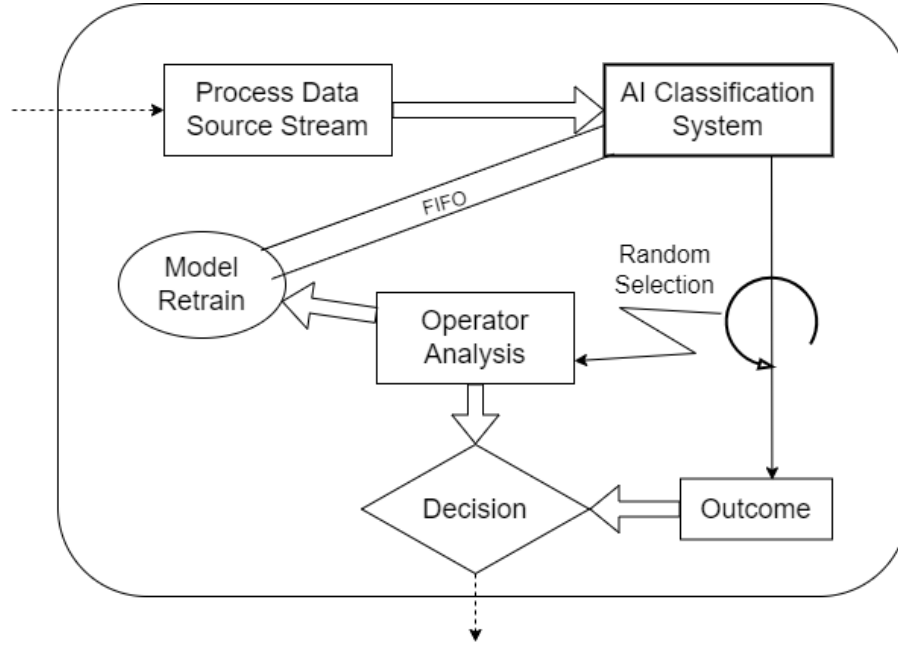


Figure 3.1: System Flow Diagram [Own image]

3.2 Requirements

In this section, we will present the main constraints that were taken into account when carrying out the project.

1. Scenarios Structure:

One of the main constraints is the design of the structure of the simulation scenarios. This structure can be found in the yaml files of each section of the project code and allows the user to create scenarios according to their interests.

For the Deep Learning and Constrastive Learning section, there are two main aspects that can be varied when building a Convolutional Neural Network (CNN). Firstly, the architecture or topology of the network, which refers to the number of layers of each type and the order in which they are arranged. Secondly, the parameters that can be varied within each type of layer.

For the Deep Transfer Learning section, the main aspects to vary are, on one hand, the pre-trained model chosen from which to extract knowledge for our task, and on the other

hand, the number of layers to freeze to enable DTL. In its image classification version, YOLOv8 is a model with 10 main layers that group other types of layers. Thus, the possibilities for freezing layers will range from 0 to 9 because the classifying layer can not be frozen. (See Figure 3.2)

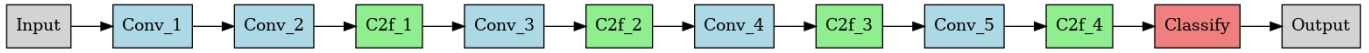


Figure 3.2: YOLOv8 Architecture [Own image]

As for the pre-trained models, there are several different ones that are explained later in the SOTA. Each of these models will be analyzed, except for the most computationally expensive one, due to time and resource management reasons. The impact of the number of epochs on performance, a variable that traditionally has a significant effect on results in DTL models, will also be studied.

These are the variants that will be studied in this project, but the system designed allows for easily specifying other parameters or variants to analyze and obtain results due to the generality and power of the system. This flexibility ensures that the system can adapt to different requirements and explore a wide range of configurations, making it a robust tool for various investigative and developmental purposes in any of these deep learning techniques.

2. Use of dedicated GPUs:

GPUs are utilized in Deep Learning model simulation projects due to their capability for massive parallelism, which allows many operations to be executed simultaneously. They also have a higher memory bandwidth that facilitates the rapid transfer of large volumes of data. Additionally, Deep Learning libraries and frameworks are optimized to leverage their capabilities, thereby enhancing performance and efficiency. This makes GPUs an indispensable resource for handling the computationally intensive tasks involved in training and running Deep Learning models. [32]

3. Use of Open Source Tools:

A decision of this project has been to use open-source technologies and tools. This approach not only reduces costs but also encourages innovation and the sharing of knowledge within the community. Tools like TensorFlow, Keras, PyTorch and Evidently AI are examples of such open-source frameworks used in this project, providing robust and scalable solutions for developing and deploying AI models.

4. Installation of Required Packages:

A critical constraint is the installation of the required packages specified in the requirements.txt file to launch the system. This file lists all the dependencies and versions needed for the project to function correctly.

4 Description of the proposed solution

In this part of the document, we explore the proposed solution designed to achieve the project’s objectives, leveraging the advanced technologies highlighted in the state of the art. The primary objective is to improve the efficiency and accuracy of defect detection in hot dip galvanized steel coils used in the automotive industry.

To address the challenges identified, we integrate several cutting-edge machine learning techniques. Firstly, we employ deep learning, particularly Convolutional Neural Networks (CNNs), to robustly identify and classify defects on steel surfaces. The power of CNNs lies in their ability to automatically learn hierarchical feature representations from raw image data, making them ideal for complex image recognition tasks.

Furthermore, we incorporate transfer learning to expedite the model training process and improve performance, especially when dealing with limited labeled data. By utilizing pre-trained models and fine-tuning them on our specific dataset, we can leverage the knowledge acquired from large-scale datasets and adapt it to our specific defect detection needs.

In addition, contrastive learning is applied to enhance the feature learning process. This technique, which focuses on learning representations by contrasting similar and dissimilar pairs, helps in achieving more discriminative features, thereby improving the model’s ability to distinguish between different types of defect.

In the subsequent sections, we will detail the specific methodologies employed, the architecture of the proposed models, and the integration of these technologies into a cohesive system aimed at revolutionizing defect detection in the automotive industry. This comprehensive approach not only addresses the current limitations of human-dependent quality inspection but also sets the foundation for more automated, reliable, and scalable solutions in the future.

4.1 Dataset

This section addresses the preliminary work performed, which was fundamental for the execution of this project. The data set was provided by the industrial partner.

The industrial partner provided us with precollected, processed, and manually labeled data from operators regarding the coils to be classified as acceptable (OK) or non-acceptable (NOK). The zinc coating on each coil was measured by nine sensors on each side, resulting in 264 measurements per sensor.

The figure of 264 measurements is a constraint we established. To achieve this, each coil is preprocessed according to its length, dividing it into 264 segments. This preprocessing step allows us to leverage the power of tensor operations effectively. Consequently, for coils shorter than the majority, this approach results in a higher sampling density. Each of the nine sensors on each side captures these 264 measurements (See Figure 4.1), leading to a comprehensive dataset. This configuration forms a data structure whose shape is $(N, 264, 18)$, representing N matrices of 264×18 , which can be visualized as images.

Given the imbalance in the dataset, with significantly more acceptable coils than non-acceptable ones, due to the manufacturing process tending to produce acceptable coils, data augmentation [74] was used. This technique allows for transformations on existing data to generate more NOK samples. Various transformations were applied to the matrices, such as row and column

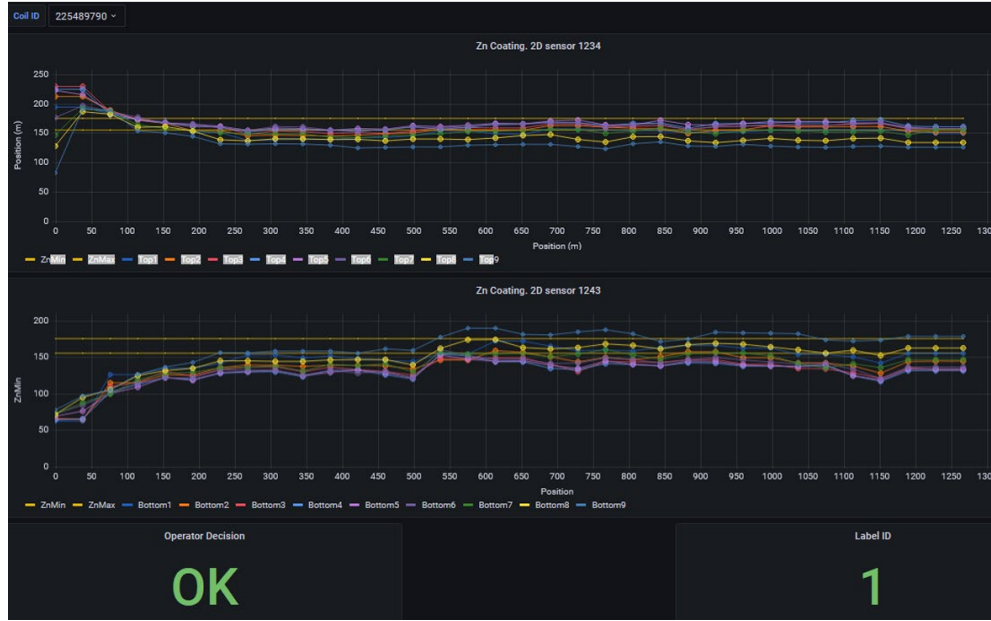


Figure 4.1: Zinc Measures

permutations, among others. This approach is crucial to balance the data set and enhance the model's ability to generalize to unseen cases [26].

One set consists of 4482 images, which will be divided into traditional training, validation, and test sets. Additionally, a small group of 60 images (30 OK and 30 NOK) was selected to obtain the 'gold standard', which is a separate dataset of unknown data. This set is critical for having a definitive reference with which to assess the performance of different models by predicting with different models on exactly the same data. This technique is inspired by the research done by [42] on "Adoption of machine learning technology for failure prediction in industrial maintenance: a systematic review".

4.2 Evaluation metrics

Regarding the metrics used to quantify the performance of the different models, they will be used uniformly for all evaluations. As explained in [12] Section 4.1.4 'Model Evaluation' of the paper "Adoption of machine learning technology for failure prediction in industrial maintenance: A systematic review", many metrics used in studies are inadequate for imbalanced datasets. For instance, accuracy and specificity can be misleading in these cases, as they do not adequately reflect the model's performance on minority classes. Conversely, recall only represents one aspect of the model's performance, omitting other critical aspects.

Therefore, the following metrics have been selected: Precision, Recall, and F1-Score [62], which more adequately characterize the results for an imbalanced data set. Additionally, ROC/AUC [21] will be included to complement the analysis, providing a more comprehensive view of the model's performance beyond the traditional confusion matrix (See Figure 4.2).

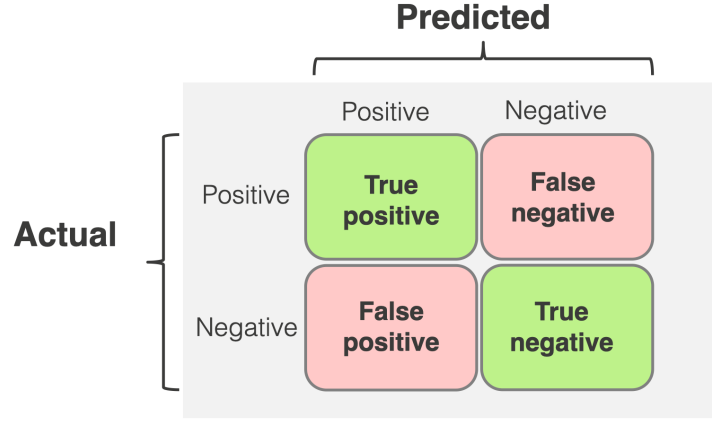


Figure 4.2: Explained Confusion Matrix [18]

- **Precision:** Precision measures the accuracy of positive predictions. It is defined as the ratio of true positives to the sum of true positives and false positives. This metric is crucial in scenarios where the consequences of false positives are significant.

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **Recall:** Also known as sensitivity, recall measures the model's ability to identify all relevant instances within a dataset. It is defined as the ratio of true positives to the sum of true positives and false negatives.

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- **F1-Score:** The F1-Score is the harmonic mean of precision and recall, providing a balance between the two by taking into account both false positives and false negatives. It is especially useful when classes are imbalanced.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **ROC/AUC:** The Receiver Operating Characteristic (ROC) curve is a graphical plot that illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied. The Area Under the ROC Curve (AUC) measures the entire two-dimensional area underneath the entire ROC curve and provides an aggregate measure of performance across all possible classification thresholds.

$$\text{AUC} = \int_{x=0}^1 \text{ROC}(x) dx$$

These metrics allow for evaluating both the model's ability to correctly identify NOK coils and its precision in doing so, providing an integral assessment of its performance.

4.3 Deep Learning

For this section of the solution's design, the objective was to conduct a systematic study of the various possibilities offered by Keras and TensorFlow when constructing a Deep Learning model and to analyze the results in our specific goal of defect detection. The original simulation plan was as follows:

- **Simulation Plan**

- **Topology**

- * Number of convolutional layers before flattening: [3, 5, 7, 10]
 - * Number of normalization layers: [None, one at the end of the last convolutional layer, behind each convolutional layer, every two convolutional layers]
 - * Number of pooling layers: [None, one before flattening, behind each normalization layer]
 - * Number of dense layers: [3, 7, 11 (the number of neurons ranging from 2048 in the first layer to 2 in the last layer, distributed according to the number of dense layers)]
 - * Number of dropout layers: [None, one behind each dense layer, one every three dense layers]

- **Configuration**

- * Dropout probability: [0.5, 0.3]
 - * Number of filters: [16, 32]
 - * Kernel width: [50 constant, 20 constant, 5 constant, increasing from 5 to 20 according to the convolutional layer, decreasing from 20 to 5 according to the convolutional layer]

A YAML [48] file was generated to read the information needed to perform the simulation, which contains three keys, which are as follows:

- **Evidently Cloud Data:** This section contains the credentials and necessary information to interact with Evidently Cloud services. It allows the system to generate and upload reports and test suites automatically, ensuring seamless integration with cloud-based evaluation tools. The required camps are the `https://app.evidently.cloud` of the cloud service, your personal token and the project id where you want to upload the reports and test suites.
- **Architectures:** List specifying the layers to define an architecture. The list format for architectures enables the definition of complex neural network topologies with ease. By using simple notation such as "C" for convolutional layers and "F" for dense layers, the system can dynamically construct neural networks. Each element represents a layer and can be one of the following:
 - 'C': Convolutional layer
 - 'N': Normalization layer /
 - 'P': Pooling layer /
 - 'flatten': Flatten layer /
 - 'F': Dense layer/

– 'D': Dropout layer

- Configurations: For the configurations, the values are stored as strings which are later parsed and processed. This approach allows for dynamic adjustment of parameters such as kernel widths, filter sizes, and dropout rates. The system can thus adapt these configurations to fit any architecture defined in the YAML file. Also in configurations are included some thresholds that are used while creating test suites, to check if a metric is above or under one threshold. The required camps are the kernel width, the number of filters and the dropout rate.

To generalise architectures easily, a simple and scalable notation was needed. This way, an architecture can serve for any possible configuration that may arise. Therefore, the following notation was chosen:

List specifying the layers to define an architecture. Each element represents a layer and can be one of the following: 'C': Convolutional layer / 'N': Normalization layer / 'P': Pooling layer / 'flatten': Flatten layer / 'F': Dense layer / 'D': Dropout layer

Regarding the configurations, a similar approach was required to ensure that a single configuration could apply to any architecture that might be used. Consequently, each value for kernel widths, filters, and dropouts was designed to be a string that is subsequently processed.

Once the YAML file structure with the information is determined, it is imperative to generate models using Keras and TensorFlow. To achieve this objective, three distinct classes were created: one for constructing CNNs based on architectures and configuration parameters, other for processing configuration values read as strings to adapt them to the required architecture on each occasion, and another for, combining the previous two, create tuples for each model including the model, the keras model and its metadata, tags and test suite thresholds. Consequently, the same kernel width value serves to create a 3-layer convolutional architecture, implying that the CNN constructor will require 3 values for the kernel width, as for a 10-layer convolutional architecture, which will need 10.

In this manner, configurations are processed to obtain arrays with as many elements as convolutional layers indicated in the architecture for kernel widths and the number of filters, or dropout layers in the case of dropout rates. This class will also generate an array indicating the number of neurons to be included in each fully connected layer, ranging from 2048 to 0 proportionally according to the number of dense layers.

To utilize the Evidently AI functionality of tags and metadata, it is imperative to generate them based on the architecture and configuration received. Subsequently, graphs and results can be obtained by filtering through these tags or metadata to visualize trends or draw conclusions. To achieve this, several tags will be generated based on various factors.

These tags will include:

- Architecture name
- Number of convolutional layers
- Number of fully connected layers
- Normalization type (None / Partial - one at the end of the last Convolutional layer / Full - behind each Convolutional layer / Full - every two Convolutional layers)

- Pooling type (None / Partial - pooling before Flatten layer / Full - pooling after each Normalization)
- Dropout type (None / Partial - Dropout after 3 Dense layers / Full - Dropout after each Dense layer)
- Kernel width value
- Number of filters value
- Dropout rate value

The model's metadata will include the values from which it is formed in the YAML file.

With this method of structuring and processing information to generate models, we obtain a highly general and potent approach for conducting extensive simulations to fulfill the objective of the section, which is to conduct a systematic study of the parameters and architectures that can yield optimal performance in detecting defects and classifying coils as OK or NOK. Using the Evidently platform, we can systematically analyze the results obtained from these simulations, enabling informed decisions on model refinement and enhancement.

All the information from each combination of architecture and configuration, once read and processed, is passed to the neural network constructor. The purpose of this class is to utilize the processed information, which includes configuration, tags, and metadata, along with architecture, to construct the CNN using Keras and TensorFlow.

For example, the resulting CNN model formed by this information, replicating a case on the yaml, would be the following: (See Figure 4.3)

- Information:
 - architecture: ['C', 'C', 'C', 'N', 'P', 'flatten', 'F', 'F', 'F', 'D']
 - configuration: ['kernel widths': '50 cte', 'filters': '32 cte', 'dropouts': '0.5 cte', 'test suite thresholds: [0.6, 0.65, 0.7, 0.75, 0.8]]

Once all this is done, the model is ready to go into the simulation.

The simulator operates in such a way that, once all the models (CNNs) are generated, it iterates over them to obtain the results for each one.

In this section, cross-validation is implemented through the sklearn library [60]. The metrics calculated for each of the total number of splits will include precision, recall, F1-Score, ROC/AUC, and the confusion matrix. The detailed results of each split from the cross-validation of every iteration will be stored in a JSON file, allowing for identification of the results in the test set for the corresponding iteration and split of each model. In addition, the indices of the images used for training and evaluation in each case will also be stored to have complete trazability and to ensure that the results are reproducible.

With respect to the management of the report generation and test suites, the following strategy was adopted. To optimize resource management, the best model among the 50 will be selected as the representative. The F1-Score was chosen as the indicator to select the best model. This metric was selected for two main reasons: first, being the harmonic mean of precision and recall,

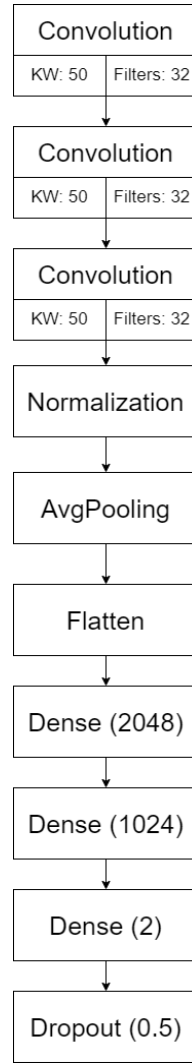


Figure 4.3: Example Model [Own Image]

it encapsulates both variables into one. The second reason is its usefulness in classification problems with imbalanced classes, which is the case here.

The model identified as the representative of the group is stored. This model is then reloaded, and a prediction is performed on the 'gold standard' set to obtain the final results from the best model of each type. Based on this prediction, both the report, for visualizing graphs and metrics, and various test suites with different thresholds derived from the configurations, for verifying if the metrics surpass these limits, are created and uploaded to Evidently Cloud Service. This strategy provides a consistent methodology for model simulation and result storage for subsequent analysis.

Having examined all aspects of the system, we can now understand it in its entirety on Figure 4.4

Upon launching the main script, the user will input arguments specifying parameters essential for conducting simulations (See Figure 4.5). These parameters include the number of cross-validation splits, the number of repetitions, and the YAML file from which the data will be sourced. In particular, all architectures have been segmented into four distinct files, each corresponding to different convolution quantities, as defined in the simulation plan. However, this

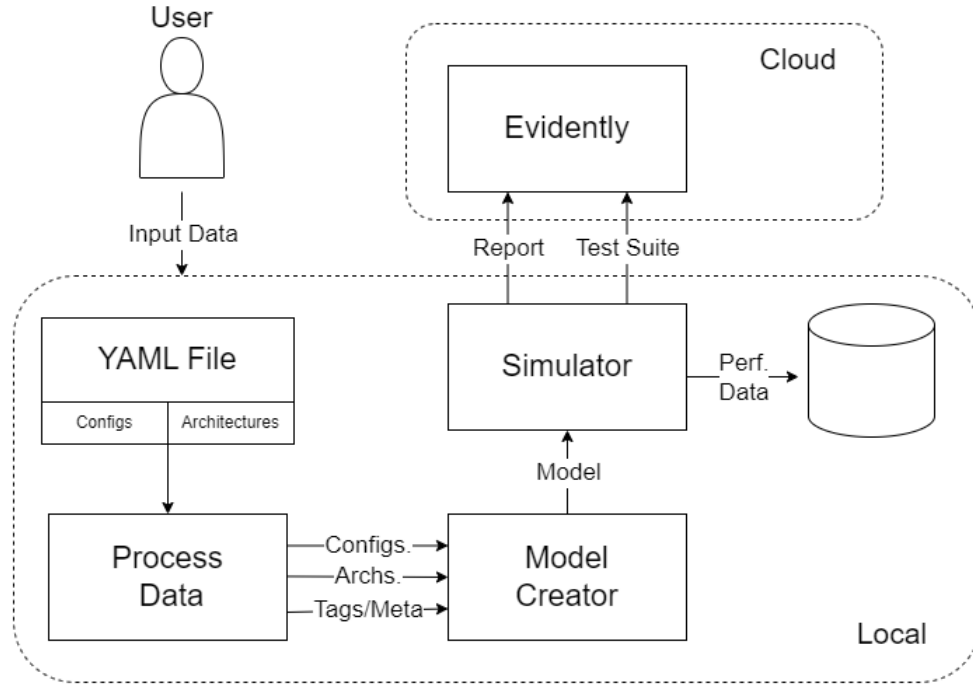


Figure 4.4: Complete System [Own Image]

segmentation is flexible and can be adjusted as needed.

```
# Create an ArgumentParser object to handle command-line arguments
parser = argparse.ArgumentParser(description='Model training and evaluation simulator.')
parser.add_argument('-r', '--repetitions', type=int, required=True,
                    help='Number of simulations')
parser.add_argument('-s', '--splits', type=int, required=True,
                    help='Number of folders for cross-validation')
parser.add_argument('-a', '--architecture', type=str, required=True,
                    choices=['3convs', '5convs', '7convs', '10convs'],
                    help='Architecture type (3convs, 5convs, 7convs, 10convs)')
parser.add_argument('-v', '--verbose', type=int, default=0,
                    help='Verbosity level (0: no messages, 1: specific messages, 2: general messages, 3: detailed messages, 4: all messages)')
# Parse the command-line arguments
args = parser.parse_args()
```

Figure 4.5: Arguments [Own Image]

Depending on the selected YAML file, the data are read and forwarded to the configuration processor. The processor adapts these data to generate inputs for the neural network constructor.

Subsequently, using a dedicated class, Keras and TensorFlow models are instantiated using the constructor and processed data. These models are then passed to the simulator to execute training and evaluation procedures as specified by the user. The results are subsequently uploaded to Evidently for further analysis.

For this study, as mentioned in the methodology in the Introduction section 1, each model will be tested using cross-validation with five splits. This process will be repeated 10 times to ensure statistical representativeness.

To complete all simulations, 4 scripts are launched, each varying in the file they read from. On a Unix-based operating system, the nohup command can be used to run these processes in the background.

```
1 nohup python3 SIM_DL.py -r 10 -s 5 -a 3convs
2 nohup python3 SIM_DL.py -r 10 -s 5 -a 5convs
3 nohup python3 SIM_DL.py -r 10 -s 5 -a 7convs
4 nohup python3 SIM_DL.py -r 10 -s 5 -a 10convs
```

4.4 Deep Transfer Learning

4.4.1 Dataset processing

In this section, as mentioned in the State of the Art review , we will use a cutting-edge model called YOLOv8, the latest advancement in this model family. These models operate on image input data for various tasks such as object detection, image classification, instance segmentation, or pose estimation. For our purposes, we will focus on the classification task. Consequently, we need to process the original Dataset, which, as we recall, is structured as (N, 264, 18), representing N matrices of dimensions 264x18. The cross-validation set comprises 4482 matrices, while the gold standard set contains 60 matrices.

In this section, it is necessary to transform these 4542 matrices of 264x18 into square images of 264x264 in a standardized image format (png, jpeg, etc.) (See Figure 4.6). To this end, a specific class was created to handle the data processing, which takes as arguments both sets. The primary method of this class, named 'preprocess', takes the directory path where the preprocessed data will be saved as an argument. This method reads the matrices and their labels and processes them as follows:

1. Pixel Duplication:

A duplication factor is determined to adjust the image size to the new desired dimensions. This factor is calculated as the ratio of the desired size (264 pixels) to the current dimension of the image. Since the original images are 18 pixels wide, the duplication factor will be $264/18 = 14.6$. This factor is rounded up to ensure the image is sufficiently large after duplication.

2. Application of Duplication Factor:

Each pixel of the original image is repeated as many times as indicated by the duplication factor along the horizontal axis, thus expanding the image.

3. Cropping to Desired Size:

After duplication, the resulting image may exceed 264 pixels in one direction. Therefore, the image is cropped to the exact size of 264x264 pixels.

4. Scaling and Conversion to Grayscale:

Once the image size is adjusted, the pixel values are scaled to the range of 0 to 255 and converted to grayscale. This scaling is done by first normalising the values to the range [0, 1] and then multiplying them by 255. Finally, the image is converted to the uint8 data type, which is suitable for grayscale images.

5. Transpose:

Finally, the images are transposed to restore the original orientation of the coils, which is how the data will be presented to the model during prediction in the real-world use case.

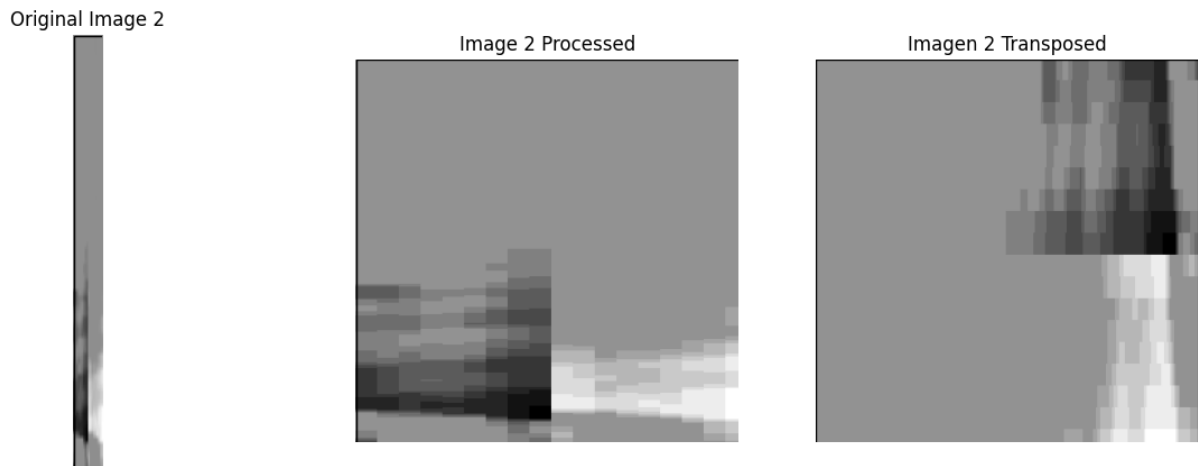


Figure 4.6: Image Processing [Own Image]

After that, the processed images are saved in PNG format in the corresponding directories, and the labels are stored in text files for a reason that will be explained just after this. The PNG file names include a number from 0 to 4541 to ensure each of the 4542 images has a unique name, along with a label indicating whether the image is "OK" (0) or "NOK" (1).

This method of image processing was chosen primarily to preserve the spatial relationship between features, in other words, to maintain the spatial relationship between areas with thicker zinc layers and those with thinner layers.

4.4.2 Cross validation

YOLO is a family of models that reads input data through a directory system structured in this way:

The root folder will contain all the image subfolders. Within this root folder, three main subfolders should be created: train, test, and optionally val. The train subfolder contains all the images intended for model training. Within the train, each object class should have its own subfolder. For this case, working with the 'OK' and 'NOK' classes, there will be subfolders named 'OK' and 'NOK', respectively. Each of these subfolders will contain the images corresponding to that class, each uniquely named. Similarly, the test subfolder contains the images for model evaluation and follows the same subfolder structure by class as the train.

Once this is understood, cross-validation for YOLOv8 can be prepared. Based on the previous explanation, five 'root' folders should be generated, each representing one of the five splits. Within each root folder, there will be the corresponding train and test sets for that split. A function called 'generate_yolov8_cross_validation' has been developed to generate the cross-validation for YOLOv8, and it works as follows:

1. Initialization and Data Collection:

The function takes as arguments the dataset path and the number of partitions for cross-validation. It also identifies all the label files (.txt) in the specified dataset directory for each corresponding image.

2. Label Counting:

A data structure (Pandas [76] DataFrame) is created to store the number of labels for each

class for each image.

3. Cross-Validation Setup:

The cross-validation scheme is configured using KFold from sklearn library [60], specifying the desired number of partitions and ensuring that the data split is random but reproducible.

4. Fold Assignment:

The data is divided into N partitions, assigning each image and its labels to a training or test set in each fold. This is stored in a data structure that maps each image to its corresponding assignment in each fold.

5. Label Distribution Evaluation:

For each fold, the distribution of labels in the training and test sets is calculated, ensuring that the proportion of labels is balanced across the different folds. This helps to avoid biases and ensures that each fold is representative of the entire dataset.

6. Directory Preparation and Image Distribution:

The necessary directories are created to store the images organised by fold. Then, the images are assigned and copied to the corresponding directories according to the determined assignment for each fold. Each image is placed in the appropriate subdirectory (training or test, OK or NOK) based on its label and fold partition.

4.4.3 DTL Systematic Study

For the transfer learning section, a systematic study of the possibilities offered by YOLOv8 was conducted using the Ultralytics library [34]. The following simulation plan was prepared:

- **Simulation Plan**

- **Parameters to vary**

- * YOLOv8 pretrained model used: [n, s, m, l]
 - * Number of epochs to consider: [50, 100, 300]
 - * Number of final layers to 'freeze' to allow DTL: [0, 3, 5, 8, 9]

Unlike the Deep Learning section, this simulation plan includes varying the number of epochs, as they traditionally have a more significant impact on the results [36].

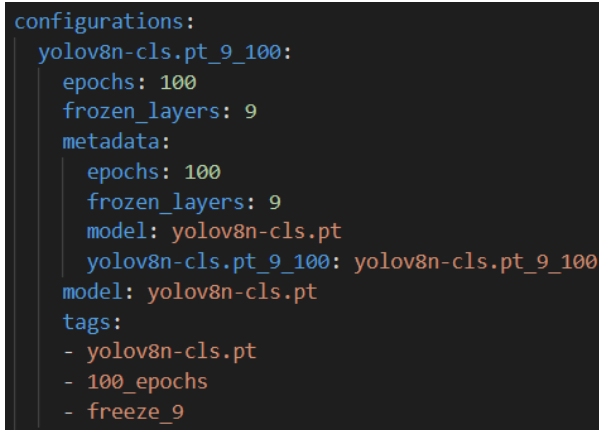
In this case, another YAML file was generated from which to read the necessary data for the simulations. Each combination of parameter values to be varied will be referred to as a configuration. Due to the number of possibilities to vary—4 for the YOLO model, 3 for the epochs, and 5 for the layers to freeze. Combinatorially, there are 60 different configurations to simulate.

The keys contained by the YAML file in this case are only two:

- **Evidently Cloud Data:** As in Deep Learning section, it contains the credentials and necessary information to interact with Evidently Cloud services. The difference is that reports and test suites from Deep Learning and Transfer Learning are going to different projects. That means the reports/test suites are not all mixed together.

- **Configurations:** Configurations store data for building the models. The name of each configuration will be a combination of possible values from the simulation plan taken. It will follow the format pretrained model namenumbers of frozen layersnumber of epochs. Regarding the fields, it will include one field for each value from the simulation plan to vary, along with metadata and tags for Evidently that will include information about the model's composition using these parameters. This will facilitate filtering in the results and deriving conclusions from them.

For example, this is a real case from the YAML file: (See Figure 4.7)



```
configurations:
  yolov8n-cls.pt_9_100:
    epochs: 100
    frozen_layers: 9
    metadata:
      epochs: 100
      frozen_layers: 9
      model: yolov8n-cls.pt
      yolov8n-cls.pt_9_100: yolov8n-cls.pt_9_100
    model: yolov8n-cls.pt
    tags:
      - yolov8n-cls.pt
      - 100_epochs
      - freeze_9
```

Figure 4.7: Example Config DTL [Own Image]

This configuration results in a model that uses the pre-trained model 'yolov8n-cls.pt', the smallest among the others, and that has all its layers frozen.

Thanks to the YOLO library, it is not necessary to construct the models explicitly, since YOLO itself handles this task. It only requires the pre-trained model to be used, the training parameters (epochs, batch size, patience, etc.), and the training data for the 'train' case. For making predictions, it only needs the test data and the newly trained model, not the pre-trained ones. This reduces the need for extensive preprocessing steps before starting the simulation, unlike in the previous section, where configurations had to be processed and models constructed.

For this section, the strategy to upload the reports and test suites to Evidently will be the same as in the previous section: the model with the highest F1-Score on its corresponding test set among the total number of models generated will be selected. The weights are stored, reloaded, and predictions are made on the gold standard set, which will be uploaded to the Evidently cloud service to obtain the results.

In this section, the overall system will be simpler compared to the previous one (as can be seen on Figure 4.8), as it is not necessary to process the data or construct the models. The YOLO library itself will handle these tasks. Consequently, the diagram for this section will feature significantly fewer elements and will be more straightforward.

Lastly, as in the previous section, the parameters for the simulations will be specified in the command used to launch the simulation. The same nomenclature will be used to indicate the number of cross-validation splits, the number of repetitions per cross-validation, verbosity, and the file from which to read. In this instance, the models have been divided into YAML files based on their pre-trained models, resulting in four files with 15 models each.

This approach allows the simulation to be launched from the same script with the desired

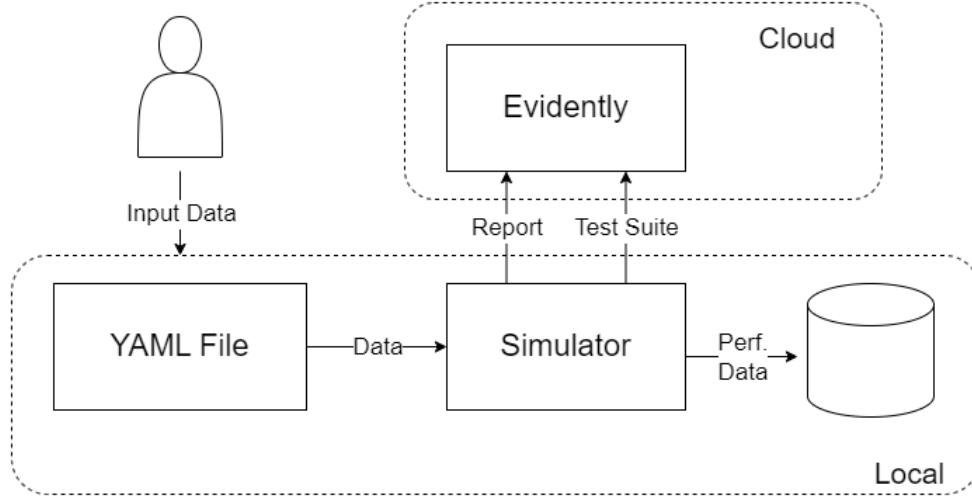


Figure 4.8: Complete System DTL [Own Image]

parameters.

Following the methodology outlined in the introduction, the script will be run with 5 splits for cross-validation and 10 repetitions. Thus, using four commands, the entire set of models to be simulated can be covered.

```

1 nohup python3 SIM_DTL.py -r 10 -s 5 -a N
2 nohup python3 SIM_DTL.py -r 10 -s 5 -a S
3 nohup python3 SIM_DTL.py -r 10 -s 5 -a M
4 nohup python3 SIM_DTL.py -r 10 -s 5 -a L

```

4.5 Contrastive Learning

4.5.1 Image Pairs Generation

As mentioned in the state-of-the-art section [2], it is necessary to create pairs of images to train the Deep Learning model for this section, as the model's output is the similarity between two images. For this purpose, a function has been designed to accomplish this task.

The function "make_pairs" is designed to create pairs of images along with the corresponding labels to train a machine learning model. The function is instrumental in generating training data that can be used to teach a model to distinguish between matching and non-matching pairs of images.

This function accepts two arguments: x, a list of images, and y, a list of the corresponding labels. Each element in y is an integer that represents the class or category to which the corresponding image in x belongs. The primary goal of this function is to output a tuple consisting of two numpy arrays. The first array contains pairs of images, and the second array contains binary labels indicating whether the images in each pair belong to the same class (0) or different classes (1).

The function then initializes two empty lists: pairs and labels. It iterates over each image in x,

using its index to maintain alignment with the labels in y . For each image, the function performs two main tasks:

1. Creating Matching Pairs:

It selects an image (x_1) and its corresponding label ($label_1$). It then randomly selects another image (x_2) from the same class ($label_1$). This pair of images, both belonging to the same class, is appended to the pairs list, and a label of 0 (indicating a matching pair) is added to the labels list.

2. Creating Non-Matching Pairs:

The function then needs to create a pair of images from different classes. It randomly selects a new label ($label_2$) that is different from $label_1$. This is achieved using a while loop to ensure that $label_2$ is distinct. Once a different class is chosen, an image (x_2) from this new class is randomly selected. This pair, consisting of images from different classes, is added to the pairs list, and a label of 1 (indicating a non-matching pair) is appended to the labels list.

4.5.2 Reference Images

For this section, it was necessary to obtain reference images for each class. The objective was to make a systematic approach to obtaining reference images that closely represent each of the two clusters identified by the DBSCAN and KMeans clustering algorithms. Initially, the data set consists of images which are flattened into vectors and stored in the variable X . To facilitate the clustering process, the Principal Component Analysis (PCA) algorithm is first applied to the data set to extract the principal components of the images. This step reduces the dimensionality of the data while preserving as much variance as possible, making subsequent processing more efficient and effective. Following the PCA, the Uniform Manifold Approximation and Projection (UMAP) algorithm is applied to further reduce the dimensionality of the data. UMAP is particularly useful for clustering, as it preserves both the local and global structure of the data better than PCA alone, facilitating more meaningful and interpretable cluster formation.

Afterwards, the data can be visualized separated by their true label to verify their actual separation. Six graphs will be created using six distinct distances [13] for this purpose:

- Euclidean Distance: The Euclidean distance is the straight-line distance between two points in a Euclidean space.

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2} \quad (4.1)$$

- Manhattan Distance: The Manhattan distance is the sum of the absolute differences of the coordinates of two points.

$$d(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^n |q_i - p_i| \quad (4.2)$$

- Jaccard Distance: The Jaccard distance measures dissimilarity between two sets. It is the complement of the Jaccard index, which measures similarity.

$$d(\mathbf{p}, \mathbf{q}) = 1 - \frac{|\mathbf{p} \cap \mathbf{q}|}{|\mathbf{p} \cup \mathbf{q}|} \quad (4.3)$$

- Cosine Distance: The Cosine distance measures the cosine of the angle between two non-zero vectors in an inner product space. It is derived from the cosine similarity.

$$d(\mathbf{p}, \mathbf{q}) = 1 - \frac{\mathbf{p} \cdot \mathbf{q}}{\|\mathbf{p}\| \|\mathbf{q}\|} \quad (4.4)$$

- Bray-Curtis Distance: The Bray-Curtis distance measures the compositional dissimilarity between two different sites. It is often used in ecological studies.

$$d(\mathbf{p}, \mathbf{q}) = \frac{\sum_{i=1}^n |p_i - q_i|}{\sum_{i=1}^n (p_i + q_i)} \quad (4.5)$$

- Yule Distance: The Yule distance is a dissimilarity measure used for binary (presence/absence) data.

$$d(\mathbf{p}, \mathbf{q}) = \frac{2 \cdot b \cdot c}{a \cdot d + b \cdot c} \quad (4.6)$$

where a , b , c , and d are the counts of:

- a : number of elements where both $\mathbf{p}$ and $\mathbf{q}$ are 1,
- b : number of elements where $\mathbf{p}$ is 1 and $\mathbf{q}$ is 0,
- c : number of elements where $\mathbf{p}$ is 0 and $\mathbf{q}$ is 1,
- d : number of elements where both $\mathbf{p}$ and $\mathbf{q}$ are 0.

This analysis allows us to generate graphs that visualize the distribution or concentration of samples. In the same graph, samples from both the test set and the gold standard will be presented.

First, we will visualize the diagram generated using Yule's distance (See Figure 4.9). This graph represents the projection of high-dimensional data into a two-dimensional space in such a way that the structures of the data are preserved as well as possible. The x and y axes represent each of the two dimensions of the two-dimensional space where the data are projected. As can be read in the legend, the blue dots are samples from the OK class of the test set, while the green dots are samples from the NOK class. For this set, the red crosses belong to the OK class, while the brown crosses belong to the NOK class.

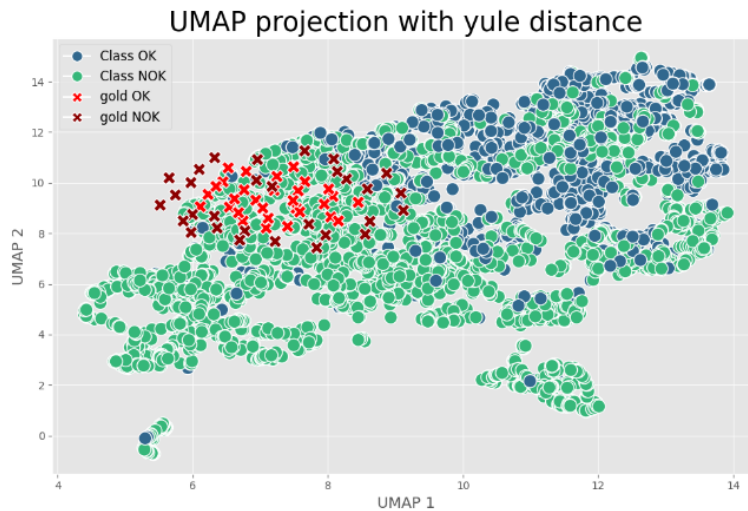


Figure 4.9: Single Cluster [Own Image]

4. Description of the proposed solution

In this specific case, it can be observed that a separation between the data can be more or less distinguished. It can be seen that the samples of the OK class are located more to the right and upwards, while the samples of the NOK class are more to the left and downwards. Regarding the gold standard set, the trend seems to change, as it can be seen how the samples of the OK class are located in the center, surrounded by samples of the NOK class.

Having visualized and explained this first graph using Yule's distance, we can now view it alongside other graphs generated with different distance measures to determine which offers better separation of the data. (See Figure 4.10)

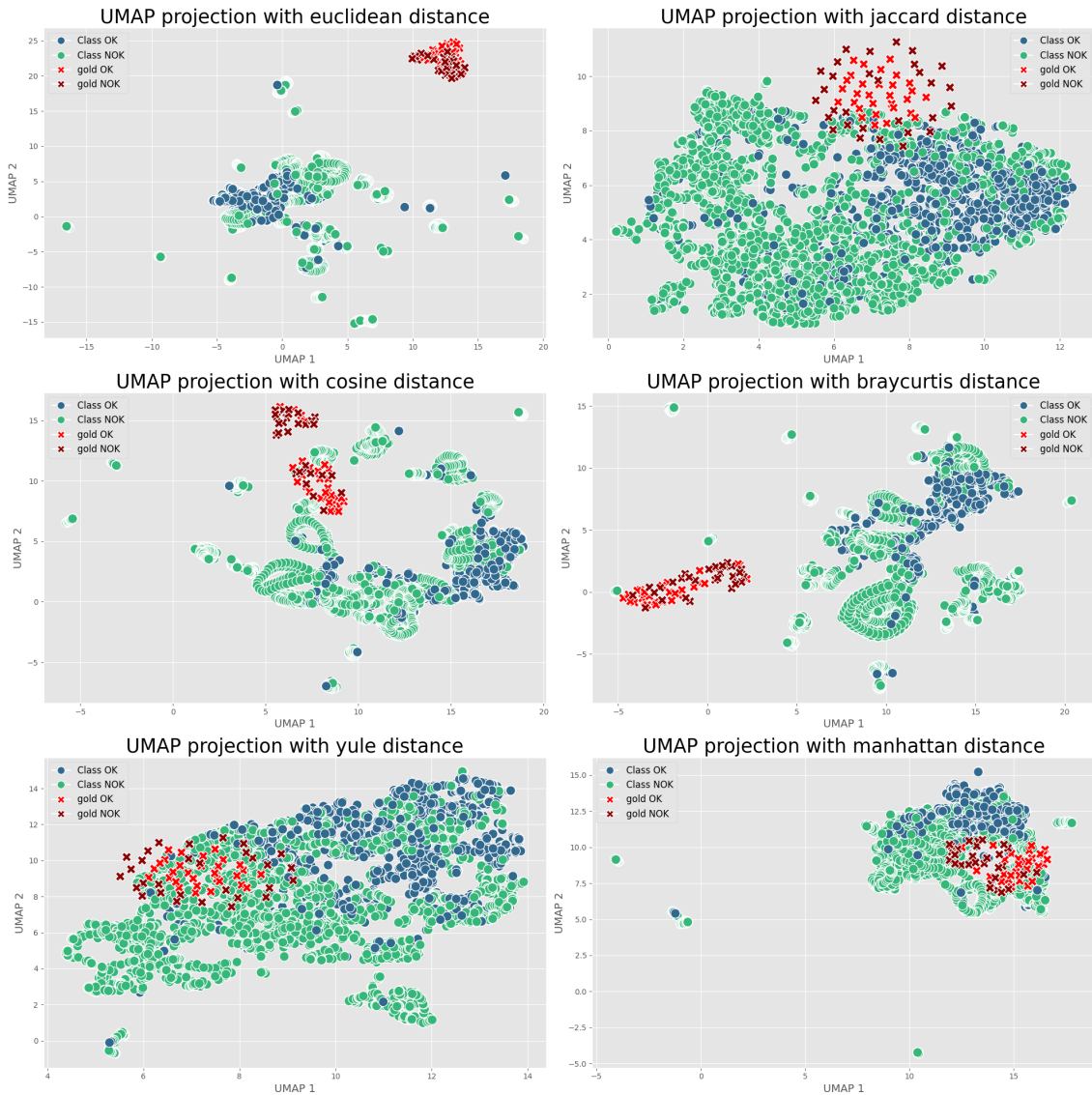


Figure 4.10: Clusters [Own Image]

After viewing all the graphs, it can be seen that generally there is not a very good arrangement of the data in this two-dimensional space, which can hinder the task of the classifier. None of the graphs seem to offer a good distribution, except for the Manhattan and Yule distances. For Yule, the distribution has already been discussed, and for Manhattan, it is noted that both sets have a relatively good separation, with the test set positioned above and below, and the gold standard to the left and right.

Following this analysis, the Manhattan distance was chosen as the preferred metric to be used

as a reference image for classification purposes in subsequent tasks.

As can be seen in Figure 4.11, subsequently, both DBSCAN and KMeans algorithms are employed inside each of the clusters. In this way, we perform a separation within the cluster itself and identify several representatives that maximally describe the different small groups within each cluster. Consequently, we will have several images that are not very similar to each other, but all of them describe the same cluster. This is done for both of the clusters, OK and NOK.

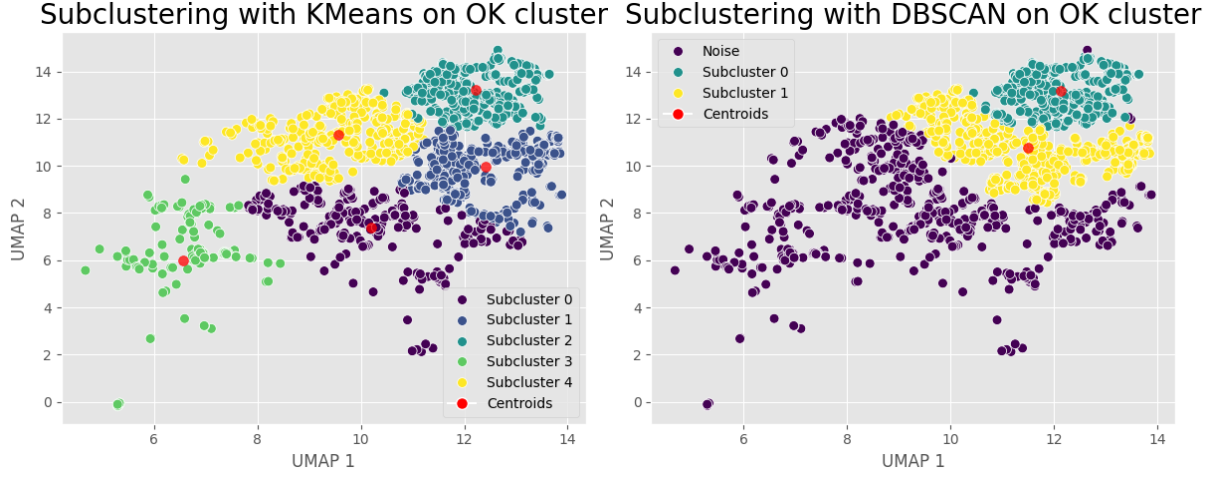


Figure 4.11: Subclustering on OK Cluster [Own Image]

To identify the images that are the most representative of each cluster, the function "pairwise_distances_argmin_min" is employed. This function calculates the pairwise distances between the cluster centroids and the data points, identifying the images closest to the centroids of both the DBSCAN and KMeans clusters. Subsequently, the images corresponding to these indices are extracted from the original X data set and reshaped to their original dimensions (264x18 pixels) for visualization purposes. (See Figure 4.12) This method ensures that the most representative images of each cluster are effectively identified and visualized, providing clear insights into the characteristics of each cluster.

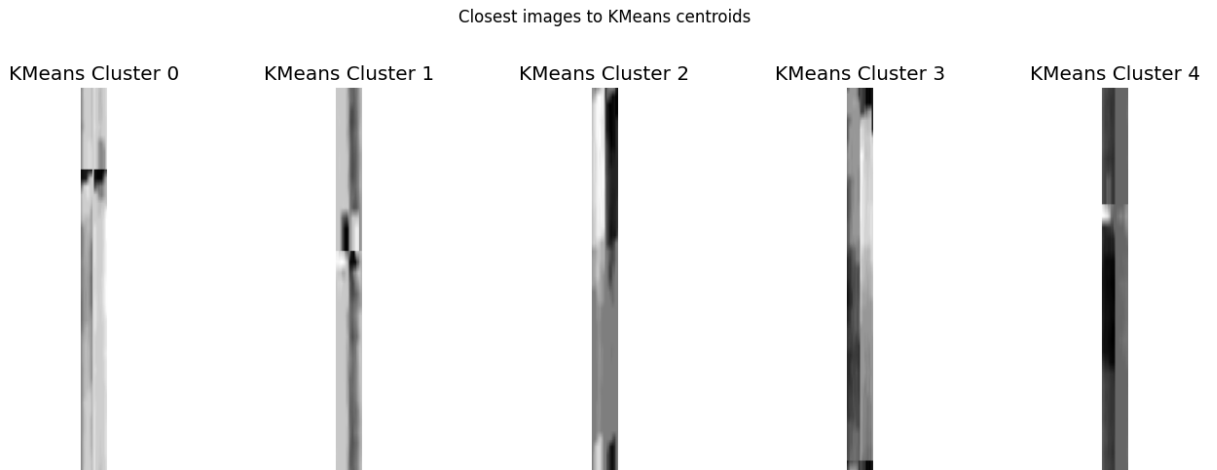


Figure 4.12: KMeans Reference Images of OK Cluster [Own Image]

4.5.3 Classification method and Study

For this third section of the solution design, the objective was to perform classification using the contrastive learning technique introduced in the State of the Art. This technique, as explained earlier, mainly involves calculating the similarity between two images and learning to distinguish between pairs of the same class and different classes. This is achieved using two "siblings" CNNs that serve as encoders to extract features from each image pair. Each pair is labeled: 0 if the images belong to the same class and 1 if they belong to different classes. Subsequently, utilizing the Euclidean distance between the feature vectors of the two images, the siamese network learns to discern between same-class and different-class pairs, whether they are labeled OK or NOK.

For this initial phase, the CNNs used as encoders will be selected based on their performance in the first section, where they demonstrated superior classification ability. These CNN architectures are chosen because they have shown better performance compared to others in terms of both image classification and feature extraction. Therefore, instead of reevaluating numerous different CNN architectures and configurations, a select group that has already proven to be effective will be utilized. It would be impractical to re-evaluate all combinations of architectures and configurations, as those that did not perform well in the initial assessment are unlikely to suddenly achieve better results in this section.

However, this alone does not constitute classification; it only determines whether a pair of images belongs to the same class and with what confidence. The objective is to take advantage of this similarity calculation to perform effective classification tasks.

To achieve this goal, the following approach was employed:

As discussed previously, the siamese network will be trained using pairs of images along with their corresponding labels. Once trained, this network will output a value between 0 and 1. A value closer to 0 indicates a higher likelihood that the images belong to the same class (based on the learned similarity), while a value closer to 1 indicates a higher likelihood of belonging to different classes.

To perform a classification, an image to be classified will be entered. Two instances of the Siamese network will be defined, each with two inputs. This results in a total of 4 inputs for classification. Specifically, the first input for each siamese network will be the image to be classified. The second input for each network will be a reference image that represents each class (OK/NOK). Thus, each siamese network will predict the similarity between its reference image(s) and the incoming image to be classified.

Therefore, each siamese network will output a similarity score between the incoming image and its respective reference image. The network that outputs a score closest to 0 (indicating high similarity) is more likely to suggest that the incoming image belongs to that class (OK or NOK). However, using only one reference image per class could yield misleading results, as there might be images within each class (OK or NOK) that are not very similar to each other.

To address this, a small group of representative images have been selected to best represent each class. The similarity between the incoming image and each of these reference images is predicted. Subsequently, the similarity score for each of the reference images (OK and NOK) is computed.

The incoming image will be assigned to the class in which the smallest similarity value is found between this incoming image and the reference images. This approach is used because within a single class, there may be different clusters; an incoming image might be very similar to one

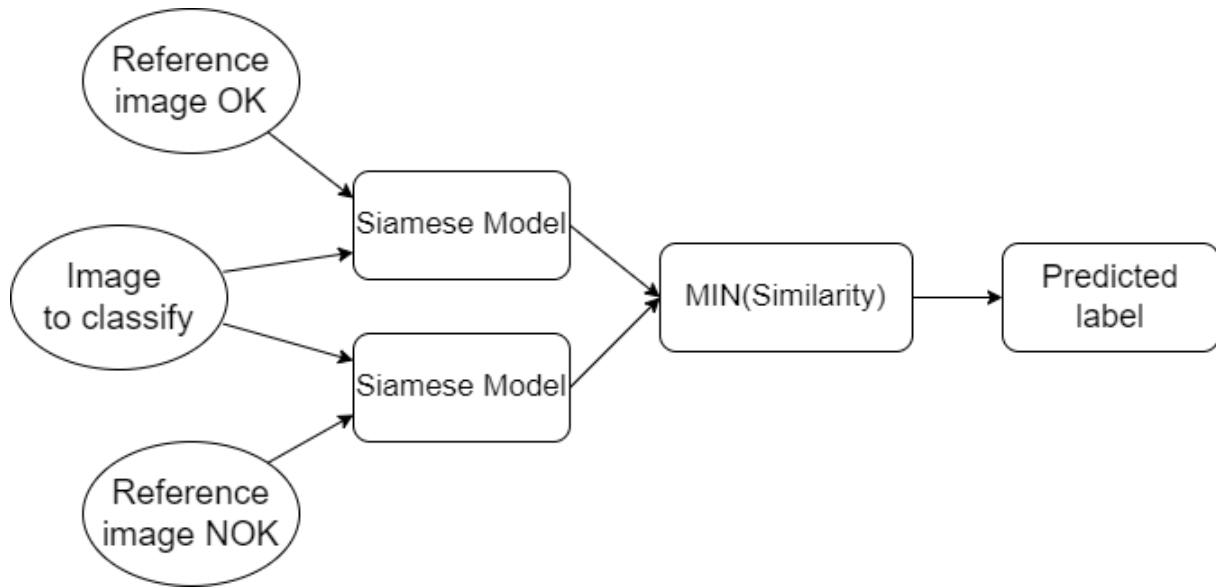


Figure 4.13: CL Classifier [Own Image]

reference image but not as much to another. Using the average could obscure such nuances. Therefore, the classification is done in such a way that if the image is highly similar to any one of the reference images—and this similarity is the lowest among all classes—it will be sufficient to assign the image to that class.

This method ensures that the image is accurately classified based on its most comparable reference, taking into account the variability within classes.

In this section, data containing the specifications for the simulations will also be read from a YAML file. Instead of conducting a systematic study of the different architectural or configuration possibilities, different models will be tested based on the results obtained in previous sections, more specifically the Deep Learning section. Therefore, the YAML file will contain only two main keys, which will be the following:

- Evidently Cloud Data: As in the two previous sections, connection data to Evidently will be included. Again, the only difference in content compared to previous sections is the project in which the reports and test suites will be stored within Evidently.
- Models: The other key will be for storing the data for constructing the models. This information will include:
 - Architecture specification: An array that will define the layers of the neural network, which in this case will serve as an encoder. This array will be constructed in exactly the same manner as described in the systematic study of Deep Learning. This ensures consistency in the network architecture across different sections of the project, allowing for direct comparisons of results and efficient reuse of the established neural network configurations.
 - Architecture Name: The name of the architecture that will be tested.
 - Kernel Widths, Filters and Dropouts: In these three different fields, the values desired for these parameters in the models to be generated will be specified. These can take exactly the same values as those indicated in the first section of the solution design.

This alignment ensures that the models maintain consistency in terms of parameter settings across different parts of the project, facilitating a coherent approach to model evaluation and comparison.

These simulations will involve testing 24 different architectures (all with the same configuration parameters), including those that achieved the best results in the first section. The 24 architectures will be varied, meaning that models with different numbers of convolutional and dense layers will be tested to see how they impact the results. This will result in 6 architectures for each number of convolutional layers and 8 architectures for each number of dense layers.

For example, a real case from the yaml file would be: (See Figure 4.14)

```
models:
  model11:
    arch: ['C', 'C', 'N', 'P', 'C', 'flatten', 'F', 'D', 'F', 'D', 'F', 'D']
    arch_name: architecture_3conv_3dense_24
    kernel_widths: '50_cte'
    filters: '32_cte'
    dropouts: '0.5_cte'
    test_suite_thresholds: [0.6, 0.65, 0.7, 0.75, 0.8]
```

Figure 4.14: CL YAML [Own Image]

To manage reports and test suites, the same strategy as in the previous sections will be followed. The model that achieves the highest F1-Score among the 50 will be selected as the representative for the architecture. Its weights, which have been previously saved, will be loaded, and a prediction will be made on the gold standard set to obtain the metrics that will be definitive for the architecture in question.

Once we understand how the system works, for the third and final time, we can view it in its entirety on Figure 4.15.

In this case, we have a system very similar to the one described in the first section. This is because, in this scenario, the architecture of the model to be built and the configuration parameters of the different layers are also specified. These values from the YAML file are read and processed by a dedicated class to convert them into the appropriate format so that the model creator can call the constructor and thus build the corresponding tuple for each model. This tuple will contain the model itself built with Keras, the name of the architecture used, tags, metadata, and thresholds for the tests.

These tuples are what will reach the Simulator for the simulation to be executed, and locally stored files with the performance records of the models, and in the cloud, the reports and test suites that are uploaded to Evidently.

As in the previous sections, the same established methodology will be followed. Cross-validation with five splits will be used, and these will be repeated 10 times. The number of repetitions and splits to simulate will be specified again from the command line when launching the script with the simulation. Regarding the file from which to read the data, as there are not too many models to test in this case, it is not necessary to split them into several files, and consequently, it is not necessary to pass as a parameter from which file to read. In the event that more models were to be tested and separated into different YAML files, the file from which to read the data could be indicated in the same way as in the previous sections.

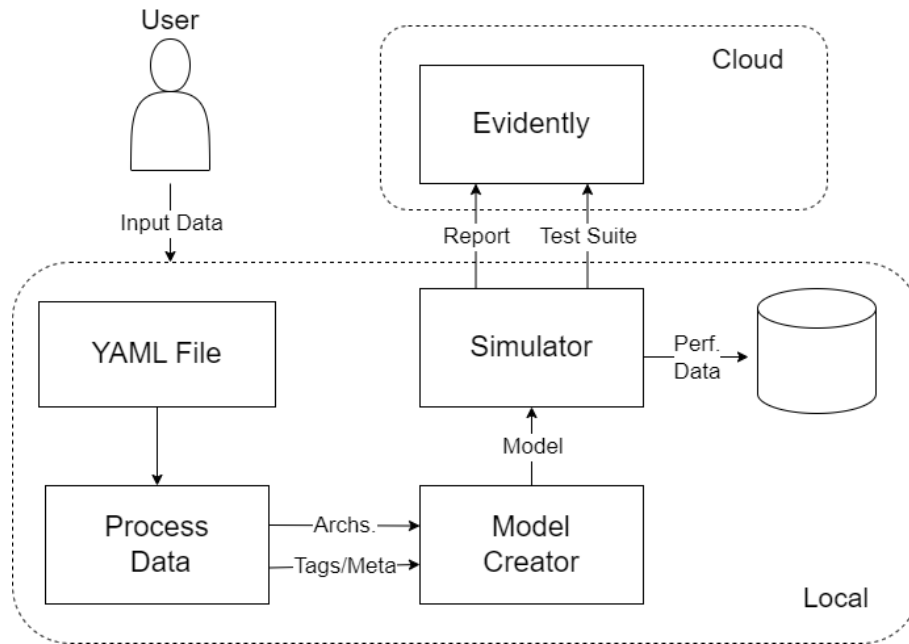


Figure 4.15: CL System [Own Image]

In this case, it will only be necessary to launch a single script, and it can be done in the following manner:

```
1 nohup python3 SIM_CL.py -r 10 -s 5
```


5 Results

In this section, the results obtained from the simulations and experiments conducted with the deep learning models developed are presented and analyzed. The structure of this section is organized to provide a clear and detailed view of the performance of each proposed approach.

First, the results of deep learning models are discussed, where various configurations of CNNs are evaluated. The findings related to transfer learning are presented next. Subsequently, the performance of the contrastive learning approach is analyzed.

Each subsection includes graphs, tables, and evaluation metrics that illustrate the performance of the models, comparing their accuracy, recall, F1-Score, and other relevant metrics.

5.1 Deep Learning

In this first section of the results, let us first recall the vast number of scenarios that have been simulated. According to the simulation plan outlined in the Deep Learning section, for the topology, generating all possible combinations results in 432 different architectures to evaluate. Regarding configurations, there are two possible dropout probability values, two possible filter number values, and five possible kernel sizes. Through combinatorial calculation, this totals 20 configurations to test in total. As the goal is to simulate as many scenarios as possible, each architecture must be paired with each configuration. This results in 432 architectures multiplied by 20 configurations, yielding 8640 distinct scenarios to simulate. For these scenarios, traceability and reproducibility are ensured by storing all metrics from each evaluation and saving the *.keras* models themselves.

Once everything was set up, the simulation was commenced as outlined in the design section.

After some time and having tested a considerable number of models (configuration + architecture), it was observed that there was little variability within the same architecture when varying the configuration. This observation raised doubts about whether it was worthwhile to test so many configurations for the same architecture, especially since the observed variations were not significant.

In the interest of time and resource management, a decision was made to simulate several architectures with markedly different variations, such as architectures with 3 convolutions, 10 convolutions, with and without dropout layers, with and without pooling layers, etc.

In total, 442 combinations of configurations and architectures were simulated, which roughly equaled the testing of around 20 configurations with notable variations between them. These 442 models were taken as a sample to extrapolate the results to the rest of the models. This approach aims to determine whether it is worthwhile to test so many different configurations or whether focusing on distinct architectural variations provides sufficient insight.

At this moment, to conduct an initial analysis, one can study the variation of the F1 score based on the architecture given certain fixed values of the kernel width and the dropout rate.

Let's first visualize just one of the cases (See Figure 5.1), specifically the case with a kernel width of 50 and a dropout rate of 0.5. In this graph, the x-axis displays the different architectures sampled in this initial analysis. The names of the architectures indicate the number of convolutional and dense layers each architecture possesses. Within each combination of layer

numbers, there are 36 distinct models. On the y-axis, we can see the average F1 score of the various configurations for each architecture.

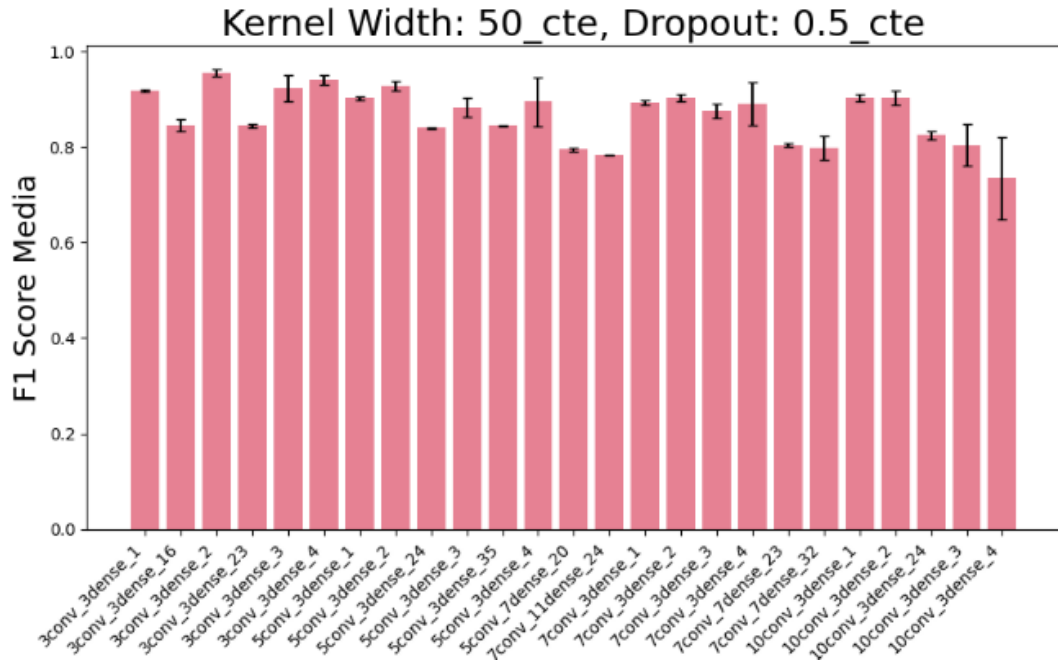


Figure 5.1: Architecture Comparaision 1 [Own Image]

Now if we see Figure 5.2, it is noticeable that the distribution of F1 scores is quite homogeneous, although there are a few valleys of performance in certain architectures. It is also evident at first glance that the architectures with fewer convolutional and dense layers exhibit lower deviation, indicating that their behavior is more predictable. This graph provides a visual representation that helps in understanding how architectural complexity influences model stability and performance in terms of F1 scores.

Having analyzed the graph, we can now proceed to explore additional combinations of fixed values to see if there are clear trends in the distribution of the F1 score (See Figure 5.2). This further analysis will help determine if certain architectures consistently perform better or worse.

Upon analyzing the performance graphs of various CNN architectures, it is observed that all exhibit a very similar general shape and distribution. In each configuration of kernel width and dropout, the average F1 scores are high, generally exceeding 0.8, indicating reasonable performance across all evaluated architectures. This uniformity in the shape and distribution of the graphs suggests that, regardless of variations in hyperparameters, the overall performance of the architectures remains within a reasonable and consistent range.

A notable trend is that architectures with a larger number of convolutional and dense layers tend to show more consistent F1 scores and less variability. However, upon closer examination, the 3conv_3dense_2 architecture stands out as the most resilient. This architecture presents a quite high average of F1 scores and minimal standard deviation across multiple kernel and dropout configurations. This combination of high average and low variability indicates that 3conv_3dense_2 is especially robust and reliable, consistently maintaining superior performance.

In summary, architectures initially display varying performance among themselves, indicating that topologies are a significant determinant of overall model performance. This suggests that the structural configuration of the networks, including the number and types of layers used, critically influences how effectively a model can learn and generalize from the data.

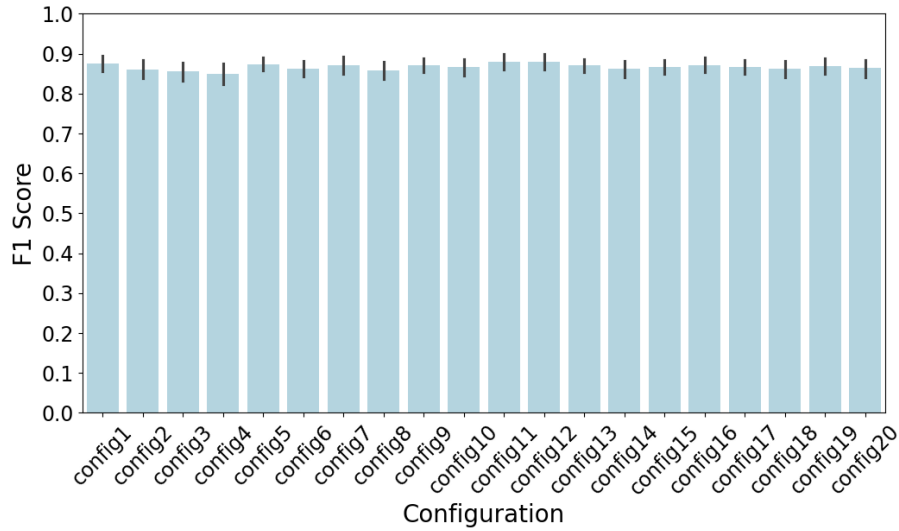


Figure 5.3: Compare 20 configurations [Own Image]

urations.

Based on these results, it was proposed to quantitatively analyze the statistical difference that may exist between the performance that one configuration or another can offer. If two configurations produce performances that could be considered from the same statistical distribution, it does not make much sense to continue simulating both, and pruning should be considered. This would allow for a reduction in the time and costs of the project.

An analysis was performed using a Jupyter notebook to perform a statistical examination of the results to determine if there were significant statistical differences or if the observed variations could be attributed to chance. The analysis performed was a t test, also known as the Student t-test, which is used to discern whether two samples come from populations with the same mean. This test will help us determine whether the results of one model and those of another belong to the same population, that is, whether they follow the same distribution of results.

The Student's t-test requires certain conditions to be met in order to be performed. These conditions are: [38]

- Independence: The observations must be independent of each other.
- Normality: The populations being compared must be normally distributed.
- Equality of variance (homoscedasticity): The variance of both populations being compared must be equal.

To perform the test, we need to verify these conditions for our data. For example, in the case of the dropout rate, there are two possible values: a rate of 0.3 and a rate of 0.5. There are 220 registers with a rate of 0.3 and 222 registers with a rate of 0.5. We will analyze whether they belong to the same population or not (See Table 5.1).

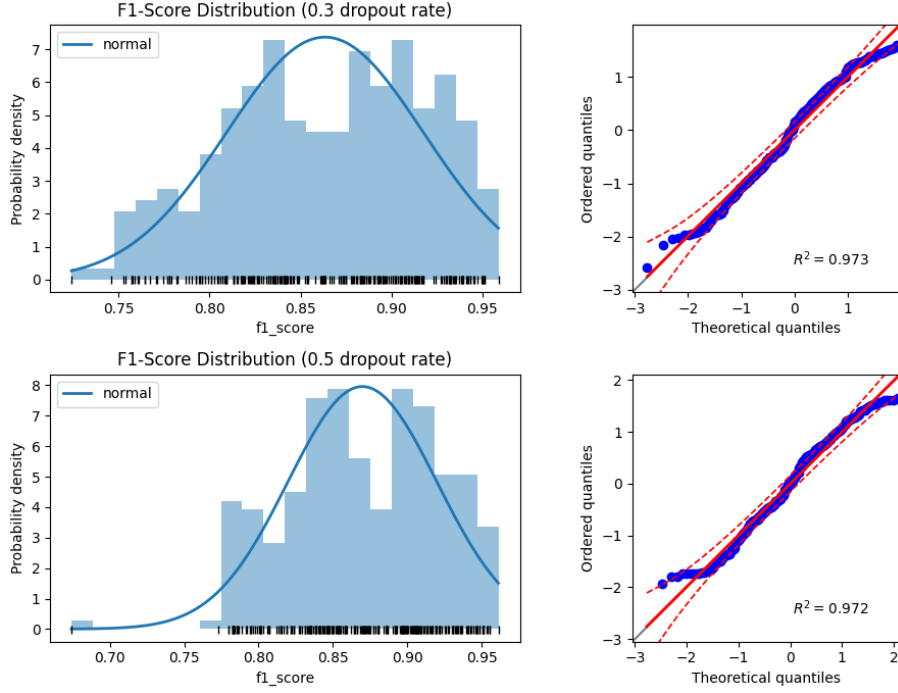


Figure 5.4: Q-Q Plot [Own Image]

dropouts	W	pval	normal
0.5_cte	0.970983	0.000060	False
0.3_cte	0.969541	0.000041	False

Table 5.1: Shapiro-Wilk Test's Results

Quantile-quantile (QQ) plots (see Figure 5.4) show asymmetry, and the Shapiro-Wilk test provides significant evidence that the data do not come from normally distributed populations. However, given that the size of each group is greater than 30, the t-test can still be considered sufficiently robust. Given that the normality criterion is not met, one of the recommended tests to check the equality of variances is the Levene test [72].

	W	pval	equal_var
levene	2.535367	0.111963	True

Table 5.2: Levene's Test Results

No significant evidence (for $\alpha = 0.05$) was found (See Table 5.2), implying a confidence level 95% that the variances are different between the two populations.

Once the conditions have been checked, we are ready to perform the Student's t-test, which was implemented using the `ttest` function from the Pingouin package [82]. This function returns the p-value, confidence intervals, and effect size.

• Hypothesis

- H0: There is no difference between the population means.
- H1: There is difference between the population means.

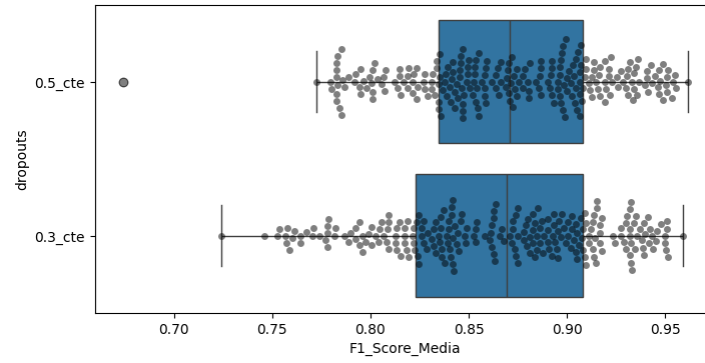


Figure 5.5: Dropout Rate Scatter/Box Plot [Own Image]

	T	dof	alternative	p-val	CI95%	cohen-d	BF10	power
T-test	-1.357573	492	two-sided	0.175222	[-0.02, 0.0]	0.122161	0.245	0.273034

Table 5.3: T-Test Results for Dropout Rate

Given that the p-value (0.175222) is greater than the significance level alpha (0.05) as can be seen on Table 5.3, there is not enough evidence to conclude that there is a real difference in performance between the models tested with a dropout rate of 0.3 and those tested with a rate of 0.5, so we can refuse the alternative hypothesis. The distribution of the F1 score of the models depending on the dropout rate can be visualized in Figure 5.5

This process is repeated for the rest of the categorical variables that stem from the configurations, and these are the results of the distribution of the f1 score and the hypothesis tests for the number of filters and for the kernel size variables.

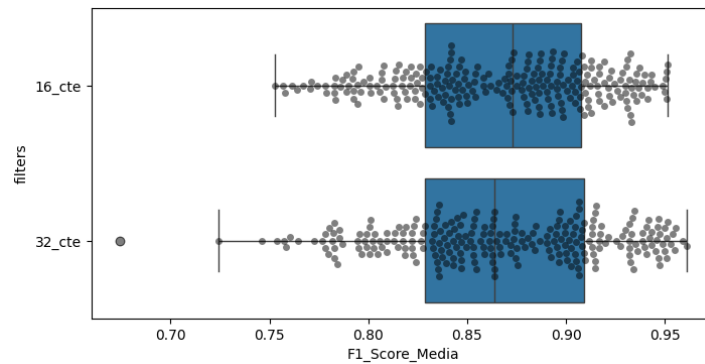


Figure 5.6: Number of Filters Scatter/Box Plot [Own Image]

• Number of Filters:

In this case (See Table 5.4 and Figure 5.6), for the number of filters, the p-value (0.713405) is also greater than the significance level. So, the alternative hypothesis can be rejected once again.

	T	dof	alternative	p-val	CI95%	cohen-d	BF10	power
T-test	0.367523	440	two-sided	0.713405	[-0.01, 0.01]	0.034964	0.113	0.065544

Table 5.4: T-Test Results for Number of Filters

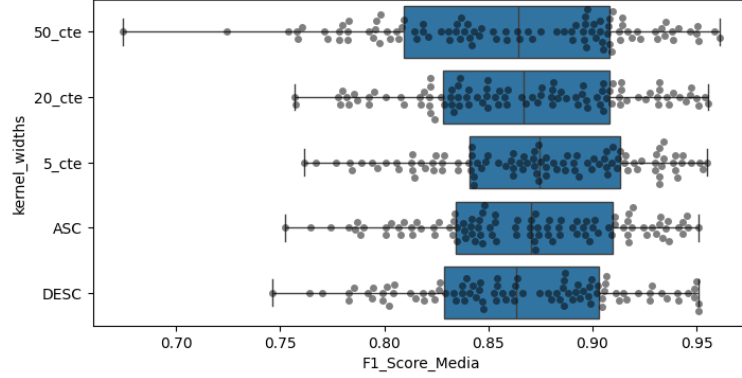


Figure 5.7: Kernel Width Scatter/Box Plot [Own Image]

• Kernel Width:

Group1	Group2	T-statistic	p-value
50_cte	20_cte	0.773393	0.440293
50_cte	5_cte	1.978267	0.049424
50_cte	ASC	1.084496	0.279612
50_cte	DESC	0.946527	0.345223
20_cte	5_cte	1.326034	0.186509
20_cte	ASC	0.336238	0.737087
20_cte	DESC	0.215552	0.829596
5_cte	ASC	1.012325	0.312772
5_cte	DESC	1.090528	0.277042
ASC	DESC	0.111790	0.911125

Table 5.5: T-Test Results for Kernel Width

Finally, for almost all these hypothesis tests conducted on the kernel width property, the p-value is greater than the significance level α (0.05) as can be seen on Table 5.5. Therefore, as in the previous case, we can reject the alternative hypothesis. The only p-value lower than the significance level is between the kernel width values of 50 constant and 5 constant, where the p-value is 0.049424. This result is expected because it makes sense that there would be statistically significant differences between the maximum and minimum kernel sizes in the simulation plan.

Thus, it was concluded that it was reasonable to reduce the number of configurations to simulate with the architectures, as it has been demonstrated that many configurations yield results belonging to the same population, which does not provide additional information on new results. Therefore, we reduced from 20 configurations per architecture to only 2: one using a kernel width of 50 constant, and the other using a kernel width of 5 constant. This reduction resulted in a ten-fold decrease in computational and time load, representing a significant saving in both time and computational resources.

From now on, Configuration 1 corresponds to the one with a constant kernel width of 50, and Configuration 2 corresponds to the one with a constant kernel width of 5. For the other configuration values, the same settings were chosen: 16 filters and a dropout rate of 0.5. This choice was made because, as demonstrated, selecting one value over another is indifferent since there are no statistically significant differences.

Once this process was completed, which reduced the number of configurations to be simulated, we were able to start the simulation. The number of models (configuration + architecture) was reduced from 8640 to 864. The simulation was then carried out as outlined in the solution design, running to completion. Now, we will study the main results obtained for this section.

Next, we will present the model from the Deep Learning section that performed the best in predicting the gold standard set.

This model consists of Configuration 1 and the architecture with 10 convolutional layers and 11 dense layers, number 12. Using the notation from the simulation plan, its normalization type is Full, with five normalization layers. The pooling type is also Full, with 5 average pooling layers, and the dropout type is None, meaning that it has no dropout.

Of the 50 models generated with this combination, the one that performed the best in the test set was the first split of the seventh repetition. Thanks to the simulation methodology, we can read from the JSON file the detailed metrics for each model and verify those obtained by the best of the 50, which are as follows (See Table 5.6):

F1 Score	Recall	Precision	ROC/AUC
0.9305	0.9214	0.9398	0.9586

Table 5.6: DL Metrics Test Set

We can also display the confusion matrix with the mean results for the 50 models as can be seen on Figure 5.8:

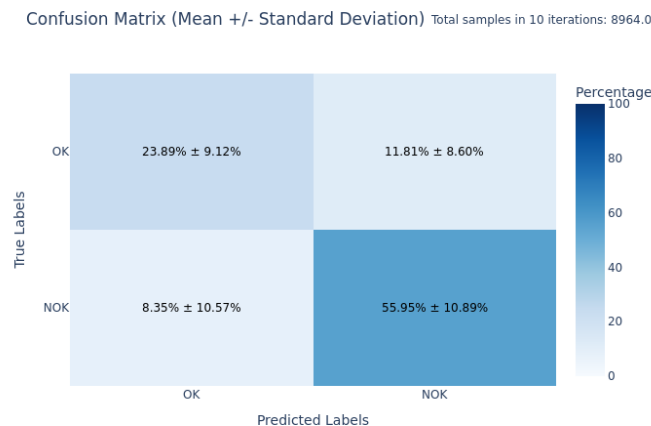


Figure 5.8: DL Mean Confusion Matrix Test Set [Own Image]

This model appears to perform pretty well in classifying coils between OK and NOK. As seen in the confusion matrix, approximately 80% of the images are correctly predicted at the ten repetitions. Additionally, all metrics exceed 0.9, indicating expected reasonable behavior of the classifiers.

Next, thanks to Evidently, we can access through the Cloud service and its user interface the specific report generated by the predictions of the best model among the 50. The report displays various types of graphs and tables regarding the model’s performance, including overall performance metrics (See Table 5.7), the confusion matrix (See Figure 5.9), class-specific metrics (See Figure 5.10), the ROC/AUC curve, and more.

Accuracy	Precision	Recall	F1	ROC AUC
0.7	0.643	0.9	0.75	0.763

Table 5.7: DL Metrics Gold Standard Set [Own Image]

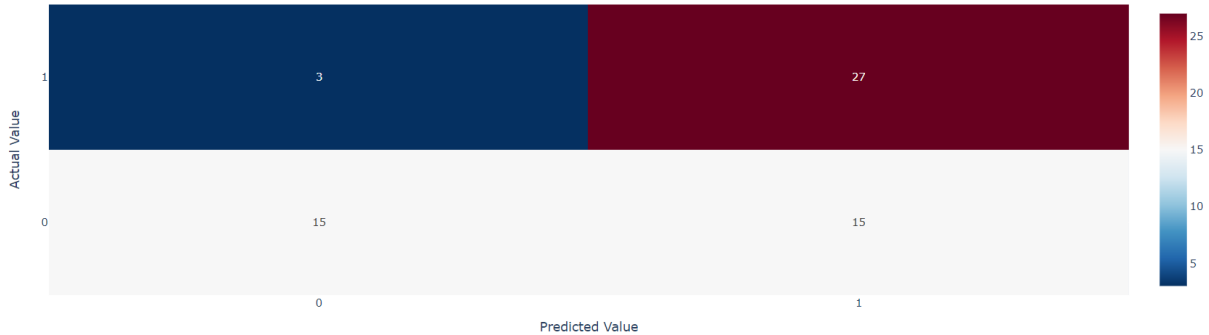


Figure 5.9: Confusion Matrix Gold Standard Set [Own Image]



Figure 5.10: Metrics per Class Gold Standard Set [Own Image]

The results show a decent level of performance in several key metrics. To begin with, the model achieved an accuracy of 70%, which means that it correctly predicted the outcome 70% of the time. This is complemented by a precision score of 64.3%, indicating that of all the positive predictions, 64.3% were true positives. Furthermore, the model has a high recall rate of 90%, reflecting its effectiveness in identifying 90% of actual positive cases. The F1 score, which balances precision and recall, is solid 0.75, highlighting a reasonable balance between these two important metrics.

Furthermore, the model’s ability to differentiate between positive and negative classes is demonstrated by an ROC/AUC score of 0.763. This score suggests that the model performs significantly better than random guessing. However, the LogLoss value of 0.692 points to a moderate level of certainty in the predictions. Ideally, we would like a lower LogLoss value, which would indicate higher confidence in the model’s predictions.

When we examine the metrics per class, we observe some variability. For the first class, the model achieved a high precision of 83.3%, but the recall was only 50%, indicating that while most positive predictions were correct, the model missed many actual positive cases. On the other hand, the second class had a high recall of 90%, but a lower precision of 64.3%, suggesting

that although the model captured most of the actual positives, there were quite a few false positives.

Once the model that maximized the F1 score on the gold standard set has been examined, we can analyze which parameters tested in the simulation improve or worsen the model's performance when predicting on the gold standard set.

Firstly, regarding which of the two configurations performs better, there is no great difference between config 1 and config 2 as can be seen in Figure 5.11. But config 1 still performs a bit better on the gold set. The model that maximizes F1-Score uses config 1.

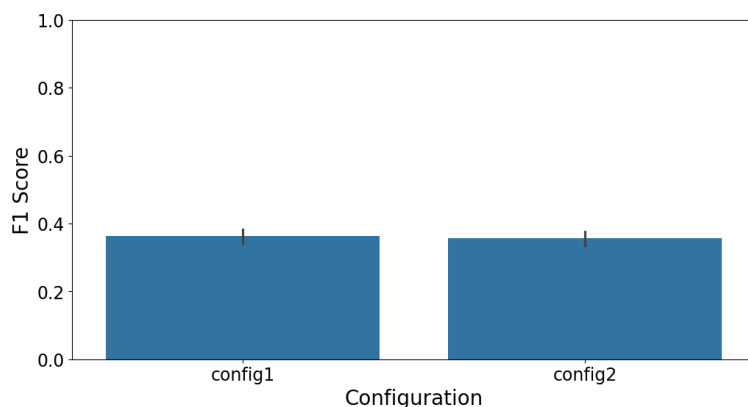


Figure 5.11: Compare Configs [Own Image]

Afterwards, we can continue analyzing how architectures are structured to observe their impact on the results obtained. Among these aspects to study, according to the simulation plan, are the types of normalization, pooling, and dropout. Regarding normalization, it will be None if there are no normalization layers, Partial if there is only one normalization layer after the last convolution, and Full in two scenarios, specifically when normalization occurs after each or every two convolutional layers. For pooling, it will be None if there is no pooling, Partial if there is only just before the flatten layer, and Full if pooling is performed after each normalization. Dropout will be None if there are no dropout layers, Partial if a dropout layer is added every 3 dense layers, and Full if dropout is used after each dense layer. Knowing this, we can understand and discuss the following graphs.

In these graphs, one can appreciate the performance of the models based on the type of pooling, normalization, and dropout that they use.

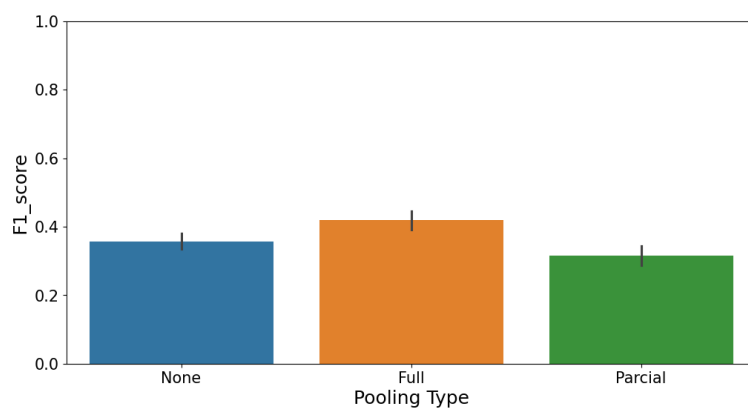


Figure 5.12: Pooling Type [Own Image]

Regarding the type of pooling, in Figure 5.12 it is clear that the partial pooling type on average achieves better results, despite, for example, the best model having a 'Full' pooling type. In this case, there are also notably more samples with Full pooling due to the nature of the architecture structure.

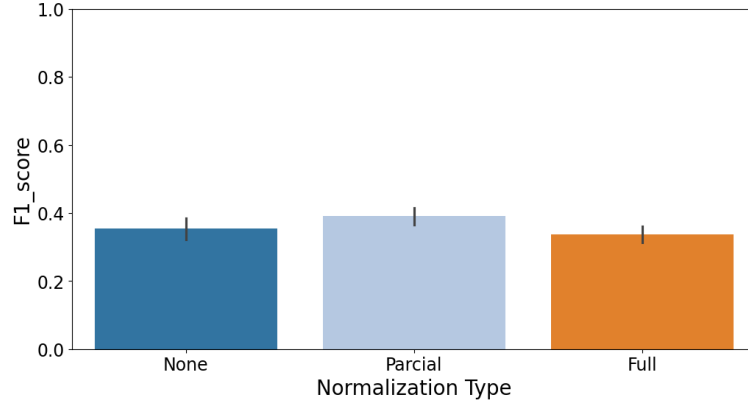


Figure 5.13: Normalization Type [Own Image]

Regarding the type of normalization, as seen in Figure 5.13, the type that maximizes the F1-Score on average is Partial, which means using a single normalization layer after the last convolutional layer helps improve the model performance. It is also worth noting that in this case, being Full in two scenarios rather than one like the rest, there are around 100 more samples of the Full type compared to the others.

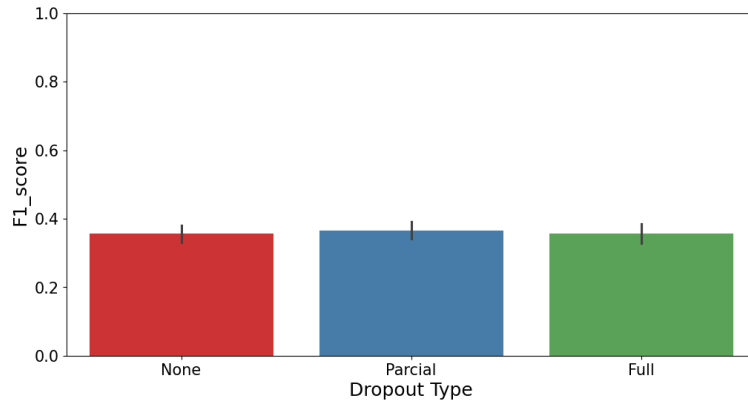


Figure 5.14: Dropout Type [Own Image]

Finally, in relation to the type of dropout, in Figure 5.14, there are minimal differences between one type and another, since we are dealing with differences in the hundredths. This suggests that there are no significant differences between the use of one type of dropout or another. For example, the model that achieved the highest F1-Score does not use dropout layers and yet does not appear to suffer from overfitting.

Apart from these variables, we can observe how the broader divisions outlined in the simulation plan, the number of convolutional layers and dense layers, affect the model's performance on the gold standard set. Recall that architectures could have 3, 5, 7, and 10 convolutional layers, while for dense layers, there could be 3, 7, and 11.

It can be seen from the graph on Figure 5.15 that the highest F1 score is achieved on average using 11 dense layers. This may be reasonable because each additional layer allows the network to progressively learn higher-level features and abstractions, potentially capturing more nuanced

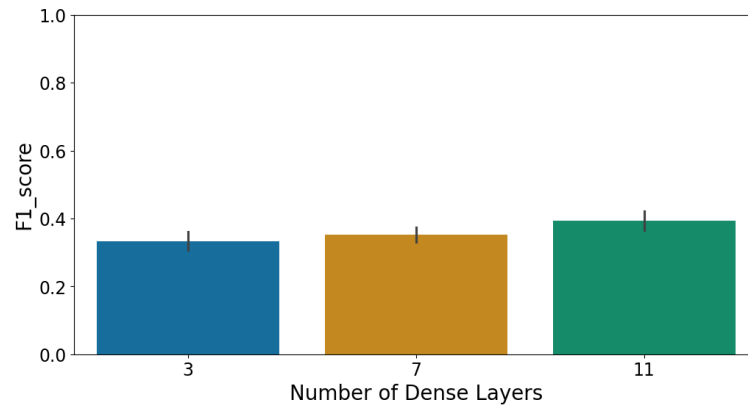


Figure 5.15: Dense Layers Number [Own Image]

relationships within the data. This increased depth enables the model to handle more complex datasets that may exhibit hierarchical or nonlinear relationships between input features and output predictions.

However, let us delve deeper into this. If we look at the same graphs regarding the number of dense layers but for precision and recall, we can better understand what happens in this case.

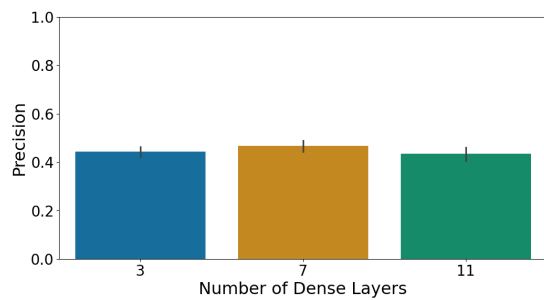


Figure 5.16: Dense Precision [Own Image]

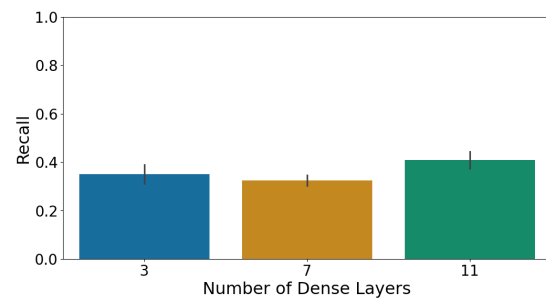


Figure 5.17: Dense Recall [Own Image]

In these two graphs, it can be observed that, on the one hand, precision increases with the number of dense layers, particularly peaking at seven fully connected layers. This indicates that with seven layers, more positive cases are predicted correctly compared to other numbers of layers. On the other hand, in the recall graph, there is a significant increase with 11 dense layers and a notable decrease with 7 dense layers. This suggests that using 11 fully connected layers greatly enhances the model's performance in predicting positive cases. The F1 score, which is the harmonic mean of precision and recall, shows that a higher F1 score is achieved with 11 dense layers because the significant increase in recall compensates for any slight loss in precision. Therefore, another metric that could further elucidate the situation is the ROC/AUC curve.

In this final graph it is clear that the number of dense layers that most positively impacts the model in distinguishing between positive and negative cases is 11. This confirms that the initial result obtained with the F1 score was accurate, despite having slightly lower precision compared to other numbers of dense layers.

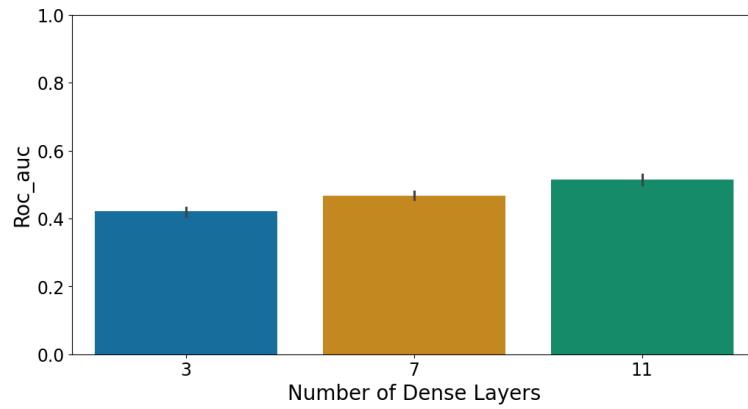


Figure 5.18: Dense ROC/AUC [Own Image]

Now, we will analyze the influence of the number of convolutional layers on the model's performance.

For this analysis, we will visualize the five metrics defined in the Evaluation metrics section: accuracy, precision, recall, F1-Score, and ROC/AUC.

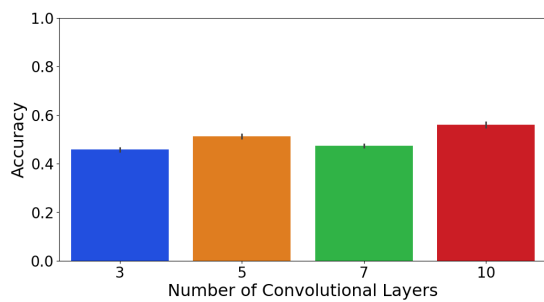


Figure 5.19: Convolutions Accuracy [Own Image]

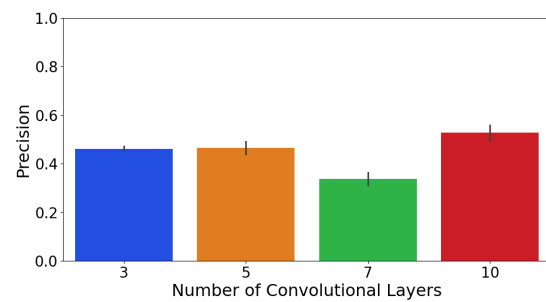


Figure 5.20: Convolutions Precision [Own Image]

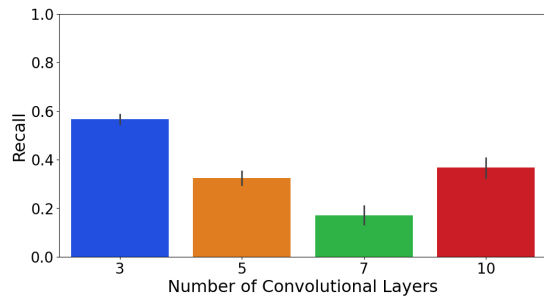


Figure 5.21: Convolutions Recall [Own Image]

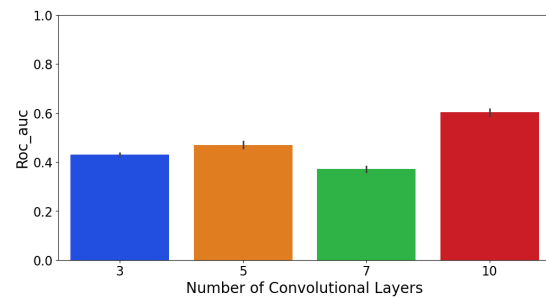


Figure 5.22: Convolutions F1 Score [Own Image]

Firstly, looking at the accuracy graph on Figure 5.19, we observe that accuracy increases with the number of convolutions. This suggests that increasing the number of convolutions helps the model capture more features and patterns in the data, thereby improving its classification capability. Moving on to the precision graph on Figure 5.20, it also improves with more convolutions, peaking at 10 convolutions. Increasing the number of convolutions seems to help the model avoid false negatives, which is what this metric reflects. Regarding the recall graph on Figure 5.21, we notice the first change in trend in terms of performance. The high value with 3

convolutions suggests that the model is initially good at identifying positive instances, although at the expense of more false positives. The drop in recall with 5 and 7 convolutions indicates that the model is failing to identify many positive instances, and the increase in recall with 10 convolutions suggests a better ability of the model to capture positive instances, although it is still not optimal compared to the precision for this same number of convolutions.

Regarding the F1-Score graph on Figure 5.22, it follows the same trend as the recall, and its behavior reflects the combined trend of recall and precision. The significant drop in the F1-Score with 5 and 7 convolutions indicates that neither precision nor recall is adequate in these models, reflecting overall poor performance. The improvement in the F1-Score with 10 convolutions indicates a better balance between precision and recall, although there is still considerable room for improvement.

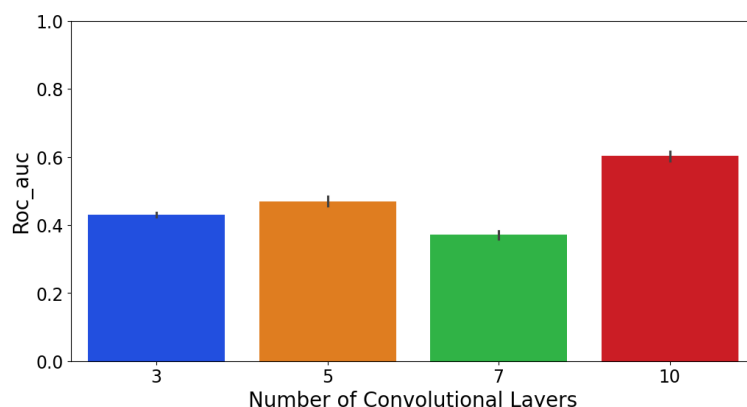


Figure 5.23: Convolutions ROC/AUC [Own Image]

Finally, to provide another perspective through a different metric, we focus on the ROC/AUC in relation to the number of convolutions, which measures the model's ability to distinguish between classes.

For this metric, it is clearly observed on Figure 5.23 that the number of convolutions yielding the best performance is 10. This may be due to more complex models having a greater capacity to capture distinctive features between classes, despite a drop in performance with 7 convolutions compared to 5.

Now, let us proceed with the final conclusions regarding the last variable under analysis. In general, a model with 10 convolutions provides the best balanced performance in terms of accuracy, precision, and discriminatory power (ROC/AUC). For models with 3 convolutions, although these models have a high recall, their overall capability is inferior to that of models with 10 convolutions. This indicates that while they can identify more positive instances, they also generate more false positives. Lastly, models with 5 and 7 convolutional layers show a notable decline in various metrics, especially sensitivity and the F1-Score. This suggests fitting issues that affect the model's ability to generalize correctly.

To conclude this section, we can analyze the influence of the number of convolutional layers and the number of dense layers on the same graph to try and visualize the two trends together. (See Figure 5.24)

In the histogram for architectures with 3 convolutional layers, it is observed that most configurations of dense layers have F1 scores in the range of 0.4 to 0.6, with the majority clustering around 0.5. This indicates that while there is variability in performance, most configurations achieve moderate F1 scores.

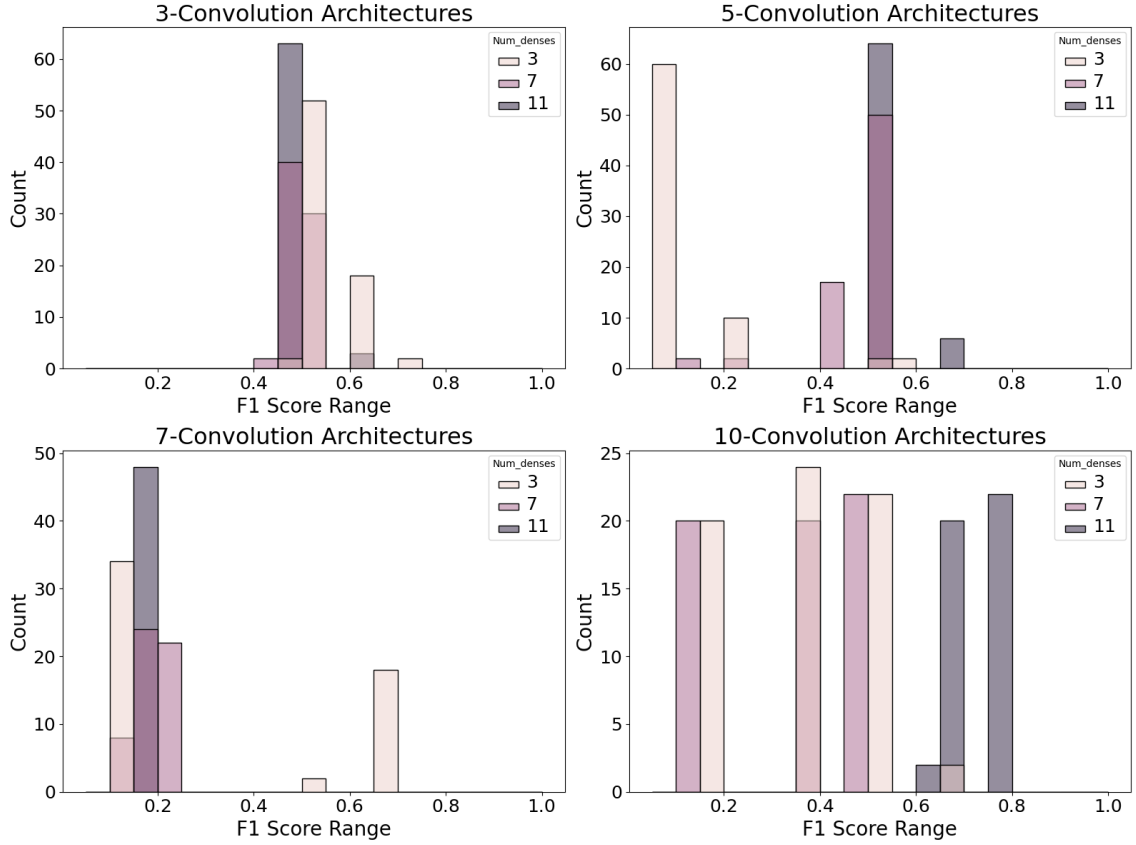


Figure 5.24: Compare Conv/Dense [Own Image]

The histogram for architectures with 5 convolutional layers shows a greater dispersion in F1 scores. Configurations with 3 dense layers tend to have lower scores, with a concentration around 0.2. On the other hand, configurations with 7 and 11 dense layers have a broader range of scores, extending from 0.2 to 0.6.

In architectures with 7 convolutional layers, a trend similar to that with 5 layers is observed. Configurations with 7 and 11 dense layers tend to have lower F1 scores, around 0.2. It is interesting to note that in this graph, models with 3 dense layers have both the worst and the best results, showing a notable difference from configurations with 7 and 11 dense layers, which suggests that performance can be highly variable in these architectures.

The histogram for architectures with 10 convolutional layers shows a more uniform distribution of F1 scores compared to the other histograms. Configurations with 3 dense layers are concentrated around 0.4, while those with 7 and 11 dense layers have scores ranging from 0.2 to 0.8. Configurations with 11 dense layers, in particular, show a tendency to achieve the highest F1 scores, indicating that a greater number of both convolutional and dense layers may lead to superior performance.

Overall, it is observed that architectures with 3 dense layers tend to have lower F1 scores and are concentrated in the range of 0.2 to 0.4, while those with 7 and 11 dense layers tend to achieve scores in a broader range, especially in architectures with more convolutional layers. As the number of convolutional layers increases, the distribution of F1 scores becomes more uniform and less concentrated at low ranges. Architectures with 10 convolutional layers show the widest and most uniform distribution, suggesting that a greater number of convolutional layers can enhance the consistency of performance.

This analysis agrees with what was studied from the previous single-variable graphs where it could be seen that more convolutional and dense layers help improve the performance and robustness of the model.

5.2 Deep Transfer Learning

For this second part of the results, let us once again recall the various scenarios to be simulated in order to study their impact on the final outcome. As indicated in the DTL Systematic Study, there are three parameters that must be varied in the models. These are the pre-trained model selected for extracting learning, the number of layers to freeze among the 10 available for allowing transfer learning, and the number of epochs during which the model is trained.

With 4 options for pre-trained models, 5 options for the number of layers to freeze, and 3 options for the number of epochs, this means that there will ultimately be 60 different models tested.

Once the plan is defined, the simulation can be set in motion as indicated in the solution design.

Upon completion of the simulation, the results obtained can be analyzed. Firstly, we will present the model that achieved the best performance on the gold standard dataset for this approach. This will be the model that obtained the highest F1-Score among all those tested.

This model uses the pre-trained model N, which is the smallest with the fewest parameters. Regarding the number of layers frozen to allow transfer learning, it uses 9 layers, which means that all possible layers are frozen. Lastly, regarding the number of epochs, it uses 100 epochs for training, which is the intermediate option in the simulation plan.

Among the 50 identical models generated, the one that obtained the best results and was thus chosen as the representative of that combination of values is in the second split of the first repetition. Once again, thanks to the methodology used for conducting the simulations and validation, we can obtain the metrics generated by this model when predicting on the test set corresponding to its split (See Table 5.8). To ensure reproducibility, the indices of these samples used for both training and evaluation are also stored in a JSON file along with their resulting metrics.

In this case, these were the results for the aforementioned representative model.

F1 Score	Recall	Precision	ROC/AUC
0.9726	0.9693	0.9759	0.9936

Table 5.8: DTL Metrics Test Set

As in the previous case, the confusion matrix can also be visualized on Figure 5.25 along with statistical data generated from the model predictions on the test sets of the 50 different splits.

This model seems to perform quite well in classifying the coils. Based on the information from the image, it can be inferred that the model correctly classifies between OK and NOK in about 95% of the cases in 50 cross-validation splits. As in the previous section, the metrics for this representative model all exceed 0.9, indicating once again that the model performs reasonably well in classification.

Now, we will analyze the results obtained on the gold standard set by this best model among the 60 simulated. This can be done through the Evidently Cloud service, which allows for

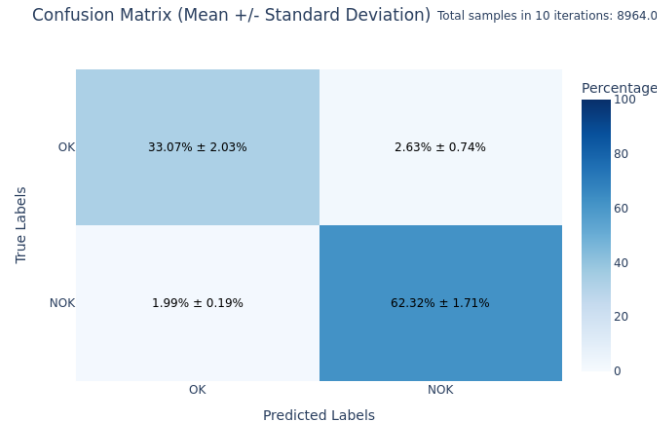


Figure 5.25: DTL Mean Confusion Matrix Test Set [Own Image]

visualization of graphs to understand the model's performance when classifying. It can be seen the global metris on Table 5.9, the confusion matrix on Figure 5.26 and th metrics per class on Figure 5.27

Accuracy	Precision	Recall	F1	ROC AUC
0.817	0.913	0.7	0.792	0.7853

Table 5.9: DTL Metrics Gold Standard Set [Own Image]

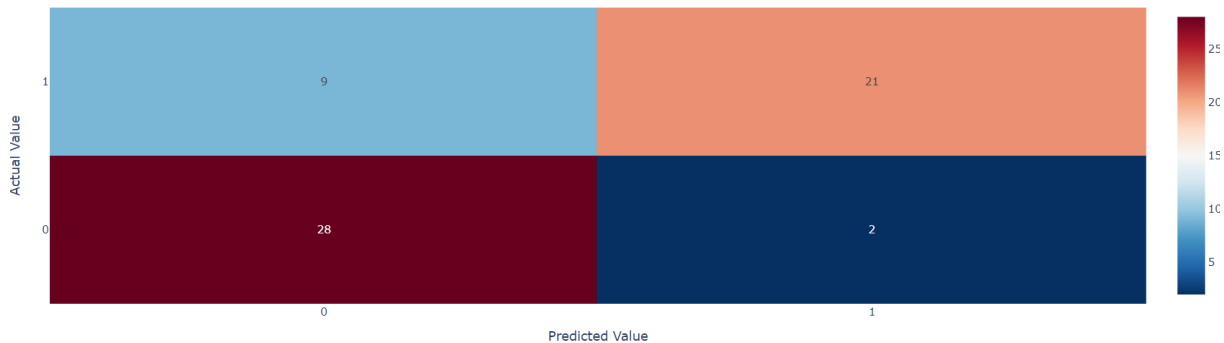


Figure 5.26: DTL Confusion Matrix Gold Standard Set [Own Image]

The model under review has an overall accuracy of 81.7%, indicating substantial effectiveness in correctly predicting both positive and negative instances. Additionally, a specific precision of 91.3% highlights that the model's positive predictions are largely reliable, a crucial aspect especially in applications where false negative errors can have serious consequences.

However, despite the high precision, the model has a general recall of only 70%. This suggests that it is failing to identify all true positives. The F1-Score of 79.2% suggests a balance between precision and recall, but also highlights the need to improve the detection of all relevant cases.

Moreover, the ROC/AUC of 85.3% provides a measure of the model's ability to effectively differentiate between classes, which is encouraging. This high value suggests that, overall, the model has good sensitivity and specificity. However, a LogLoss of 0.559, while indicating a decent level of reliability in the probabilities predicted by the model, also points to room for improvement in calibrating these probabilities.



Figure 5.27: DTL Metrics per Class Gold Standard Set [Own Image]

The confusion matrix shows that the model correctly predicts 28 true positives and 21 true negatives, consistent with the high precision observed. However, the presence of nine false positives and only two false negatives reinforces the idea that the model is efficient at avoiding critical errors (false negatives) but might be overestimating the presence of the positive class.

Regarding class-specific metrics, for class 0, the model has an excellent recall of 0.933, indicating a great ability to correctly identify true negatives. However, the precision for this class is 0.757, implying that there is a significant number of positive instances mistakenly classified as negative. In contrast, class 1 shows a very high precision at 0.913. However, a recall of 0.7 for this class suggests that the model does not identify a considerable proportion of true positives.

Having analyzed the model that performed best in the gold standard set, we can now examine how changes in the variables of the simulation plan affect the results obtained by the different models. This analysis will help identify which factors most significantly influence model performance and may guide further optimizations or adjustments in the simulation strategy.

Firstly, we will analyze the influence on the outcomes based on the chosen pre-trained model among those provided by YOLOv8. This analysis will explore how different pre-trained models impact the effectiveness of the transfer learning process and the overall performance metrics, such as accuracy, precision, recall, and F1 score. Understanding these differences is crucial for selecting the most appropriate pre-trained model for specific tasks and conditions.

Starting with accuracy, on Figure 5.28, the smallest model achieves the best score, indicating a better ability to correctly classify instances. As the model size increases, the accuracy tends to decrease, which might suggest that larger models are prone to overfitting due to the use of more parameters.

The smallest model, N, exhibits superiority in terms of recall, as can be seen on Figure 5.30, which implies a notable ability to identify positive cases. In contrast, larger models show better precision (See Figure 5.29), suggesting increased effectiveness in minimizing false positives. This balance between recall and precision is reflected in the F1-Score (See Figure 5.31), where model N emerges as the most balanced and effective, offering an optimal compromise between capturing true positives and avoiding false positives. Interestingly, this trend is consistent with the fact that model previously analyzed maximizes the F1-Score, providing the best overall performance as observed in the graphs.

Lastly, the ROC/AUC on Figure 5.32 remains quite stable across the models, with slight variations. This indicates that the models' ability to distinguish between positive and negative classes is not dramatically affected by the size of the model. The slight stability in ROC/AUC suggests that all models are reasonably good in terms of general discrimination, although they do not show significant improvements or deteriorations as they increase in size.

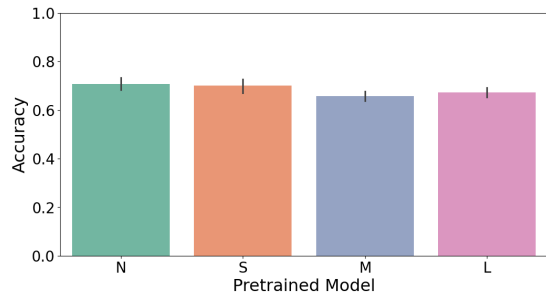


Figure 5.28: Pretrained Model Accuracy [Own Image]

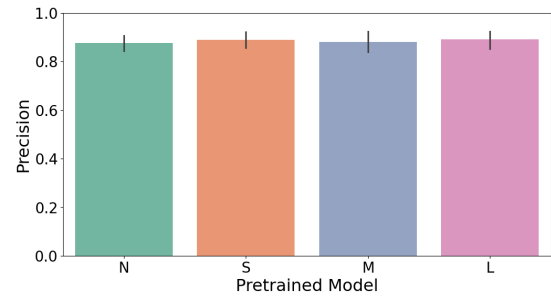


Figure 5.29: Pretrained Model Precision [Own Image]

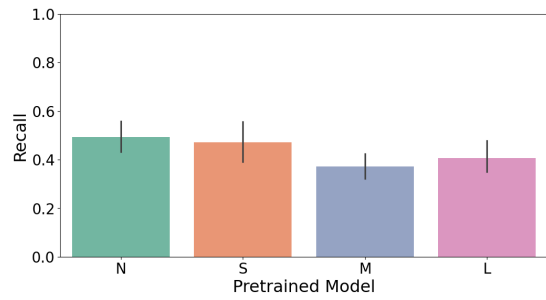


Figure 5.30: Pretrained Model Recall [Own Image]

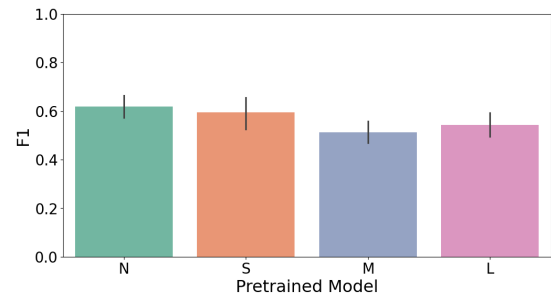


Figure 5.31: Pretrained Model F1 Score [Own Image]

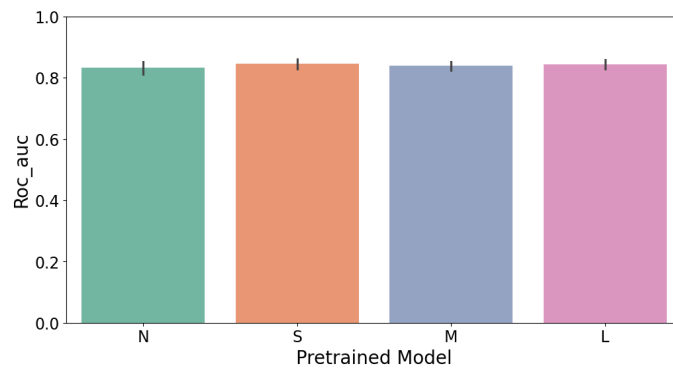


Figure 5.32: Pretrained Model ROC/AUC [Own Image]

We will now proceed to examine the second variable from the simulation plan, which is the number of layers in the model to freeze in order to preserve the weights of the previously mentioned pre-trained models. This approach consequently facilitates transfer learning, which is the essence of this section. This analysis will help determine the optimal number of layers to freeze to maximize model performance, balancing the benefits of leveraging learned features against the need for model adaptation to new tasks.

Examining how the number of frozen layers affects performance metrics in transfer learning models reveals clear and distinct trends that provide insights into the optimal model configuration depending on the specific goals of a classification task. This study focuses on models with 0, 3, 5, 8, and 9 frozen layers, assessing how each configuration impacts accuracy, precision, recall, F1-Score, and ROC AUC metrics.

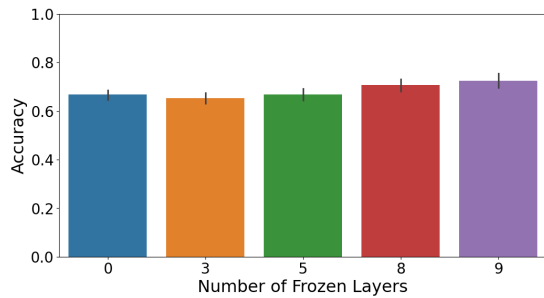


Figure 5.33: Frozen Layers Accuracy [Own Image]

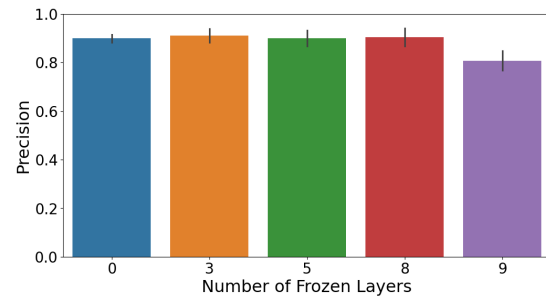


Figure 5.34: Frozen Layers Precision [Own Image]

Accuracy (See Figure 5.33), which reflects the proportion of correct predictions (both positive and negative) made by the model, shows a significant improvement as the number of frozen layers increases. This trend suggests that greater rigidity in the model's lower layers, by preserving more general learned features, possibly helps maintain stability in overall classification. Notably, freezing 9 layers leads to the highest observed accuracy, which may indicate that retaining prelearned knowledge across most of the model's layers helps maintain consistency in classification across different datasets or domains.

Precision, on Figure 5.34, contrasts with accuracy in terms of its behavior when adjusting the number of frozen layers. Although freezing 3 layers maximizes precision, increasing the number of frozen layers beyond this point results in a progressive decrease. This phenomenon could be explained by the fact that freezing more layers makes the model less flexible in adjusting to the specificity of new data, which could lead to an increase in the number of false positives.

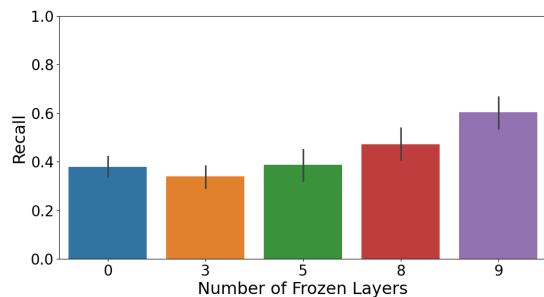


Figure 5.35: Frozen Layers Recall [Own Image]

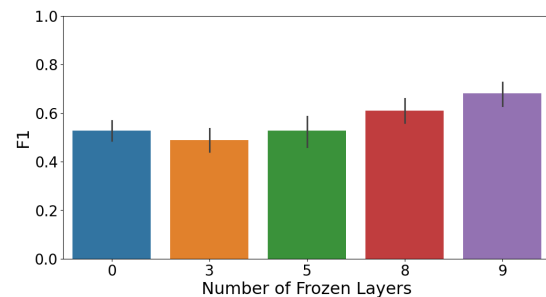


Figure 5.36: Frozen Layers F1 Score [Own Image]

Regarding recall on Figure 5.35, there is a very clear upward trend. As the number of frozen

layers increases, the model improves its ability to detect all relevant positive cases. This increase in recall is especially significant when 9 layers are frozen, suggesting that a more rigid model in its lower layers is more efficient in identifying positive cases, possibly because more general features are sufficient to capture the presence of these cases.

The F1-Score (See Figure 5.36) reaches its highest value when 9 layers are frozen. A high F1-Score indicates an adequate balance between the ability to detect positives and the precision with which these detections are made. Although precision decreases, the considerable increase in recall compensates for this decline, resulting in a model that, while not the most precise in terms of avoiding false positives, is highly effective in not overlooking positive cases.

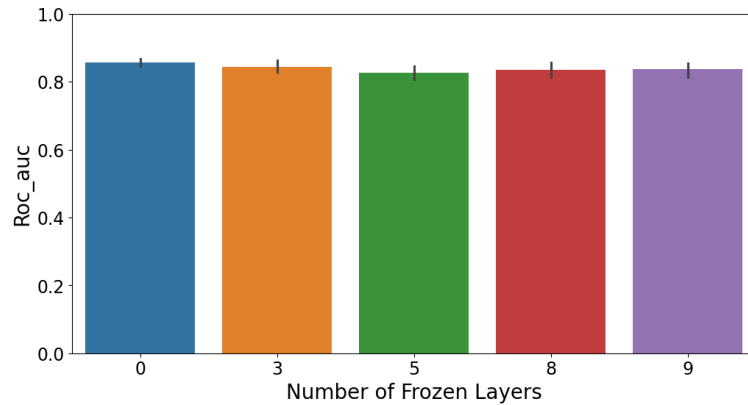


Figure 5.37: Frozen Layers ROC/AUC [Own Image]

Interestingly, the ROC AUC, on Figure 5.37, which assesses how well the model discriminates between classes across different thresholds, does not show significant improvements with an increase in frozen layers and tends to decrease slightly. This could indicate that although freezing more layers helps in certain specific metrics like recall, it does not necessarily improve the model's ability to classify between classes.

Finally, to conclude this section, we will examine the impact of varying the number of epochs on the training of the model. This analysis will help us understand how the duration of training influences the model's ability to learn and generalize from the training data, affecting metrics such as accuracy, precision, recall, F1-score, and ROC AUC. This exploration is crucial for optimizing the model's performance while ensuring efficient use of computational resources.

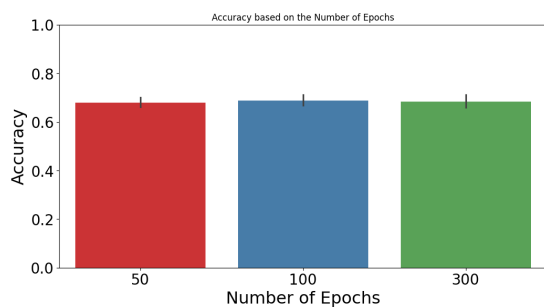


Figure 5.38: Epochs Accuracy [Own Image]

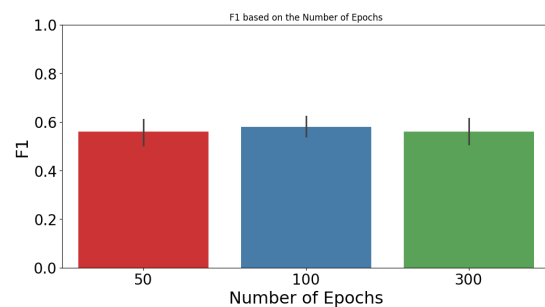


Figure 5.39: Epochs F1 Score [Own Image]

In this case, both the accuracy graph on Figure 5.38 and the F1-Score graph on Figure 5.39 show no significant differences between the results of varying the number of epochs. For the F1-Score, this also represents the joint trend between precision and recall, which, like these metrics, do not show substantial differences between the various results. However, it appears that the number

of epochs that have achieved the best results, albeit with minimal differences, is 100, which is the midpoint among the available options.

This trend, where there are no significant changes between different numbers of epochs, may be due to the fact that the best weights for each model are obtained early in the training process. This is largely due to the mechanism known as Early Stopping. This mechanism is designed to stop training if, over a number of epochs set by the user, there are no improvements in accuracy and validation loss. It is used to avoid wasting computational resources and time on training that does not improve the model's performance but, at best, stays the same or worsens. Thus, it can be demonstrated that the number of epochs, at least in this specific case with the particular dataset, is not as crucial a parameter in affecting outcomes as were the number of layers to freeze and the chosen pre-trained model previously.

After verifying that the number of epochs is not a decisive factor in influencing the model's performance, we can visualize the evolution of the F1 Score based on the two most relevant variables in a single graph. (See Figure 5.40)

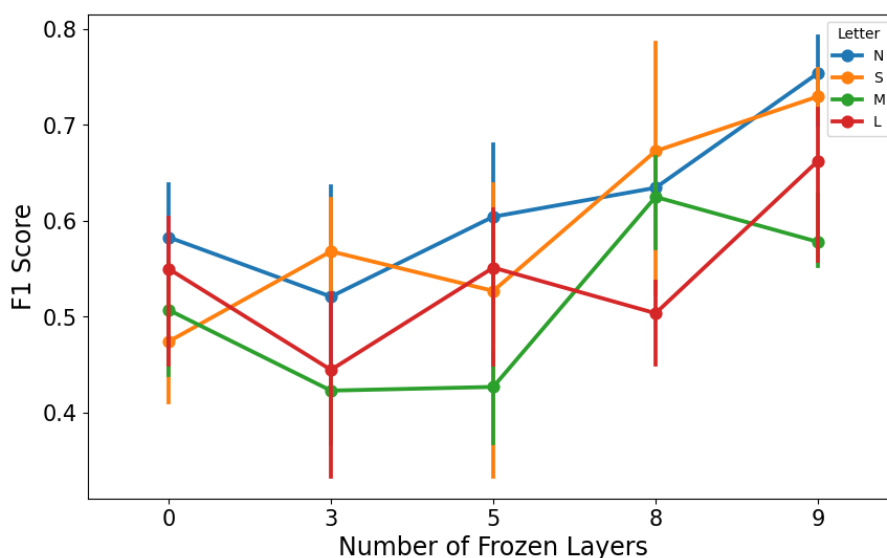


Figure 5.40: Frozen Layers / Pretrained Model [Own Image]

Overall, there is an upward trend in F1 scores as the number of frozen layers increases. The model N, the smallest in terms of parameters, shows a steady improvement from approximately 0.55 to 0.75 as more layers are frozen. Similarly, the model S improves from 0.50 to about 0.75 with 9 frozen layers, suggesting that these smaller models significantly benefit from freezing additional layers.

On the other hand, the model M displays more variable performance. Although it starts with an F1 Score around 0.50, it shows fluctuations across increments in frozen layers, with a notable improvement between 5 and 8 layers, but a drop again with 9 frozen layers. This indicates less consistency in its performance.

The model L, which is the largest, also improves with more frozen layers, reaching about 0.65 with 9 frozen layers, though it shows greater variability. This is evident as the average F1 Score fluctuates up and down depending on the number of frozen layers, not following a clear or monotonic trend but rather the opposite.

In conclusion, freezing more layers particularly benefits models N and S, with consistent im-

provements in the F1 Score. However, for models M and L, although they also improve, the variability in the results indicates the need for a more careful approach when deciding the optimal number of layers to freeze. This nuanced analysis helps to tailor the deep learning approach to specific model architectures, maximizing performance while managing the risks of overfitting or underfitting.

5.3 Contrastive Learning

For this section, let's first recall what the simulation was intended to encompass. The scope of the simulation was to test 24 different architectures (all with the same configuration parameters as it has been demonstrated that this does not significantly impact the final results), diverse in terms of the number of layers and their arrangement. This was to study which architectures perform best as encoders, making it easier to classify. Therefore, it might be interesting to examine the impact of parameters such as the number of convolutions, number of dense layers, type of normalization, type of dropout, type of pooling, and overall architectural configurations, etc.

Once defined and recalled, the simulation can be launched as indicated in its respective section of the solution design. From here, results can be obtained and analyzed.

Firstly, as in previous results sections, the model that has achieved the best performance in predicting on the gold standard set, or what is considered equivalent to maximizing the F1 score on this set, will be presented to see what results it yields.

This model, which achieved an F1 score of 0.66, utilizes 10 convolutions and 7 dense layers. It features 'Partial' pooling, 'Partial' normalization, and 'Full' dropout. At this point, a primary issue with this method of learning becomes apparent—the result was obtained because all samples were classified as belonging to the NOK class. With 60 samples, 30 from one class and 30 from another, it also has a recall of 1 and an accuracy and precision of 0.5. In this case, the network appears to have failed to learn to clearly distinguish between samples from different classes and those from the same class, reflected in the outcome where all samples were predicted as NOK, which is nonsensical.

Given the issue with the model that achieved the highest F1 score, it makes sense to examine how the second-highest scoring model performed.

This model, which achieved an F1 score of 0.6101, features an architecture with 7 convolutional layers and 7 dense layers. It employs 'Full' normalization, 'Full' pooling, and 'None' for dropout, meaning it does not utilize dropout. Among the 50 models generated of this type, the one chosen as the representative of the set for having achieved the best performance is the one generated in the third split of the fifth repetition. Thanks again to the methodology used, traceability of the results obtained by this model on the test set can be maintained. (See Table 5.10)

F1 Score	Recall	Precision	ROC/AUC
0.5461	0.6214	0.3739	0.6312

Table 5.10: CL Metrics Test Set

Overall, the model has achieved fairly mediocre metrics on the test set, where, for example, only 37% of the positive predictions are correct. The recall and ROC/AUC are not bad but still leave much room for improvement.

Now, let's analyze the results obtained by this second model with the highest F1 score on the test set and the gold standard set. To do this, we can use Evidently Cloud to visualize the reports obtained and get a clearer idea of the model's performance. The results are as follows (See Table 5.11 and Figures 5.41, 5.42):

Accuracy	Precision	Recall	F1	ROC AUC
0.616	0.6101	0.6	0.6206	0.6166

Table 5.11: CL Metrics Gold Standard Set [Own Image]

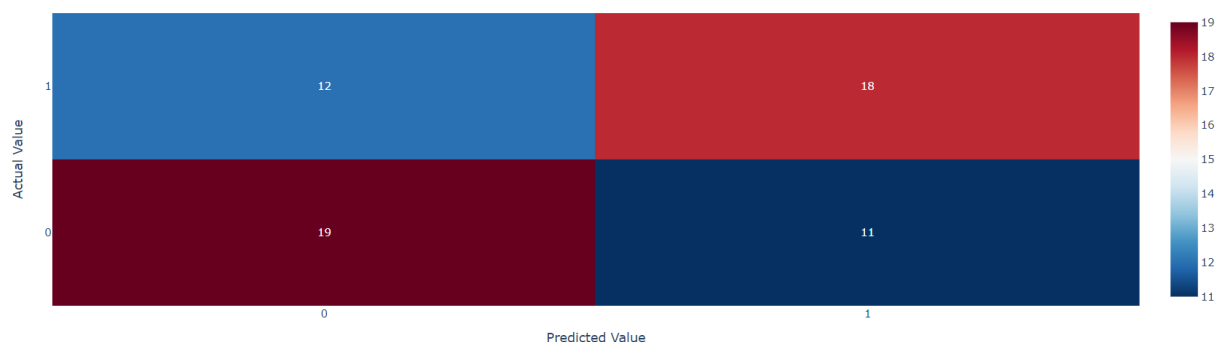


Figure 5.41: CL Gold Confusion Matrix [Own Image]



Figure 5.42: CL Metrics per Class [Own Image]

To begin, let's consider the general metrics of the model. An F1-score of 0.610 indicates that the model has a reasonable balance between precision and recall, suggesting that it is adequately handling both false positives and false negatives. However, a ROC AUC value of 0.617 indicates that there is room for improvement in the model's ability to distinguish between classes, suggesting that the model is not sufficiently robust in all classification situations.

The confusion matrix provides a detailed view of the model's specific errors. In this matrix, we observe that the model correctly classified 18 instances of class OK and 19 of class NOK. However, it also committed 12 false negatives and 11 false positives. These errors indicate that there are a significant number of cases where the model incorrectly predicts classes. The 11 false positives are cases where the model incorrectly predicted class NOK, while the 12 false negatives are instances where it incorrectly predicted class OK.

Regarding metrics by class, these figures indicate that the model performs slightly better on class OK than on class NOK, although the values are quite similar. The similarity in precision and recall values suggests that the model is not biased towards a specific class and manages both classes in a balanced manner.

After analyzing the model that achieved the best performance, we can proceed to study how the parameters that vary between different architectures affect the results. It would be expected that

these trends would be similar to those obtained in the first section of the results, as architectures that performed well there should also serve as effective encoders for siamese networks.

Firstly, we can study the impact of different types of pooling (See Figure 5.44), normalization (See Figure 5.43), and dropout (See Figure 5.45) to see if some clearly perform better than others or not.

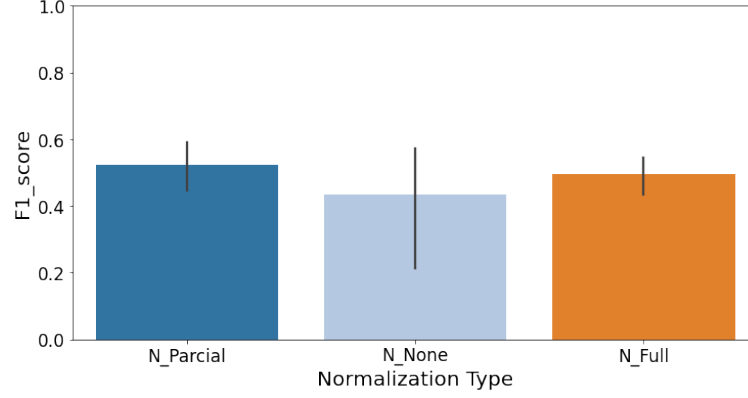


Figure 5.43: CL Normalization Type [Own Image]

For the type of normalization, it can be clearly seen that the 'None' type shows significantly poorer performance, which indicates that normalization is a key aspect that notably helps the model achieve better results, or in other words, not using normalization in the architecture significantly worsens the performance of the model. It is also worth noting that the 'None' type undoubtedly shows the most variability in results. Among the remaining types, 'Partial' appears to function better, though the difference is not substantial. None of the better-performing types reaches an average F1 score of 0.6.

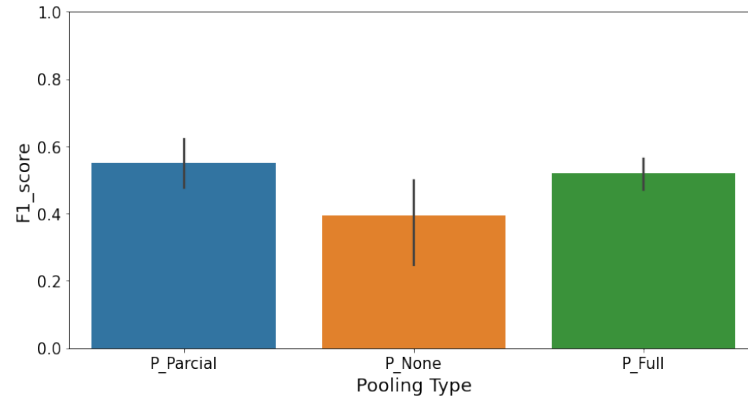


Figure 5.44: CL Pooling Type [Own Image]

For the type of pooling, it can be seen that it follows the same trend as the previous graph, where the 'None' type clearly offers worse results compared to the other two. The 'Partial' type again provides the best performance, being minimally superior to the 'Full' type.

Regarding the type of dropout, here the trend changes. The graph shows that the type of dropout that has achieved the best average performance is 'None'. It is also worth noting that for the simulations of these architectures, the dropout rate passed as a parameter was 0.5. In this case, the next best performing type is 'Partial', followed very closely, with a slight difference, by the 'Full' type. From this, it can be inferred that not turning off neurons when classifying helps the model to do a better classification.

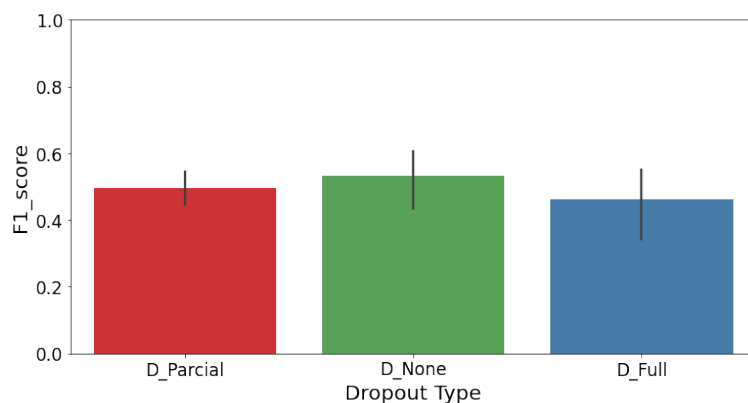


Figure 5.45: CL Dropout Type [Own Image]

Having examined these parameters, we can move on to study the impact of the number of convolutional layers (See Figure 5.46) and dense layers (See Figure 5.47) in the architectures on the results.

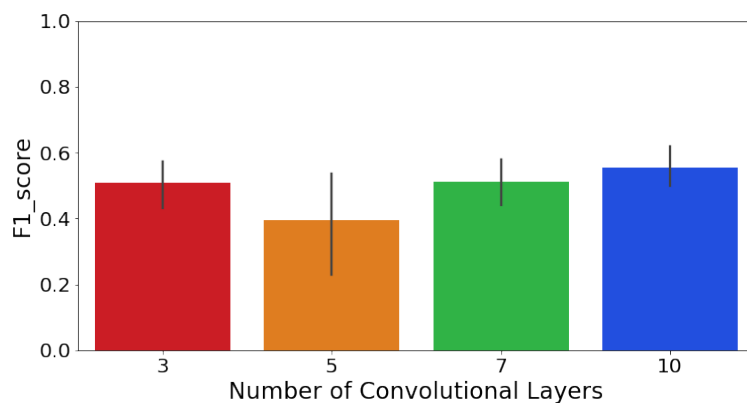


Figure 5.46: CL Convolutions F1 Score [Own Image]

Firstly, in terms of the number of convolutional layers, the graph related to the F1 score shows that this metric starts at around 0.5 for 3 convolutional layers and then decreases for 5 convolutional layers, also scoring around 0.5. From there, as the number of convolutional layers increases, the performance of the model improves, rising to just over 0.5 with 7 convolutional layers and around 0.6 for the highest number of convolutional layers, which is 10. This result shares a commonality with the deep learning results in that the number of convolutional layers again peaks at 10, although here the architectures with 7 layers have performed considerably better. In the rest of the metric graphs, very similar results are obtained that follow the already discussed trend, except for the ROC/AUC, which shows a very similar ability to discriminate between classes across all numbers of convolutional layers, around 0.6.

It is also worth mentioning that the number of layers that shows the greatest variability in results, considerably more than the rest, is 5. This variability could indicate that architectures with 5 convolutional layers may be particularly sensitive to specific data features or training conditions.

Finally, for the number of dense layers in the architectures, the graph clearly shows that the number that has offered the best performance is 7. This number of dense layers results in an F1 score slightly above 0.5, which is not particularly encouraging. The worst performance is offered by 3 dense layers, followed by 11 dense layers. In terms of variability of the results, it is clearly

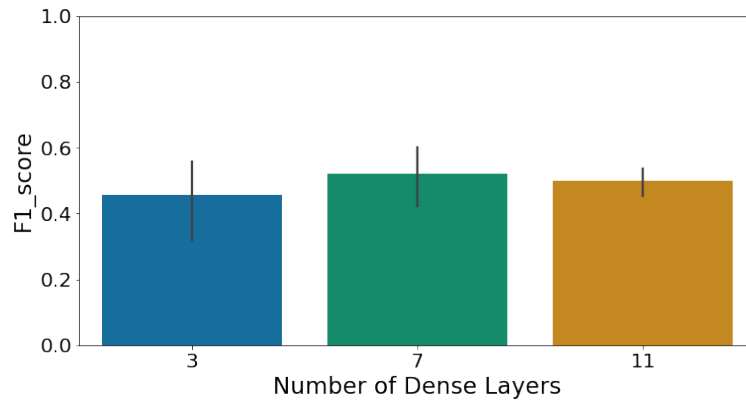


Figure 5.47: CL Dense F1 Score [Own Image]

ordered from left to right or from lesser to greater, depending on how you look at it, with 11 dense layers offering much greater stability than 3.

Once the analysis of the various factors has been completed, a preliminary comparison with the Deep Learning results section can be made. As previously mentioned in the corresponding graph, it is worth highlighting that the number of convolutional layers that performed best in both cases was 10. This suggests that the models require a large number of layers to capture some local features that aid the network in classification.

Regarding the number of dense layers, there is a notable difference. In the first section, 11 dense layers performed best, whereas, in this case, 7 dense layers yielded better results. A smaller number of dense layers in this case achieves better performance than 11. For the type of normalization, 'Partial' again proves to be the most effective, although with generally better results. Concerning the type of pooling, the best performer here is 'Partial,' while in the previous section, it was 'Full.' For the type of dropout, the trend changes again. In the first case, dropout type had little influence on the final results, as shown in the graph, whereas in this case, 'None' clearly outperforms the others.

After completing this general analysis and not obtaining great results, it was decided to test the model's performance with the same architectures simulated with the same parameters, methodology, and everything identical, but with different data. For this second analysis, the decision was made to use the square images generated in the Dataset processing section to train and evaluate the models in this section. The aim was also to study the influence of this rescaling of the images, as preserving the spatial relationship between the different thicknesses of zinc on the coils might make it easier for the networks to capture certain features that were previously local or difficult for the algorithm to perceive.

In this manner, the same 24 architectures were simulated again, and these were the results obtained:

In this case, the best-performing model also had 10 convolutional layers, similar to the previous case, but with only 3 dense layers. This model was obtained in the second split of the first repetition out of the 10 specified in the methodology. Regarding the types of dropout, normalization, and pooling, the 'Partial' type was consistent across all these categories. These were the results obtained by this model, which was chosen as the representative among the 50 models of its architecture after achieving the highest F1 score. (See Table 5.12, Figure 5.48, Figure 5.49)

Accuracy	Precision	Recall	F1	ROC AUC
0.65	0.655	0.633	0.644	0.65

Table 5.12: CL Metrics Gold Standard Set Square Images [Own Image]

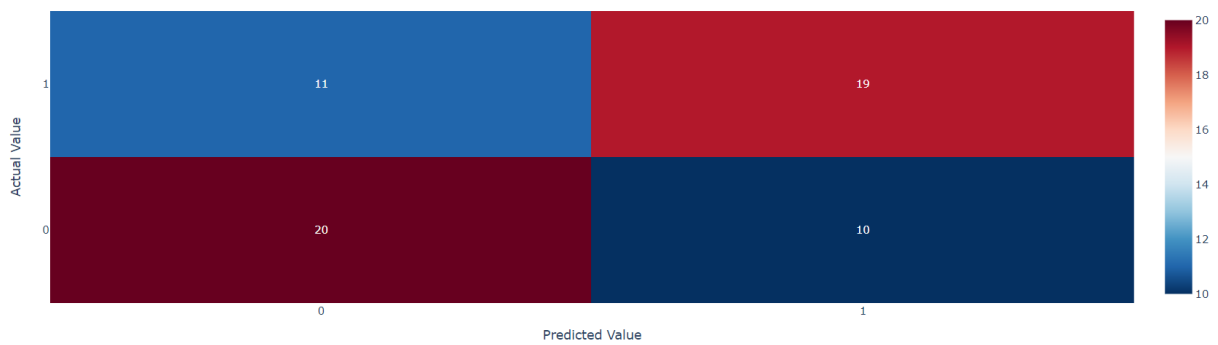


Figure 5.48: CL Gold Confusion Matrix Square Images [Own Image]



Figure 5.49: CL Metrics per Class Square Images [Own Image]

Overall, whether in the global metrics, class-specific metrics, or the confusion matrix, the results are very similar to the previous case where the original data was used. Two more instances were correctly classified compared to the previous case, which is not a significantly notable difference when comparing the models. Therefore, at the moment, the general performances are quite similar concerning the best model obtained from each simulation.

Having reviewed the best model, we will again, and for the last time, study the impact on the results based on the data used to build the models. First, we will visualize how the types of dropout (See Figure 5.52), normalization (See Figure 5.50), and pooling (See Figure 5.51) affect the results.

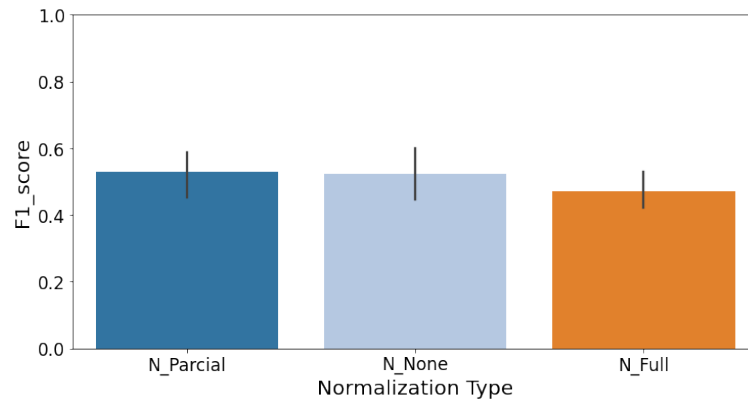


Figure 5.50: CL Normalization Type Square Images [Own Image]

Firstly, regarding normalization, there is an initial difference. In the previous graph, the best type of normalization was 'Partial,' with 'None' being the worst. However, in this case, the best types are these two mentioned, with very similar results, with 'None' shifting from the worst type to being tied as the best along with 'Partial.'

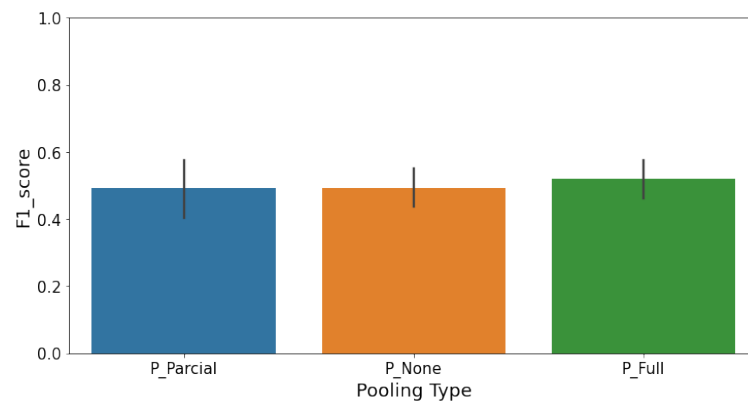


Figure 5.51: CL Pooling Type Square Images [Own Image]

Regarding pooling, the trend changes again, as it becomes quite balanced among the three types, with 'Full' performing slightly better than the other two. In contrast, in the simulation with the original images, the 'None' type of pooling was clearly the worst with much more variability in results, and the 'Partial' type was the best with some margin.

For dropout, there are differences again. The 'Partial' type, which was previously the second best, is now the best, reaching 0.6, with a significant difference compared to the others. The 'None' type for this simulation with squared images seems to have hindered the model's ability to generalize, unlike in the previous case where it helped the model classify better. It is also

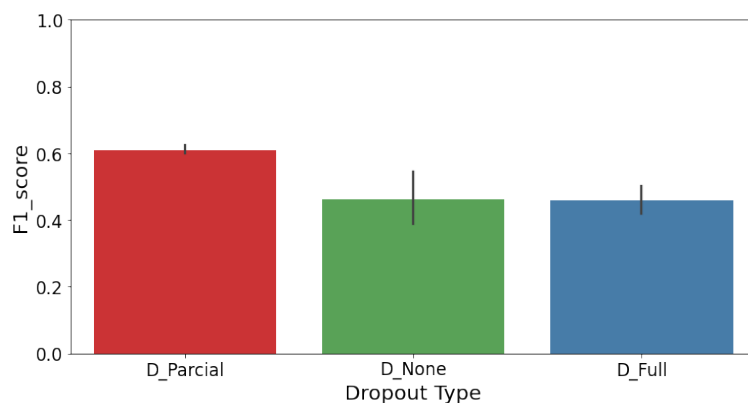


Figure 5.52: CL Dropout Type Square Images [Own Image]

worth noting the minimal variability presented by the 'Partial' type, making it very balanced and robust.

Finally, after examining these parameters, we can study for the last time how the number of dense layers (See Figure 5.53) and convolutional layers (See Figure 5.54) affect the model's performance for these data transformed into square images.

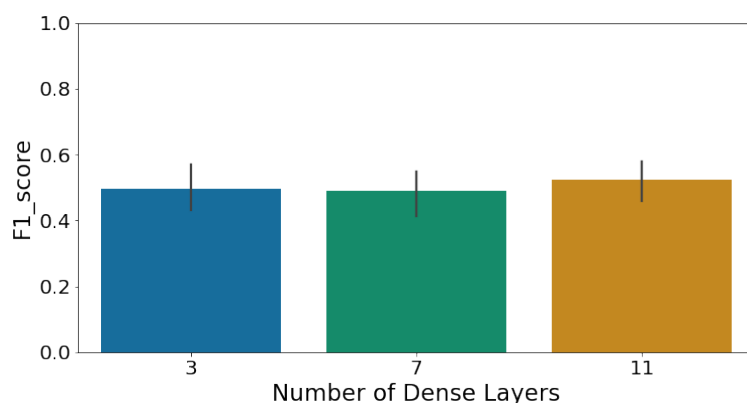


Figure 5.53: CL Dense F1 Score Square Images [Own Image]

Regarding the number of dense layers, the performance is quite balanced among the three amounts of dense layers. Architectures with 11 dense layers seem to classify slightly better, and those with 3 and 7 layers are very well balanced. Here again, there is a difference compared to the previous simulation, where the number of dense layers that achieved the best results was 7. In neither case do the F1 score metrics average above 0.6. In the first section of Deep Learning, it was also the case that architectures with 11 dense layers performed better.

Lastly, let's discuss the number of convolutional layers.

In this graph, we can see the most significant difference between using the original data and using the transformed data. The graph shows that the trend is almost the opposite of the previous section, where the model's performance dropped slightly from 3 to 5 layers and then steadily increased until reaching 10 convolutional layers, where it peaked. In contrast, in this case, the trend is monotonically decreasing as the number of convolutional layers increases, with performance consistently dropping from 3 layers, where the highest average performance was achieved, to 10 layers, where the lowest performance was recorded. In absolute terms, the 3-layer configuration with squared images performed slightly better than the 10-layer configuration with

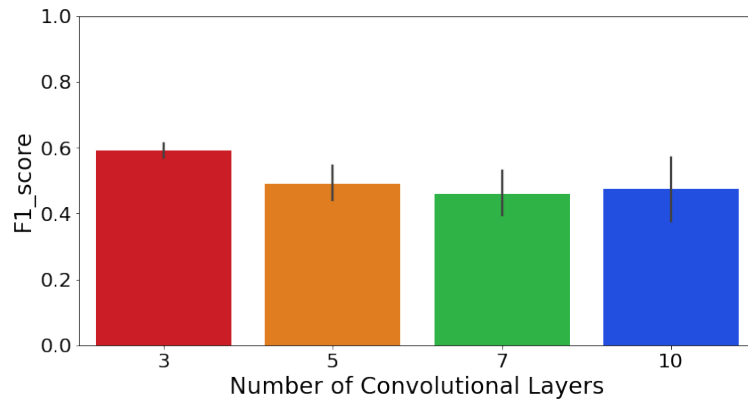


Figure 5.54: CL Convolutions F1 Score Square Images [Own Image]

the original images, reaching an F1 score of 0.6, indicating a good balance between precision and recall.

From this difference, it can be deduced that using squared images, which preserve the spatial relationship between zinc thicknesses, allows the network to more easily capture the features needed for classification in the dense layers, leading to better results. With the original images, it was shown that the network needed many more layers to capture the characteristics or features of each class effectively.

Even though better results were not obtained, the mere fact of achieving very similar results using an architecture with three times fewer convolutional layers—significantly reducing parameters and computational cost—is beneficial. This demonstrates that with a simple data transformation, very similar results can be achieved with much lower computational, economic, and energy costs.

6 Deployment

The deployment of the proposed solution is based on a microservices-oriented architecture, using Apache NiFi™ technology to manage various data flows. The use of NiFi allows for seamless and scalable integration of the different components of the system, ensuring efficient and automated management of quality processes in the production of galvanized steel coils. Here is a detailed explanation of how the system functions, followed by a description of the three main workflows (See Figure 6.1) and how they interact to ensure effective deployment.

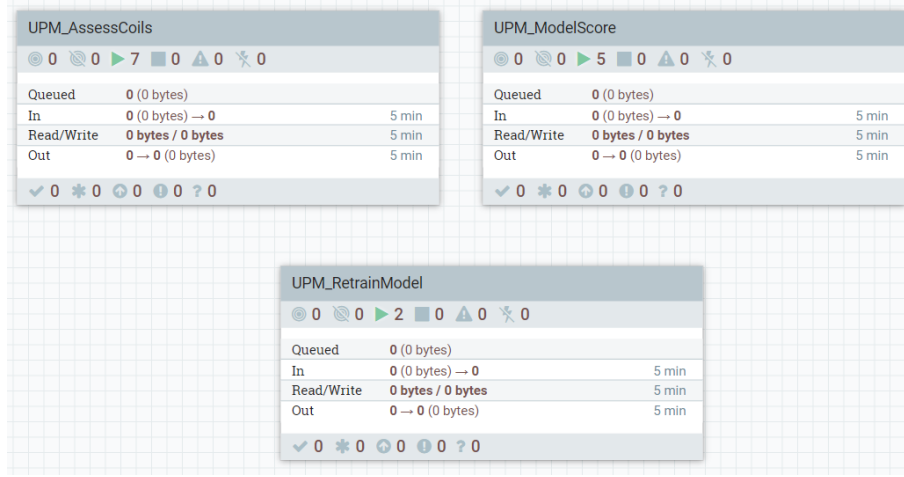


Figure 6.1: Main Workflows [Own Image]

The system is designed to assess the quality of the zinc coating on steel coils in an automated and accurate manner. The infrastructure is deployed in an environment orchestrated by Kubernetes, which allows for scalable and efficient resource management. The essential services required include Python with TensorFlow and PyTorch libraries, Bash, PostgreSQL, and NiFi. This setup ensures that the system can handle large volumes of data and perform precise and rapid evaluations.

6.1 Coil Assessment Workflow

The first workflow, named UPM_AssessCoils, is responsible for assessing the quality of the zinc coating on steel coils. (See Figure 6.2)

The process begins with the reception of data about the coils via MQTT [27] messages, which notify the arrival of new coils for evaluation. This data is then normalized using the `prep_features.py` script, which generates a compressed file with the normalized information. Following this, the `test_cnn-model.py` script evaluates the coil using all available models, recording the classification and confidence of each one. The final decision is made by averaging the classifications and confidences from all models, updating the label column in the `V_Coils` table. Finally, an MQTT message is sent notifying that the coil has been evaluated, allowing users or systems to access the results through the `V_Coils` table.

This workflow ensures a systematic and automated process for evaluating the quality of zinc coatings, leveraging the power of deep learning models to provide reliable and scalable assessments.

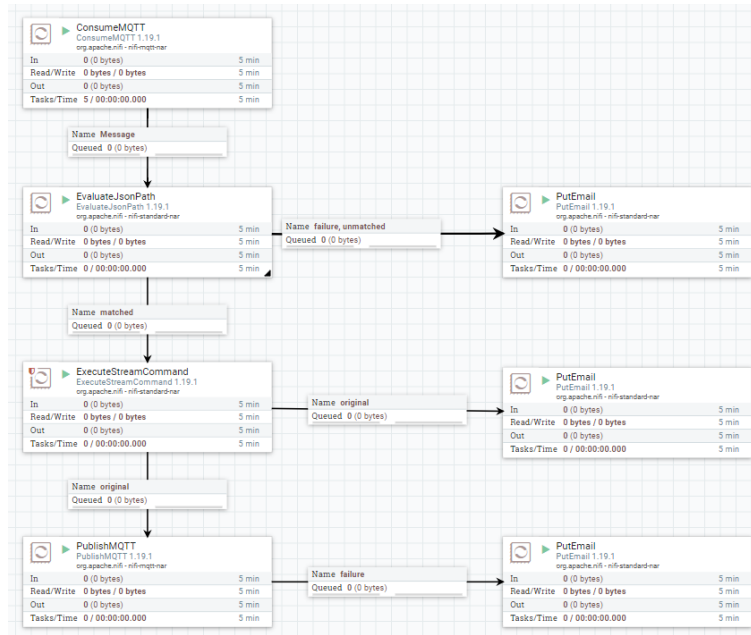


Figure 6.2: Assess Coils Workflow [Own Image]

6.2 Model Score Workflow

The second workflow (See Figure 6.3) focuses on evaluating the classification models to ensure their continuous and accurate performance.

This process starts with the reception of feedback from the operators about the coils, which is stored in the feedback table. The `feedback.cnn-model.py` script analyzes the performance of each model based on this feedback, recording the results in the mperformance table. The evaluated coils receive an update of their label in the V_Coils table, and the data is transferred to the T_V_Coils and T_GR_QPValues tables for future reference and training.

This workflow is crucial for maintaining the integrity and efficacy of the predictive models. By incorporating real-world feedback into the evaluation process, it allows for ongoing adjustments and improvements, ensuring that the models remain robust and effective in predicting the quality of zinc coatings on steel coils.

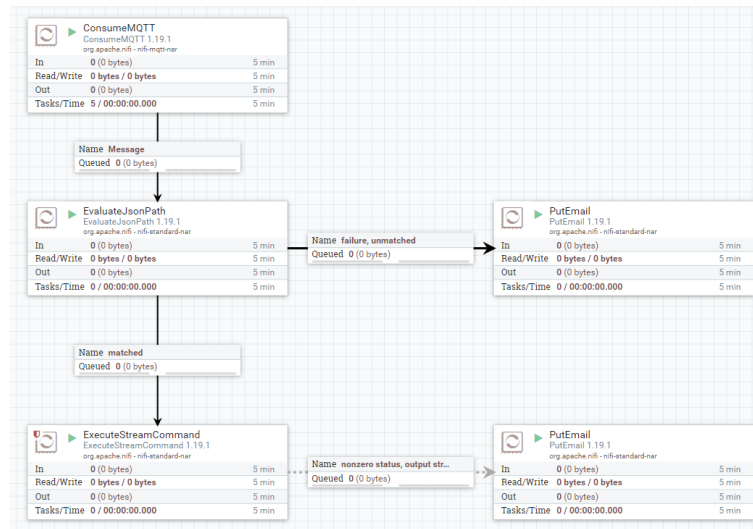


Figure 6.3: Model Score Workflow [Own Image]

6.3 Model Retraining Workflow

The last workflow (See Figure 6.4), known as the model retraining workflow, is activated periodically to identify and retrain models that have shown performance below expectations.

Every week, the system identifies models that have evaluated more than 10 coils with a score lower than 75%. The normalized data are prepared for the training phase, and the `retrain_cc-model.py` script retrains the models, updating the system with the new models and marking the previous ones as invalid.

This process ensures that the classification models remain up-to-date and continue to perform effectively, adjusting to any shifts in data trends or operational conditions that may affect model accuracy. By implementing this regular retraining cycle, the system maintains high standards of performance, thus ensuring the quality and reliability of the evaluations of the zinc coatings on steel coils.

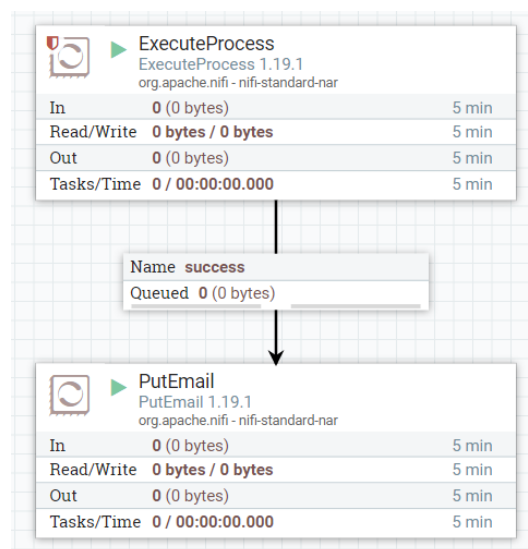


Figure 6.4: Retrain Model Workflow [Own Image]

The system deployment involves several key stages.

First, the infrastructure is set up with the minimum required services and a well-defined folder structure is ensured. Then, the NiFi workflows are configured and the properties of the processors are adjusted to reflect the correct paths and SMTP service configuration.

Next, PostgreSQL tables are created and configured, which are populated with initial data provided by the industrial partner. Afterward, an initial training of a set of models is carried out using the prepared and normalized data. The initial quality of the models is evaluated, and the model table in PostgreSQL is updated.

Finally, the system is continuously monitored and maintained using the NiFi workflows for coil evaluation, model assessment, and model retraining as needed. Additionally, a REST-API service can be implemented to report the system status and query the number of available models and other relevant parameters.

This deployment ensures an automated and efficient evaluation of the quality of galvanized steel coils, integrating both AI capabilities and human feedback to continually improve quality processes in the automotive industry.

7 Impacts of the Project

This section delves into the multifaceted impacts of the project and explores its potential social, economic, and technological benefits. The integration of AI in industrial processes not only transforms production efficiency, but also contributes to broader social advancements and economic growth.

7.1 Social impact

The utilization of artificial intelligence (AI) to improve the steel coil quality rating system in the automotive sector has enormous social implications. In particular, this technology enhances the safety and reliability of cars by ensuring that only high-quality steel is used in their production. This upgrade in product standards directly benefits customers, leading to safer cars with longer lifespans.

Similarly, the application of these modern techniques can reduce instances where manual quality control processes are employed, decreasing employee exposure to dangerous working environments and reducing the chances of human error. In this way, the shift from manual systems to automated ones helps make workplaces safer and more efficient, thereby contributing to worker satisfaction and well-being.

Moreover, in accordance with open-source standards and to benefit society, the project's code has been published on GitHub https://github.com/jordieres/DQ_ACA_2024. This code, excluding data with industrial copyright, includes detailed documentation that explains the project's structure and functionality. This ensures that other researchers and professionals can access and contribute to the continuous development of these technologies, promoting innovation and progress in the sector.

7.2 Economic impact

When it comes to the economy, there are a number of benefits that come with the integration of artificial intelligence-based quality control systems in the automotive industry. Manufacturers can minimize costs related to faulty parts and recalls by improving precision and timing in terms of their quality evaluations. Consequently, this decrease in wastage implies that resources are used more effectively, thus improving overall productivity as well as profitability within manufacturing firms. Also, anticipating faults before they occur reduces downtime, hence ensuring stable output levels while increasing economic efficiency on a whole.

Additionally, by incorporating these modernized technologies into business operations, they may be used in order to bolster global competitiveness of companies. Through AI driven optimization of product quality and operational effectiveness, businesses are able to provide better products at reasonable prices, which attracts more customers, thereby increasing market share. For instance, this advantage is very important because the company is operating within an environment where technology changes frequently while there is an ever present competition all over the world.

7.3 Technological impact

In technological terms, this project illustrates the infusion of artificial intelligence into classical industrial processes, thereby exemplifying a milestone towards Industry 5.0- when humans and machines work together. This represents a major advancement in the use of AI in manufacturing that includes the design and implementation of Convolutional Neural Networks (CNNs) and Deep Transfer Learning (DTL) models for quality assessment. In effect these models not only enhance quality control accuracy but also enable development of advanced autonomous systems with real-time learning capability and adaptation features.

7.4 Contribution to Sustainable Development Goals (SDG)

The project aligns with the Sustainable Development Goals (SDGs) of the United Nations, particularly Goal 8: Decent Work and Economic Growth, and Goal 9: Industry, Innovation, and Infrastructure.

- SDG 8: Decent Work and Economic Growth (See Figure 7.1). It is in line with Goal 8 to have AI-based models used in steel coil quality classification because they facilitate sustained, inclusive and sustainable economic growth through full and productive employment and decent work for all. This project has led to increased productivity and operational efficiency in the automotive industry by automating its quality control processes, thereby improving their accuracy. As such, the increase in firm efficiency entails lower costs and higher output thus contributing towards economic growth. Besides, it decreases instances of manual inspection that exposes workers to risky situations which is like asking everyone to work safely.
- SDG 9: Industry, Innovation, and Infrastructure (See Figure 7.2). The project also contributes to SDG 9 of resilient infrastructure, inclusive and sustainable industrialization, and innovation. The technological leap that has been brought into quality control processes with AI models like CNNs and DTL is huge. This not only brings better quality and consistency in steel products but sets a lead for the adoption of AI in other industrial sectors by promoting sustainable industrialisation. It develops and applies the latest AI technologies, thereby encouraging a culture of innovation within the industry.
- SDG 4: Quality Education (See Figure 7.3). By making the project's code available on GitHub and ensuring comprehensive documentation, this project contributes to SDG 4, which aims to ensure inclusive and equitable quality education and promote lifelong learning opportunities for all. The open-source nature of the project allows students, researchers, and professionals to access, learn from, and build upon the work. This promotes educational growth and skill development in the fields of AI and industrial automation, fostering a knowledgeable and skilled workforce.



Figure 7.1: SDG 8 [54]



Figure 7.2: SDG 9 [55]



Figure 7.3: SDG 4 [56]

8 Planning and Budget

In this section, the planning and budgeting of the project are detailed, covering all phases from conception to final implementation and evaluation.

First, the work plan is presented, which includes a Gantt chart on Figure 8.1 illustrating the main tasks and milestones of the project, as well as their estimated duration. This work plan provides a clear and structured guide on how each stage of the project will be carried out, from initial research and model development to implementation and validation in a production environment.

Next, the budget is detailed on Tables 8.1, 8.2 and 8.3. This includes expenses on hardware and software and human resources. Cost estimation is carried out with the aim of providing a realistic and accurate view of the investment required to successfully complete the project.

8.1 Planning

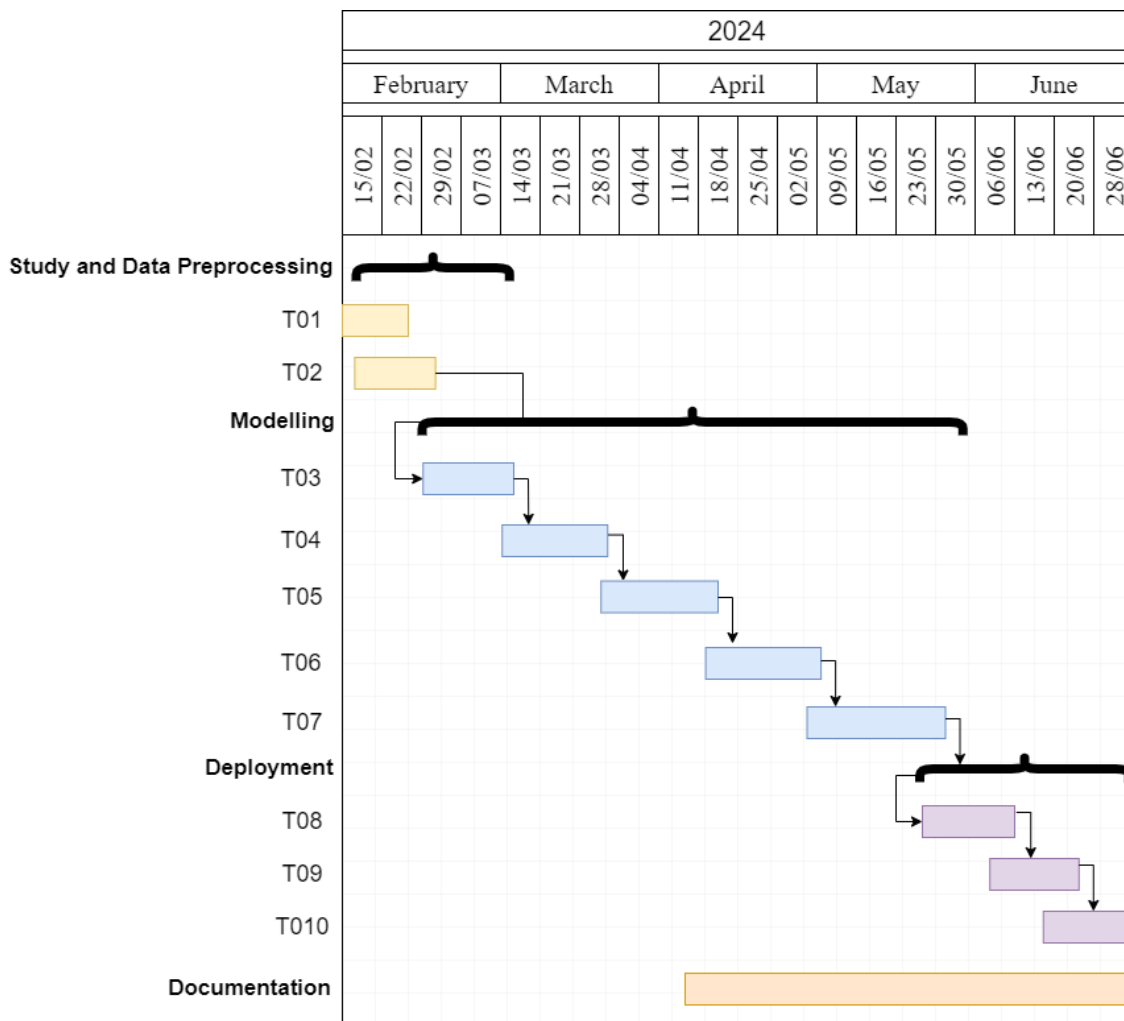


Figure 8.1: Gantt Chart [Own Image]

• Tasks List

1. T01.- Business understanding
2. T02.- To identify the available data and its preprocessing
3. T03.- CNN based models
4. T04.- DTL models based on YOLO images
5. T05.- CL models and performance comparison
6. T06.- Ensemble Learning for Quality Control Assessment of zinc coating
7. T07.- Validation Strategy
8. T08.- Literature review and workflow approach
9. T09.- Implementation of workflow for model assessment for new coils and integration of feedback from operators
10. T010.- Implementation of workflow for model retraining

8.2 Budget

In this section, the costs associated with the project are detailed, including both materials and personnel.

The total cost table is broken down as follows: The Direct Costs amount to 33,610.76 € and include all expenses necessary for the development of the project. To these costs are added the General Costs, which represent 13% of the direct costs, and the Industrial Benefits, which constitute 7% of the direct costs. Finally, VAT (21%) is applied to the subtotal of direct costs, general costs, and industrial benefits, in accordance with Spanish regulations. The resulting Final Budget is 44,078.25 €.

The data engineer salary has been taken from the mean data engineer salary in Spain, 37.500 €/year provided by the job portal Glassdoor [16]

ITEM	PRICE (€)
Windows 11 Home	145 €
IDE Visual Studio Code	0
Python packages listed in the requirements.txt	0
Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz 2.59 GHz, 16,0 GB RAM (student's computer)	1.750
NVIDIA A30 24GB (provided by the department of Ingeniería de Organización, Administración de Empresas y Estadística of the ETSIST)	4.702,35
NVIDIA A100 80GB (provided by the department of Ingeniería de Organización, Administración de Empresas y Estadística of the ETSIST)	19.818,41
TOTAL	24.815,76

Table 8.1: Materials and Costs

ITEM	PRICE (€/hr)	HOURS	SUBTOTAL (€)
Personal Costs	23.5	370	8795
Advisor Costs	50	50	2500
TOTAL			11.295

Table 8.2: Personal Costs

Concept	Amount (€)
Direct Costs	33.610,76
General Costs	4.369,39
Industrial Benefits	2.352,75
VAT (21%)	8469.91
Final Budget	48.802,81

Table 8.3: Total Costs

9 Conclusions and Future Works

9.1 Conclusions

Based on the work developed in this project, several conclusions can be drawn from the results obtained.

Firstly, the effectiveness of the deep learning models implemented is highlighted. In the section dedicated to Deep Learning, it was observed that CNN-based models showed an improvement in classification accuracy compared to traditional methods. The results indicate that models with a higher number of convolutional and dense layers, specifically those with configurations of 10 convolutional layers and 11 dense layers, provided the best balance in terms of accuracy and discriminative power (ROC/AUC). In contrast, shallower architectures generally performed worse as they failed to capture certain critical features necessary for correct predictions.

Regarding Deep Transfer Learning (DTL), it was evidenced that increasing the number of frozen layers in transfer models significantly improves the accuracy and detection capability of smaller models, such as the N and S models. This trend suggests that retaining previously learned features is beneficial for maintaining classification stability. It was demonstrated that the best possible model from this perspective is the N model with 9 frozen layers, utilizing a model with few parameters while maximizing prior learning with the 9 frozen layers, which has a great capacity to generalize. This approach yielded the best results in the study.

Concerning contrastive learning, initially using the original images, it was identified, following the trend of the deep learning results (as expected), that deeper architectures performed better for the reasons explained earlier. The network requires more convolutions to capture the necessary features for classification. It can also be concluded that the use of dropout is highly detrimental to this contrastive learning approach with the original data.

On the other hand, using the transformed data, other interesting conclusions, which were hypothesized during the project, could be confirmed. As demonstrated in the second part of the contrastive learning results section, using squared images reverses the trend in the graph of the number of convolutions, as the best-performing architectures were those with 3 convolutional layers, i.e., less deep and complex networks.

This finding is consistent with the observations in the transfer learning section, where, using squared images, the models that performed best were those utilizing fewer parameters and were less complex.

Therefore, based on the combined results of the three sections, it can be concluded that depending on the data being used and their structure, different architectures will perform better. For the original images sized 264x18, deeper architectures with more parameters perform better, while for the squared images of 264x264 generated as explained in the Dataset processing section, shallower architectures with fewer parameters perform better.

Yo diría “ Additionally, considering the discrepancy between the predictions on the test set and those on the gold standard set, it is evident that more data is required to further enhance the models. Across all the techniques employed (DL, DTL, and CL), it is clear that there is significant room for improvement by expanding the dataset from which learning occurs.”

It was also evidenced the flexibility provided by the created software tool as a framework to run a bunch of different models and being able to study the impact of the different parameters

and configurations. In particular the list of attributes implemented in the configuration files “*.yaml” proved being extremely powerful in this sense.

To put dynamically the results of the experiments in the online platforms such as evidentlyai.com was found extremely useful, although they need to improve the management capabilities in filtering the results to create dashboards and graphs.

9.2 Future Works

Regarding future results, this is a broad and extensive field in which to work. However, some future work that could be interesting and add value to the sector of quality control based on AI models could include:

- Improving and expanding the data augmentation techniques: As seen in the conclusions, the gap between the results on the test set and the gold standard set indicates that the data augmentation does not provide enough diversity to meet the learning needs. Implementing new data augmentation methods or improving existing ones could substantially enhance the results obtained.
- More models simulation: Thanks to the system’s generality, it allows for the simulation of models with desired specifications in both architecture and configuration parameters with minimal effort.

9.3 Personal Learning

This project has been immensely beneficial for both my academic and professional development and it can be seen in various aspects.

Academic Application: Firstly, this project provided an opportunity to implement Deep Learning techniques studied in class within a real system. This aspect has been particularly motivating and interesting, as many of these techniques often remain theoretical concepts that seldom leave the paper or a slide presentation. Therefore, this project has successfully sparked my interest and motivation in areas related to machine learning, computer vision techniques, artificial intelligence, etc. These fields are often described as the future but are, in fact, already the present.

Professional Experience: Secondly, undertaking a personal project in which I focused for several months has been an enriching experience as a first step towards something similar to the real world. This extended engagement allowed me to understand the complexities and challenges involved in long-term projects, including planning, execution, and troubleshooting.

Autonomous Learning: Lastly, but no less importantly, is the ability to seek information and resources independently to develop a project. I believe this is essential, encompassing the capability to be self-sufficient in finding information, resources, articles, code, or technologies that may be useful and appropriate for developing a project. These are crucial aspects of the professional skills required in the technology field, and through this project, I have significantly reinforced them.

This project has not only enhanced my technical skills but also provided valuable insights into project management and self-directed learning, preparing me for future challenges in the tech

industry.

References

- [1] C. Ahner, P. Maaß, and P. Peißker, *Handbook of hot-dip galvanization*. John Wiley & Sons, 2011.
- [2] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala, “PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation,” in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS ’24)*, ACM, Apr. 2024. DOI: 10.1145/3620665.3640366. [Online]. Available: <https://pytorch.org/assets/pytorch2-2.pdf>.
- [3] F. Artkin, *Evaluation of the industrial applications of galvanized coating*, 2023.
- [4] H. Asgari, M. Toroghinejad, and M. Golozar, “Effect of coating thickness on modifying the texture and corrosion performance of hot-dip galvanized coatings,” *Current Applied Physics*, vol. 9, no. 1, pp. 59–66, 2009.
- [5] C. F. Barbosa, “A simple formula for calculating the coating thickness of galvanized steel wires,” *IEEE Transactions on Magnetics*, vol. 50, no. 7, pp. 1–7, 2014.
- [6] e. a. Beck K., “Extreme programming explained: Embrace change.,” *Addison-Wesley Professional.*, vol. 156, pp. 50–61, May 2019.
- [7] Y. Bengio, I. Goodfellow, and A. Courville, *Deep learning*. MIT press Cambridge, MA, USA, 2017, vol. 1.
- [8] D. Berrar *et al.*, *Cross-validation*. 2019.
- [9] N. K. Chauhan and K. Singh, “A review on conventional machine learning vs deep learning,” in *2018 International conference on computing, power and communication technologies (GUCON)*, IEEE, 2018, pp. 347–352.
- [10] D. Chicco, “Siamese neural networks: An overview,” *Artificial Neural Networks*, pp. 73–94, 2021.
- [11] F. Chollet *et al.*, *Keras*, <https://keras.io>, 2015.
- [12] I. Coicoechea and M. Bolibar, “Quality management in the automotive industry,” *DAAAM International Scientific Book*, pp. 619–632, Jan. 2012. DOI: 10.2507/daaam.scibook.2012.51.
- [13] J. R. Demey, L. Pla, J. L. Vicente-Villardón, J. Di Rienzo, and F. Casanoves, “Medidas de distancia y similitud,” *Valoración y análisis de la diversidad funcional y su relación con los servicios ecosistémicos*, vol. 384, pp. 47–59, 2011.

- [14] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, Ieee, 2009, pp. 248–255.
- [15] T. Developers, *Tensorflow*, version v2.15.1, Mar. 2024. DOI: 10.5281/zenodo.10798587. [Online]. Available: <https://doi.org/10.5281/zenodo.10798587>.
- [16] G. Door, *Glass door*, 10/07/2024. [Online]. Available: <https://www.glassdoor.es/>.
- [17] A. A. Elngar, M. Arafa, A. Fathy, B. Moustafa, O. Mahmoud, M. Shaban, and N. Fawzy, "Image classification based on cnn: A survey," *Journal of Cybersecurity and Information Management*, vol. 6, no. 1, pp. 18–50, 2021.
- [18] Evidently AI, *Confusion Matrix*, Evidently AI. [Online]. Available: <https://www.evidentlyai.com/classification-metrics/confusion-matrix>.
- [19] T. Gong, X. Yuan, X. Lei, Y. Yuan, and B. Zhang, "Fault detection for rolling element bearing based on repeated single-scale morphology and simplified sensitive factor algorithm," *Measurement*, vol. 127, pp. 348–355, 2018.
- [20] S. S. Gutlapalli, "The role of deep learning in the fourth industrial revolution: A digital transformation approach," *Asian Accounting and Auditing Advancement*, vol. 8, no. 1, pp. 52–56, 2017.
- [21] G. S. Handelman, H. K. Kok, R. V. Chandra, A. H. Razavi, S. Huang, M. Brooks, M. J. Lee, and H. Asadi, "Peering into the black box of artificial intelligence: Evaluation metrics of machine learning methods," *American Journal of Roentgenology*, vol. 212, no. 1, pp. 38–43, 2019.
- [22] B. P. Harenslak and J. de Ruiter, *Data pipelines with apache airflow*. Simon and Schuster, 2021.
- [23] e. a. He K., "Deep learning-based surface defect detection for steel coils.," *Journal of Materials Processing Technology*, vol. 274, pp. 116–123, May 2019.
- [24] M. Heidari and K. Fouladi-Ghaleh, "Using siamese networks with transfer learning for face recognition on small-samples datasets," in *2020 International Conference on Machine Vision and Image Processing (MVIP)*, IEEE, 2020, pp. 1–4.
- [25] K. Henryk and S. Mariola, "Benefits and limitations of the use of pb, sn and bi alloying elements in hot dip galvanizing bath: A review," *Journal of Materials Engineering and Performance*, vol. 32, no. 13, pp. 5680–5688, 2023.
- [26] A. Hernández-García and P. König, "Further advantages of data augmentation on convolutional neural networks," in *Artificial Neural Networks and Machine Learning-ICANN 2018: 27th International Conference on Artificial Neural Networks, Rhodes, Greece, October 4-7, 2018, Proceedings, Part I 27*, Springer, 2018, pp. 95–103.
- [27] G. C. Hillar, *MQTT Essentials-A lightweight IoT protocol*. Packt Publishing Ltd, 2017.

- [28] Hito Engineering, *Hot Dip Galvanizing Plant for Gi/Gl Coils Galvalume Line (CGL)*, Made-in-China.com. [Online]. Available: <https://hitoengineering.en.made-in-china.com/product/FZyaiWhKZJtn/China-Hot-Dip-Galvanizing-Plant-for-Gi-Gl-Coils-Galvalume-Line-Cgl.html>.
- [29] F.-H. Hsu, *Behind Deep Blue: Building the computer that defeated the world chess champion*. Princeton University Press, 2002.
- [30] X. Hu, J. Yang, F. Jiang, A. Hussain, K. Dashtipour, and M. Gogate, “Steel surface defect detection based on self-supervised contrastive representation learning with matching metric,” *Applied Soft Computing*, vol. 145, p. 110578, 2023.
- [31] M. Hussain, “Yolo-v1 to yolo-v8, the rise of yolo and its complementary nature toward digital manufacturing and industrial defect detection,” *Machines*, vol. 11, no. 7, 2023, ISSN: 2075-1702. DOI: 10.3390/machines11070677. [Online]. Available: <https://www.mdpi.com/2075-1702/11/7/677>.
- [32] L. N. Huynh, Y. Lee, and R. K. Balan, “Deepmon: Mobile gpu-based deep learning framework for continuous vision applications,” in *Proceedings of the 15th annual international conference on mobile systems, applications, and services*, 2017, pp. 82–95.
- [33] Y. Ioannou, D. Robertson, R. Cipolla, and A. Criminisi, “Deep roots: Improving cnn efficiency with hierarchical filter groups,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 1231–1240.
- [34] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics YOLO*, version 8.0.0, Jan. 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [35] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics yolov8*, version 8.0.0, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [36] K. Kaur, R. Dhir, and K. Kumar, “Transfer learning approach for analysis of epochs on handwritten digit classification,” in *2021 2nd International Conference on Secure Cyber Computing and Communications (ICSCCC)*, 2021, pp. 456–458. DOI: 10.1109/ICSCCC51823.2021.9478102.
- [37] S.-S. Kim, W.-R. Lee, and J.-H. Go, “A study on utilization of spatial information in heterogeneous system based on apache nifi,” in *2019 International Conference on Information and Communication Technology Convergence (ICTC)*, IEEE, 2019, pp. 1117–1119.
- [38] T. K. Kim, “T test as a parametric statistic,” *Korean Journal of Anesthesiology*, vol. 68, no. 6, p. 540, 2015.
- [39] V. Kuklik, “Post on the issue of safety of steel structures of hot dip galvanized structural components,” *Procedia Engineering*, vol. 40, pp. 241–246, 2012.
- [40] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [41] J. Lee, H. Davari, J. Singh, and V. Pandhare, “Industrial artificial intelligence for industry 4.0-based manufacturing systems,” *Manufacturing Letters*, vol. 18, pp. 20–23, 2018.

- [42] J. Leukel, J. Gonzalez, and M. Riekert, "Adoption of machine learning technology for failure prediction in industrial maintenance: A systematic review," *Journal of Manufacturing Systems*, vol. 61, pp. 87–96, Oct. 2021. DOI: 10.1016/j.jmsy.2021.08.012.
- [43] C. Li, Y. Chen, and Y. Shang, "A review of industrial big data for decision making in intelligent manufacturing," *Engineering Science and Technology, an International Journal*, vol. 29, p. 101 021, 2022, ISSN: 2215-0986. DOI: <https://doi.org/10.1016/j.jestch.2021.06.001>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2215098621001336>.
- [44] U. Lichtenthaler, "Substitute or synthesis: The interplay between human and artificial intelligence," *Research-Technology Management*, vol. 61, no. 5, pp. 12–14, 2018.
- [45] U. K. Lilhore, S. Simaiya, A. Kaur, D. Prasad, M. Khurana, D. K. Verma, and A. Hassan, "Impact of deep learning and machine learning in industry 4.0: Impact of deep learning," in *Cyber-Physical, IoT, and Autonomous Systems in Industry 4.0*, CRC Press, 2021, pp. 179–197.
- [46] e. a. Liu Y., "Real-time monitoring and decision-making in industrial processes using machine learning," *Journal of Manufacturing Processes*, vol. 56, pp. 541–552, May 2020.
- [47] I. A. Lungu, A. Aimar, Y. Hu, T. Delbruck, and S.-C. Liu, "Siamese networks for few-shot learning on edge embedded devices," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 10, no. 4, pp. 488–497, 2020.
- [48] A. Mallett and A. Mallett, "Writing yaml and basic playbooks," *Red Hat Certified Engineer (RHCE) Study Guide: Ansible Automation for the Red Hat Enterprise Linux 8 Exam (EX294)*, pp. 63–77, 2021.
- [49] J. McCarthy, M. L. Minsky, N. Rochester, and C. E. Shannon, "A proposal for the dartmouth summer research project on artificial intelligence, august 31, 1955," *AI Magazine*, vol. 27, no. 4, pp. 12–12, 2006.
- [50] S. Mirampalli, R. Wankar, and S. N. Srirama, "Evaluating nifi and mqtt based serverless data pipelines in fog computing environments," *Future Generation Computer Systems*, vol. 150, pp. 341–353, 2024.
- [51] S. Mo, Z. Sun, and C. Li, "Siamese prototypical contrastive learning," *arXiv preprint arXiv:2208.08819*, 2022.
- [52] R. Mordia and A. Verma, "Detection of rail defects caused by fatigue due to train axles using machine learning," *Transportation Infrastructure Geotechnology*, May 2024. DOI: 10.1007/s40515-024-00418-2.
- [53] H. D. Nguyen, P. H. Tran, T. H. Do, and K. P. Tran, "Quality control for smart manufacturing in industry 5.0," in *Artificial Intelligence for Smart Manufacturing: Methods, Applications, and Challenges*, Springer, 2023, pp. 35–64.
- [54] Our World in Data, *Economic Growth*, Our World in Data. [Online]. Available: <https://ourworldindata.org/sdgs/economic-growth>.

- [55] Our World in Data, *Industry, Innovation, and Infrastructure*, Our World in Data. [Online]. Available: <https://ourworldindata.org/sdgs/industry-innovation-infrastructure>.
- [56] Our World in Data, *Quality Education*, Our World in Data. [Online]. Available: <https://ourworldindata.org/sdgs/quality-education>.
- [57] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2009.
- [58] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010. DOI: 10.1109/TKDE.2009.191.
- [59] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshein, L. Antiga, *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.
- [60] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [61] D.-L. Pham, T.-W. Chang, *et al.*, "A yolo-based real-time packaging defect detection system," *Procedia Computer Science*, vol. 217, pp. 886–894, 2023.
- [62] O. Rainio, J. Teuho, and R. Klén, "Evaluation metrics and statistical tests for machine learning," *Scientific Reports*, vol. 14, no. 1, p. 6086, 2024.
- [63] D. S. Rani, L. R. Burra, G. Kalyani, B. Rao, *et al.*, "Edge intelligence with light weight cnn model for surface defect detection in manufacturing industry," *Journal of Scientific & Industrial Research*, vol. 82, no. 02, pp. 178–184, 2023.
- [64] S. Raschka, Y. H. Liu, V. Mirjalili, and D. Dzhulgakov, *Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python*. Packt Publishing Ltd, 2022.
- [65] C. Ren, D.-K. Kim, and D. Jeong, "A survey of deep learning in agriculture: Techniques and their applications," *Journal of Information Processing Systems*, vol. 16, no. 5, pp. 1015–1033, 2020.
- [66] K. Rivers, E. Harpstead, and K. R. Koedinger, "Learning curve analysis for programming: Which concepts do students struggle with?" In *ICER*, ACM, vol. 16, 2016, pp. 143–151.
- [67] M. Romero, Y. Interian, T. Solberg, and G. Valdes, "Targeted transfer learning to improve performance in small medical physics datasets," *Medical Physics*, vol. 47, no. 12, pp. 6246–6256, 2020.
- [68] S. J. Russell and P. Norvig, *Artificial intelligence: a modern approach*. Pearson, 2016.
- [69] S. Sakib, Ahmed, A. Jawad, J. Kabir, and H. Ahmed, "An overview of convolutional neural network: Its architecture and applications," Nov. 2018. DOI: 10.20944/preprints201811.0546.v1.

- [70] R. C. Schank, "What is ai, anyway?" *AI Magazine*, vol. 8, no. 4, pp. 59–59, 1987.
- [71] C. Schröer, F. Kruse, and J. M. Gómez, "A systematic literature review on applying crisp-dm process model," *Procedia Computer Science*, vol. 181, pp. 526–534, 2021.
- [72] B. B. Schultz, "Levene's test for relative variation," *Systematic Zoology*, vol. 34, no. 4, pp. 449–456, 1985.
- [73] J. Senoner, T. Netland, and S. Feuerriegel, "Using explainable artificial intelligence to improve process quality: Evidence from semiconductor manufacturing," *Management Science*, vol. 68, no. 8, pp. 5704–5723, 2022.
- [74] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, no. 1, pp. 1–48, 2019.
- [75] C. Tan, F. Sun, T. Kong, W. Zhang, C. Yang, and C. Liu, "A survey on deep transfer learning," in *Artificial Neural Networks and Machine Learning–ICANN 2018: 27th International Conference on Artificial Neural Networks, Rhodes, Greece, October 4–7, 2018, Proceedings, Part III 27*, Springer, 2018, pp. 270–279.
- [76] T. pandas development team, *Pandas-dev/pandas: Pandas*, version latest, Feb. 2020. DOI: 10.5281/zenodo.3509134. [Online]. Available: <https://doi.org/10.5281/zenodo.3509134>.
- [77] R. Teti, K. Jemielniak, G. O'Donnell, and D. Dornfeld, "Advanced monitoring of machining operations," *CIRP Annals*, vol. 59, no. 2, pp. 717–739, 2010.
- [78] e. a. Thomas R., "Agile practices in data science: An empirical study," *Journal of Systems and Software*, vol. 156, pp. 50–61, May 2019.
- [79] S. Uchida, S. Ide, B. K. Iwana, and A. Zhu, "A further step to perfect accuracy by training cnn with larger data," in *2016 15th International Conference on Frontiers in Handwriting Recognition (ICFHR)*, 2016, pp. 405–410. DOI: 10.1109/ICFHR.2016.0082.
- [80] Ultralytics, *Models - Classify*, Ultralytics Documentation. [Online]. Available: <https://docs.ultralytics.com/tasks/classify/#models>.
- [81] Ultralytics, *YOLOv8 - Ultralytics Models*, Ultralytics Documentation. [Online]. Available: <https://docs.ultralytics.com/models/yolov8/>.
- [82] R. Vallat, "Pingouin: Statistics in python," *The Journal of Open Source Software*, vol. 3, no. 31, p. 1026, Nov. 2018.
- [83] J. VanderPlas, *Python Data Science Handbook: Essential tools for working with data.* " O'Reilly Media, Inc.", 2016.
- [84] G. Vrbančič and V. Podgorelec, "Transfer learning with adaptive fine-tuning," *IEEE Access*, vol. 8, pp. 196 197–196 211, 2020.
- [85] A. Wang, "An industrial strength audio search algorithm," in *ISMIR 2003, 4th Symposium Conference on Music Information Retrieval*, 2003, pp. 7–13.

- [86] B. Wang, R. Ma, J. Kuang, and Y. Zhang, “How decisions are made in brains: Unpack “black box” of cnn with ms. pac-man video game,” *IEEE Access*, vol. 8, pp. 142 446–142 458, 2020.
- [87] Q. Wang, Y. Ma, K. Zhao, and Y. Tian, “A comprehensive survey of loss functions in machine learning,” *Annals of Data Science*, pp. 1–26, 2020.
- [88] Y. Wang, F. Li, H. Sun, W. Li, C. Zhong, X. Wu, H. Wang, and P. Wang, “Improvement of mnist image recognition based on cnn,” *IOP Conference Series: Earth and Environmental Science*, vol. 428, no. 1, p. 012 097, Jan. 2020. DOI: 10.1088/1755-1315/428/1/012097. [Online]. Available: <https://dx.doi.org/10.1088/1755-1315/428/1/012097>.
- [89] Wikipedia contributors, *Cross Industry Standard Process for Data Mining*, Wikipedia, The Free Encyclopedia, 2024. [Online]. Available: https://es.wikipedia.org/wiki/Cross_Industry_Standard_Process_for_Data_Mini.
- [90] X. Xu, Y. Lu, B. Vogel-Heuser, and L. Wang, “Industry 4.0 and industry 5.0—inception, conception and perception,” *Journal of Manufacturing Systems*, vol. 61, pp. 530–535, 2021.
- [91] M. Zabin, A. N. B. Kabir, M. K. Kabir, H.-J. Choi, and J. Uddin, “Contrastive self-supervised representation learning framework for metal surface defect detection,” *Journal of Big Data*, vol. 10, no. 1, p. 145, 2023.
- [92] Y. Zhang, S. Ren, Y. Liu, and S. Si, “A big data analytics architecture for cleaner manufacturing and maintenance processes of complex products,” *Journal of Cleaner Production*, vol. 142, pp. 626–641, 2017.